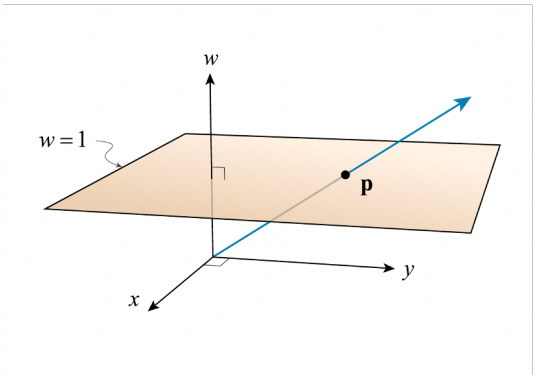
Geometric Algebra (GA)

Daniel Hug

2025-2026

- *https://github.com/Daniel-G-W-Hug/ga/tree/develop*
  (The Geometric Algebra C++ Template Library including Python and Lua wrappers)
- *https://github.com/Daniel-G-W-Hug/ga/blob/develop/ga_docu/0_ga_docu.pdf*
  (This document)



Projective point. Source: „Foundations of Projective Geometric Algebra“, presentation Eric Lengyel, 05/2024

Contents

1 GA library

1.1 Introduction to the ga library

The ga library is intended to make Geometric Algebra (GA) more accessible by providing functionality for numerical calculations required for simulation of physical systems.

It implements functionality for two- and three-dimensional spaces with Euclidean geometry. For this purpose, it provides data types and operations to handle the algebras of two- and three-dimensional Euclidean space ($EGA2D$, $EGA3D$) as well as for projective two- and three-dimensional Euclidean space ($PGA2DP$, $PGA3DP$). The latter two are modelled in a three- and four-dimensional space, respectively, since both include an additional projective dimension. The library also implements four-dimensional spacetime algebra ($STA4DS$).

For the ga library we use linear spaces $\mathbb{G}(p, n, z)$, where the arguments signify: p = number of basis vectors squaring to +1, n = number of basis vectors squaring to -1, and z = number of basis vectors squaring to 0. The linear space $\mathbb{G}$ is an extension of a linear vector space to a linear space of multivectors in the respective basis (for details see [Macdonald, 2010]).

- $EGA2D/3D$: $\mathbb{G}(2,0,0)/\mathbb{G}(3,0,0)$ (extends $\mathbb{R}^2/\mathbb{R}^3$)
- $PGA2DP/3DP$: $\mathbb{G}(2,0,1)/\mathbb{G}(3,0,1)$ (projective geometry)
- $STA4DS$: $\mathbb{G}(1,3,0)$ (Spacetime algebra with signature (-1,-1,-1,+1))

The library efficiently handles fully populated multivectors, even- and odd-grade multivectors as well as their components like scalars, vectors, bivectors, trivectors, and pseudoscalars of the corresponding linear spaces. It also provides operations to work with these objects. It was designed with extensibility in mind and could be extended to provide functionality for handling additional algebras with limited effort.

The core implementation language of the library is C++. The documentation is focused on the C++ implementation exclusively. However, the library provides wrappers for python (3.10+) in `ga_py` that is fully generated from the C++ implementation using nanobind, i.e. it covers almost 100% of C++ data types and functionality. There also is a wrapper for lua in `ga_lua` (manually generated, with some remaining gaps in coverage of C++ functionality).

The main library is a header-only library in the `ga` sub-folder, which can be used by including either

```
#include ga/ga_ega.hpp // and/or
#include ga/ga_pga.hpp
#include ga/ga_sta.hpp
```

and by making the corresponding namespaces accessible by stating

```
using namespace hd::ga;      // and
using namespace hd::ga::ega; // either/or
using namespace hd::ga::pga;
using namespace hd::ga::sta;
```

in your application code. For a simple usage example please have a look either at `ga/ga_test/src/ga_ega_test.cpp` or similar.

1.1.1 Data types – user abstractions for each algebra

At the heart of the library there are user defined types that model mathematical objects of the respective geometric algebra. They are realized using type tagging in C++ to maximize reuse and to avoid code duplication. All types are defined in `ga/detail/type_t`.

To give an example for $EGA2D$, there are type aliases defined in the file `ga/detail/type_t/type2d.hpp` which provide types for scalars, vectors, pseudoscalars, even-grade multivectors and full multivectors with the corresponding number of components. We have to distinguish between scalars and pseudoscalars at type level in order to allow for complement operations (defined later on) that can distinguish those types for each algebra. Following types are provided:

```
namespace hd::ga {

template <typename T> using Scalar2d
                           = Scalar_t<T,scalar2d_tag>;
template <typename T> using Vec2d = Vec2_t<T,vec2d_tag>;
template <typename T> using PScalar2d
                           = Scalar_t<T,pscalar2d_tag>;
template <typename T> using MVec2d_E = MVec2_t<T,mvec2d_e_tag>;
template <typename T> using MVec2d = MVec4_t<T,mvec2d_tag>;

} // namespace hd::ga
```

Based on these template aliases types like `Vec2d<T>` can be used directly. However, for increased user convenience additional template specializations have been defined in `ga/ga_usr_types.hpp` as

```
using scalar2d = Scalar2d<value_t>;
using vec2d = Vec2d<value_t>;
using pscalar2d = PScalar2d<value_t>;
using mvec2d_e = MVec2d_E<value_t>;
using mvec2d = MVec2d<value_t>;
```

where `value_t` is defined for the whole library in `ga/value_t.hpp`. Thus, the user can simply select `value_t=double` (set as default value) or `float` for the library and the user application at one location. There is no need to use template parameters at all, if the user-defined types like `scalar2d, vec2d,pscalar2d,mvec2d_e`, and `mvec2d` are used for the application.

All algebras follow the same pattern. Just to give a second example for $PGA3DP$ following user types are defined: `scalar3dp`, `vec3dp`, `bivec3dp`, `trivec3dp`, `pscalar3dp`, `mvec3dp_e`, `mvec3dp_u`, and `mvec3dp`, where we additionally have defined types for bivectors, trivectors and uneven-grade multivectors, as required to completely implement the algebra.

To work with the user defined algebraic objects, the operations documented in the next subsection using these types are defined in the ga library (and some more only documented in source code). You will find operations required to work with linear (multi-)vector spaces and operations specific to geometric algebra. In the following replace `algebra` with any of {`ega2d`, `ega3d`, `pga2dp`, `pga3dp`, `sta4ds`}:

- `ga_algebra_ops_basics.hpp` contains basic operations like reversions, conjugations, complements, duals, norms, normalization and additionally for PGA bulk, weight and unitization operations.
- `ga_algebra_ops_products.hpp` contains product operations like dot, wedge, commutator, transwedge and geometric products, and their regressive counterparts, as well as contractions, expansions, and inverses of blades.
- `ga_algebra_ops.hpp` contains general operations like angle, exp, sqrt, project_onto, reflect_from, is_congruent, and sandwich products for motion operators provided as rotate (EGA), move2dp/3dp (PGA), and transform (STA); the functions get_rotor (EGA, STA), get_motor

(PGA), and get_boost (STA) provide the input arguments for the motion operations.

- `ga_algebra_ops_mechanics.hpp` (PGA only) contains basics for rigid body dynamics calculations like inertia, its inverse and right hand sides for the dynamics equations.

All product expressions for inner, outer, interior and geometric products as well as their regressive counterparts are generated by programs in the subfolder `ga/ga_prdxpr`. User input to configure product generation is provided in the header files of the corresponding algebra. The generated coefficient expressions are used in the source code to implement dot products, wedge products, commutator products, contractions, expansions, geometric products or their regressive counterparts, as well as sandwich products describing rotors and motors of the corresponding algebra. If certain products are required for your application, but not yet implemented in the ga library, they can be easily generated within `ga/ga_prdxpr/src_prdxpr` as needed and added with minimal effort.

1.1.2 Product types – inner, interior, outer, geometric

As mentioned before, we distinguish inner, interior, outer and geometric products.

- Inner ($a \cdot b$) and interior products, like contractions and expansions ($a \vee b^\star$, $a \wedge b^\star$), are grade reducing operations. The inner product combines arguments of the same type and grade, while the interior products involve duals of one of their arguments, combining and linking arguments of different grades. Inner products are linked to the metric of the modelled space. They provide the capability of projecting objects like vectors on other objects like vectors or planes.

- The outer product ($a \wedge b$) is a grade-increasing operation adding the grades of its arguments. It extends the space that is taken by one object by the space that is taken by another object (e.g. two linear independent vectors, representing a linear subspace each, span a plane that is defined by the additive combination of those sub-spaces). The outer product (also called the wedge product) connects subspaces represented by their operands and creates new subspaces defined by the span of the subspaces of its operands. It establishes a join-operation between the involved subspaces.

The outer product also allows for direct calculation of linear independence: if vectors are linear independent their outer product is unequal zero, if they are linear dependent their outer product is zero.

- The geometric product ($a \gtrdot b$) is the combination of both types of products in one expression. They can result in multivectors consisting of parts with different grades as result of the operation. The main advantage of the geometric product is that it is invertible under certain circumstances.

- Regressive products ($a \circ b \equiv \underline{\overline{a} \cdot \overline{b}}$, $a \vee b \equiv \underline{\overline{a} \wedge \overline{b}}$, $\mathrm{rcmt}(a,b) \equiv \underline{\mathrm{cmt}(\overline{a},\overline{b})}$, $a \veebar b \equiv \underline{\overline{a} \veebar \overline{b}}$) use complement operations on their arguments before and after applying the operation[1]. The regressive wedge product finds common lower-dimensional subspaces, i.e. it effectively finds the intersection of the sub-spaces represented by its arguments, thus effectively creating a meet-operation between the involved spaces. It can be used to find the intersection of sub-spaces, like the intersection point between a line and a plane, as a very simple product operation.

- Sandwich products ($R \gtrdot u \gtrdot \widetilde{R}$ for EGA and STA, or $M \veebar u \veebar \underbrace{M}$ for PGA) are used to implement orthogonal transformations of geometric objects u. They realize rotations for EGA and general motions including rotation and translation for PGA by concatenating two consecutive reflections to implement the resulting orthogonal transformations. In $2D$ the consecutive reflections are across lines, while in $3D$ the consecutive reflections are across planes. This can be achieved algebraically by multiplying the object symmetrically from the left- and right-hand side with the appropriate (typically unitized) objects, we either call rotors R in EGA or motors M in PGA.

Following bi-argument operations `op(a,b)` = a op b and their regressive variants `rop(a,b)` = $\overline{\overline{a}\ \mathrm{op}\ \overline{b}}$ are implemented (with `op(a,b)` or `rop(a,b)` as the name of the implemented function). Using `A` vs. `a` implies `gr(A)` $\geq$ `gr(a)`, where `gr(arg)` returns the grade of the argument `arg`:

- Inner product, delivering a scalar: $a \cdot b$, `dot(a,b)`

- Outer product (wedge product) of a j- and a k-vector, delivering a $j+k$-vector in $span(a,b)$: $a \wedge b$, `wdg(a,b)`

- Geometric product: $a \gtrdot b$, `operator*(a,b)`

[1] For definition of the left and right complement operations marked with under-line and over-line, see next section.

  - For vectors a and b: $a \barwedge b = a \cdot b + a \wedge b$
  - For a vector a and a multivector B: $a \barwedge B = B \gg a + a \wedge B$
  - For a multivector A and a vector b: $A \barwedge b = b \ll A + A \wedge b$

- Commutator product (the asymmetric part of the geometric product): $\mathrm{cmt}(a, b) \equiv \frac{1}{2}(a \barwedge b - b \barwedge a)$, `cmt(a,b)`
- Regressive commutator product (the asymmetric part of the regressive geometric product):
  $\mathrm{rcmt}(a, b) \equiv \frac{1}{2}(\underline{\overline{a} \barwedge \overline{b} - \overline{b} \barwedge \overline{a}}) = \frac{1}{2}(a \vee b - b \vee a)$, `rcmt`(a, b)
- Regressive inner product, delivering a pseudoscalar: $a \circ b$, `rdot(a,b)` (for comparison of the dot product and the regressive dot product for $PGA3DP$ see table 9 at the end of section 1.2)
- Regressive outer product (regressive wedge product): $a \vee b$, `rwdg(a,b)`
- Regressive geometric product: $a \barvee b$, `rgpr(a,b)`, exclusively for PGA
- Left (bulk) contraction: $a \lfloor B = a \ll B = a_\star \vee B$, `operator<<(a,B)`, which implements `l_bulk_contract(a,B)`, exclusively for PGA
- Right (bulk) contraction: $B \rfloor a = B \gg a = B \vee a^\star$, `operator>>(B,a)`, which implements `r_bulk_contract(B,a)`, exclusively for PGA
- Left weight contraction: $a_\star \vee B$, `l_weight_contract(a,B)`
- Right weight contraction: $B \vee a^*$, `r_weight_contract(B,a)`
- Left bulk expansion: $A_\star \wedge b$, `l_bulk_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `r_bulk_expand(a,B)`
- Left weight expansion: $A_\star \wedge b$, `l_weight_expand(A,b)`
- Right bulk expansion: $a \wedge B^*$, `r_weight_expand(a,B)`

where the items from the left contraction downwards are all considered interior products. The product operation is performed *after* applying a dualization operation to one operand.

All products operate directly on the subspaces occupied by the objects that serve as arguments for the product operation.

The product tables for all products are generated in `ga/ga_prdxpr`. They can be derived directly from simple rules that define the products between basis vectors. The rules are generated for the geometric product, the outer (or wedge) product, the dot product, the complements and duals of the respective algebra. All resulting product expressions can be calculated from those few basic rules. Coefficient expressions for specific argument types like scalars, vector, bivectors, etc. can be generated based on the product tables using configuration options of `ga/ga_prdxpr` for each algebra.

PGA explicitly means *Projective Geometric Algebra* in the context of the ga library. There is also an alternative approach called *Plane-based Geometric Algebra* (also abbreviated *PGA*) that is based on a space that is dual to the space modelled and used here. The alternative approach was introduced by [Gunn, 2020] and further promoted for applications in projective geometry by [Dorst and De Keninck, 2022]. That model uses dual representations of objects represented in the algebra (it uses $\mathbb{G}^*(3,0,1)$, a space dual to $\mathbb{G}(3,0,1)$). Planes are used instead of vectors to model everything, e.g. a point is modelled by three planes intersecting in a point. Both representations are mathematically equivalent, i.e. they are dual to each other. They are linked by the definition of complements and the interconnection between geometric and regressive geometric product, as defined above: $a \vee b \equiv \underline{\overline{a} \wedge \overline{b}}$. In the plane-based approach the geometric product is used to operate on complements and duals, which are defined as the primary objects used for the approach, while the approach introduced by Lengyel in [Lengyel, 2024b] favors the use of points, vectors, bivectors and planes plus the explicit use of regressive products and complements as well as duals for applying geometric operations to these objects. The latter approach unifies the use of the wedge product for join operations and of the regressive wedge product for meet operations for applications.

[Lengyel, 2024a] points out mathematical inconsistencies in applications of geometric algebra as used in the past quite often by different authors. Further, hints on mathematical inconsistencies in typical GA approaches can be found as well in [Kritchevsky, 2024]. The goal of this work is to be as consistent and mathematically rigorous as possible implementing GA functionality, and at the same time providing an intuitively applicable framework for modelling physics and applications.

1.2 Basics of the mathematical approach

1.2.1 Basis vectors for each modelled algebra

EGA2D: The following is provided for Euclidean geometric algebra of two-dimensional space using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \text{ and } \boldsymbol{e}_2^2 = +1$$

$$s_{2d} = s\,\mathbf{1} \tag{1a}$$

$$v_{2d} = v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 \tag{1b}$$

$$ps_{2d} = ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1} \tag{1c}$$

equation (1a) is the scalar part s_{2d}, equation (1b) contains the vector part v_{2d}, and equation (1c) contains the pseudoscalar part ps_{2d}. The index $2d$ is omitted when clear from context. The four basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_{12}\}$. Using this basis a multivector M of $2d$-space is defined as

$$M = s\,\mathbf{1} + v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + ps\,\boldsymbol{e}_{12} \tag{2}$$

where equation (2) contains three parts: the scalar part $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$; if $\mathbf{1}$ is not shown, it is implicitly assumed for scalar values), the vector part v (basis elements $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$) and the pseudoscalar part $ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1}$ (basis element is $I_{2d} = \boldsymbol{e}_{12}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{2d} is a bivector in two-dimensional Euclidean space).

EGA3D: For Euclidean geometric algebra of three-dimensional space using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = +1$$

$$s_{3d} = s\,\mathbf{1} \tag{3a}$$

$$v_{3d} = v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + v_z\boldsymbol{e}_3 \tag{3b}$$

$$B_{3d} = b_x\boldsymbol{e}_{23} + b_y\boldsymbol{e}_{31} + b_z\boldsymbol{e}_{12} \tag{3c}$$

$$ps_{3d} = ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1} \tag{3d}$$

equation (3a) is the scalar part s_{3d}, equation (3b) is the vector part v_{3d}, equation (3c) is the bivector part B_{3d}, and equation (3d) is the pseudoscalar part ps_{3d}. The index $3d$ is omitted when clear from context. Compared to the $2d$-case it is obvious, that all parts depend on context, specifically on the dimensionality of the modelled space, and thus need to be defined and

treated accordingly.[2] The basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{123}\}$. Using this basis a multivector M of $3d$-space is defined as

$$M = s\,\mathbf{1} + v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + v_z\boldsymbol{e}_3 + b_x\boldsymbol{e}_{23} + b_y\boldsymbol{e}_{31} + b_z\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{123} \tag{4}$$

where equation (4) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1}$ (basis element is $I_{3d} = \boldsymbol{e}_{123}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{3d} is a trivector in three-dimensional Euclidean space).

PGA2DP: Projective geometric algebra of $2D$ Euclidean space is modelled by embedding it in a three-dimensional space. Using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = 0$$

$$s_{2dp} = s\,\mathbf{1} \tag{5a}$$

$$v_{2dp} = v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + v_z\boldsymbol{e}_3 \tag{5b}$$

$$B_{2dp} = b_x\boldsymbol{e}_{31} + b_y\boldsymbol{e}_{32} + b_z\boldsymbol{e}_{12} \tag{5c}$$

$$ps_{2dp} = ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1} \tag{5d}$$

The most obvious difference is the basis vector $\boldsymbol{e}_3$ that squares to zero and the pseudoscalar of that space that is chosen to be $\boldsymbol{e}_{321}$. The trivector $\boldsymbol{e}_{321}$ squares to 0, because of its component $\boldsymbol{e}_3$ contained as a factor. Equation (5a) is the scalar part s_{2dp}, equation (5b) is the vector part v_{2dp}, equation (5c) is the bivector part B_{2dp}[3], and equation (5d) is the pseudoscalar part ps_{2dp}. The index $2dp$ is omitted when clear from context. The eight basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{31}, \boldsymbol{e}_{32}, \boldsymbol{e}_{12}, \boldsymbol{e}_{321}\}$. Using this basis a multivector M of two-dimensional projective space ($2dp$) is defined as

$$M = s\,\mathbf{1} + v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + v_z\boldsymbol{e}_3 + b_x\boldsymbol{e}_{31} + b_y\boldsymbol{e}_{32} + b_z\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{321} \tag{6}$$

where equation (6) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the

[2] *Hint*: Since C++ is a statically typed language, those types need to be well-defined and distinguishable from each other. This is especially important for the different scalar and pseudoscalar types. The ga library uses type tagging extensively for that purpose.

[3] Here we deviate from Lengyel's approach chosen in [Lengyel, 2024b]. Since we want to model physics, it is important to generate unified handling for two- and three-dimensional cases. The chosen sequence of basis bivectors assures that lines, which are used to model forces and moments later on, have their directions encoded in a unified way.

bivector part B (basis elements $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{32}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1}$ (basis element is $I_{2dp} = \boldsymbol{e}_{321}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a trivector in two-dimensional Euclidean projective space that squares to zero).

In $2D\ PGA$ a projective point Q is modelled as a vector

$$Q_{pga2dp} = \underbrace{q_x\boldsymbol{e}_1 + q_y\boldsymbol{e}_1}_{position} + \underbrace{q_z\boldsymbol{e}_3}_{weight} \tag{7}$$

where the position is encoded in the bulk part[4] of the point, and the weight is the projective component q_z that must be unitized (=scaled to 1.0) in order to project the point into the Euclidean space.

A line L is modelled as a bivector

$$L_{pga2dp} = \underbrace{l_x\boldsymbol{e}_{31} + l_y\boldsymbol{e}_{32}}_{direction} + \underbrace{l_z\boldsymbol{e}_{12}}_{position} \tag{8}$$

where the weight part encodes the two-dimensional direction vector (l_x, l_y) of the line and the bulk part encodes the position of the line as distance to the origin $O_{2dp} = (0, 0, 1)$. When $l_z = 0$ the line passes through the origin.

PGA3DP: Projective geometric algebra of $3D$ Euclidean space is modelled by embedding it in a four-dimensional space. Using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \boldsymbol{e}_3^2 = +1, \text{ and } \boldsymbol{e}_4^2 = 0$$

$$s_{3dp} = s\,\mathbf{1} \tag{9a}$$

$$v_{3dp} = v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + v_z\boldsymbol{e}_3 + v_w\boldsymbol{e}_4 \tag{9b}$$

$$B_{3dp} = b_{vx}\boldsymbol{e}_{41} + b_{vy}\boldsymbol{e}_{42} + b_{vz}\boldsymbol{e}_{43} + b_{mx}\boldsymbol{e}_{23} + b_{my}\boldsymbol{e}_{31} + b_{mz}\boldsymbol{e}_{12} \tag{9c}$$

$$t_{3dp} = t_x\boldsymbol{e}_{423} + t_y\boldsymbol{e}_{431} + t_z\boldsymbol{e}_{412} + t_w\boldsymbol{e}_{321} \tag{9d}$$

$$ps_{3dp} = ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1} \tag{9e}$$

Now we have a basis vector $\boldsymbol{e}_4$ that squares to zero. The bivector has six components, and there is a trivector t. The pseudoscalar $\boldsymbol{e}_{1234}$ is a quadvector that squares to 0, because of its component $\boldsymbol{e}_4$ contained as a factor. Equation (9a) is the scalar part s_{3dp}, equation (9b) is the vector part v_{3dp}, equation (9c) is the bivector part B_{3dp}, equation (9d) is the trivector part

[4] Bulk and weight parts are defined further below in this section.

t_{3dp}, and equation (9e) is the pseudoscalar part ps_{3dp}. The index $3dp$ is omitted when clear from context. The sixteen basis elements of $PGA3DP$ are: $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_4, \boldsymbol{e}_{41}, \boldsymbol{e}_{42}, \boldsymbol{e}_{43}, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{423}, \boldsymbol{e}_{431}, \boldsymbol{e}_{412}, \boldsymbol{e}_{321}, \boldsymbol{e}_{1234}\}$. A multivector M of three-dimensional projective space ($3dp$) is defined as

$$\begin{aligned} M = {} & s\,\mathbf{1} \\ & + v_x\boldsymbol{e}_1 + v_y\boldsymbol{e}_2 + v_z\boldsymbol{e}_3 + v_w\boldsymbol{e}_4 \\ & + b_{vx}\boldsymbol{e}_{41} + b_{vy}\boldsymbol{e}_{42} + b_{vz}\boldsymbol{e}_{43} + b_{mx}\boldsymbol{e}_{23} + b_{my}\boldsymbol{e}_{31} + b_{mz}\boldsymbol{e}_{12} \\ & + t_x\boldsymbol{e}_{423} + t_y\boldsymbol{e}_{431} + t_z\boldsymbol{e}_{412} + t_w\boldsymbol{e}_{321} \\ & + ps\,\boldsymbol{e}_{1234} \end{aligned} \tag{10}$$

where equation (10) contains five parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$), the bivector part B (basis elements $\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$), the trivector part t (basis elements $\boldsymbol{e}_{423}$, $\boldsymbol{e}_{431}$, $\boldsymbol{e}_{412}$, $\boldsymbol{e}_{321}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1}$ (basis element is $I_{3dp} = \boldsymbol{e}_{1234}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a quadvector in three-dimensional Euclidean projective space that squares to zero).

In $3D\ PGA$ a projective point Q is modelled as a vector

$$Q_{pga3dp} = \underbrace{q_x\boldsymbol{e}_1 + q_y\boldsymbol{e}_1 + q_z\boldsymbol{e}_1}_{position} + \underbrace{q_w\boldsymbol{e}_4}_{weight} \tag{11}$$

where the position is encoded in the bulk part of the point, and the weight is the projective component q_w that must be unitized (=scaled to 1.0) in order to project the point into the Euclidean space.

A line L is modelled as a bivector

$$L_{pga3dp} = \underbrace{l_{vx}\boldsymbol{e}_{41} + l_{vy}\boldsymbol{e}_{42} + l_{vz}\boldsymbol{e}_{43}}_{direction} + \underbrace{l_{mx}\boldsymbol{e}_{23} + l_{my}\boldsymbol{e}_{31} + l_{mz}\boldsymbol{e}_{12}}_{moment} \tag{12}$$

where the weight part encodes the three-dimensional direction vector of the line $l_v = (l_{vx}, l_{vy}, l_{vz})$, and the bulk part encodes the moment of the line $l_m = (l_{mx}, l_{my}, l_{mz})$ when interpreted as a vector. Both are encoded in the bivector object that captures all parameters needed to model the line.

Additionally, there are planes T that are modelled as trivectors

$$T_{pga3dp} = \underbrace{t_x\boldsymbol{e}_{423} + t_y\boldsymbol{e}_{431} + t_z\boldsymbol{e}_{412}}_{normal} + \underbrace{t_w\boldsymbol{e}_{312}}_{position} \tag{13}$$

where the first three components represent the normal vector of the plane $n = (t_x, t_y, t_z)$ and the fourth component t_w corresponds to the signed distance to the origin $O_{3dp} = (0, 0, 0, 1)$.

Many more examples of how geometric objects are modelled in projective geometric algebra and which operations can be used to manipulated them can be found in Lengyel's book [Lengyel, 2024b]. Browne explains very well how Grassmann algebra and projective geometry and geometric algebra are related in [Browne, 2012], p. 216ff. It is helpful to fully grasp and understand that relation in order to see what is possible using the algebra effectively to model physical phenomena.

STA4DS: Space-time algebra of four-dimensional Minkowski space is modelled using an orthonormal basis $\boldsymbol{g}_1$, $\boldsymbol{g}_2$, $\boldsymbol{g}_3$, $\boldsymbol{g}_4$ with the mixed (Minkowski) signature, where the three space-like directions square to −1 and the time-like direction $\boldsymbol{g}_4$ squares to +1:

$$(\boldsymbol{g}_1)^2 = \boldsymbol{g}_1 \wedge \boldsymbol{g}_1 = \boldsymbol{g}_1 \cdot \boldsymbol{g}_1 = \boldsymbol{g}_1^2 = -1, \boldsymbol{g}_2^2 = -1, \boldsymbol{g}_3^2 = -1, \text{ and } \boldsymbol{g}_4^2 = +1$$

$$s_{4ds} = s\,\mathbf{1} \tag{14a}$$

$$v_{4ds} = v_x\boldsymbol{g}_1 + v_y\boldsymbol{g}_2 + v_z\boldsymbol{g}_3 + v_w\boldsymbol{g}_4 \tag{14b}$$

$$B_{4ds} = b_{vx}\boldsymbol{g}_{14} + b_{vy}\boldsymbol{g}_{24} + b_{vz}\boldsymbol{g}_{34} + b_{mx}\boldsymbol{g}_{23} + b_{my}\boldsymbol{g}_{31} + b_{mz}\boldsymbol{g}_{12} \tag{14c}$$

$$t_{4ds} = t_x\boldsymbol{g}_{234} + t_y\boldsymbol{g}_{314} + t_z\boldsymbol{g}_{124} + t_w\boldsymbol{g}_{123} \tag{14d}$$

$$ps_{4ds} = ps\,\boldsymbol{g}_{1234} = ps\,\mathbb{1} \tag{14e}$$

Unlike *PGA*, where one basis vector squares to zero (degenerate metric), *STA4DS* has a non-degenerate metric; but in contrast to the all-positive Euclidean signature of *EGA*, its signature is indefinite (mixed signs). Equation (14a) is the scalar part s_{4ds}, equation (14b) is the vector part v_{4ds}, equation (14c) is the bivector part B_{4ds}, equation (14d) is the trivector part t_{4ds}, and equation (14e) is the pseudoscalar part ps_{4ds}. The index $4ds$ is omitted when clear from context. The sixteen basis elements of *STA4DS* are: $\{\mathbf{1}, \boldsymbol{g}_1, \boldsymbol{g}_2, \boldsymbol{g}_3, \boldsymbol{g}_4, \boldsymbol{g}_{14}, \boldsymbol{g}_{24}, \boldsymbol{g}_{34}, \boldsymbol{g}_{23}, \boldsymbol{g}_{31}, \boldsymbol{g}_{12}, \boldsymbol{g}_{234}, \boldsymbol{g}_{314}, \boldsymbol{g}_{124}, \boldsymbol{g}_{123}, \boldsymbol{g}_{1234}\}$. The grade structure mirrors that of *PGA3DP* (both are four-dimensional algebras with sixteen basis elements); the difference is the vector metric, where *STA4DS* replaces the one degenerate and three positive directions of *3D PGA* by three negative and one positive direction. A multivector M of

space-time algebra ($4ds$) is defined as

$$\begin{aligned} M = {} & s\,\mathbf{1} \\ & + v_x\boldsymbol{g}_1 + v_y\boldsymbol{g}_2 + v_z\boldsymbol{g}_3 + v_w\boldsymbol{g}_4 \\ & + b_{vx}\boldsymbol{g}_{14} + b_{vy}\boldsymbol{g}_{24} + b_{vz}\boldsymbol{g}_{34} + b_{mx}\boldsymbol{g}_{23} + b_{my}\boldsymbol{g}_{31} + b_{mz}\boldsymbol{g}_{12} \\ & + t_x\boldsymbol{g}_{234} + t_y\boldsymbol{g}_{314} + t_z\boldsymbol{g}_{124} + t_w\boldsymbol{g}_{123} \\ & + ps\,\boldsymbol{g}_{1234} \end{aligned} \tag{15}$$

where equation (15) contains five parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{g}_1$, $\boldsymbol{g}_2$, $\boldsymbol{g}_3$, $\boldsymbol{g}_4$), the bivector part B (basis elements $\boldsymbol{g}_{14}$, $\boldsymbol{g}_{24}$, $\boldsymbol{g}_{34}$, $\boldsymbol{g}_{23}$, $\boldsymbol{g}_{31}$, $\boldsymbol{g}_{12}$), the trivector part t (basis elements $\boldsymbol{g}_{234}$, $\boldsymbol{g}_{314}$, $\boldsymbol{g}_{124}$, $\boldsymbol{g}_{123}$) and the pseudoscalar part $ps\,\boldsymbol{g}_{1234} = ps\,\mathbb{1}$ (basis element is $I_{4ds} = \boldsymbol{g}_{1234}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a quadvector in four-dimensional space-time that squares to -1 under the geometric product).

In STA a vector models a space-time event or, more generally, a four-vector

$$X_{sta4ds} = \underbrace{x_x\boldsymbol{g}_1 + x_y\boldsymbol{g}_2 + x_z\boldsymbol{g}_3}_{space} + \underbrace{x_w\boldsymbol{g}_4}_{time} \tag{16}$$

where the three space-like components are carried by $\boldsymbol{g}_1$, $\boldsymbol{g}_2$ and $\boldsymbol{g}_3$, and the time-like component by $\boldsymbol{g}_4$. Typical examples are the four-position, the four-velocity and the four-momentum. The geometric square $X^2 = X \wedge X$ is the squared Minkowski interval; it is positive for time-like, negative for space-like and zero for light-like vectors.

The bivectors carry the generators of the Lorentz transformations. The three bivectors containing the time-like direction ($\boldsymbol{g}_{14}$, $\boldsymbol{g}_{24}$, $\boldsymbol{g}_{34}$) generate boosts (relative motion between frames), while the three purely space-like bivectors ($\boldsymbol{g}_{23}$, $\boldsymbol{g}_{31}$, $\boldsymbol{g}_{12}$) generate spatial rotations. As for the rotors in EGA, the even-grade part of the algebra (scalar, bivector and pseudoscalar – the eight even basis elements) forms the group of rotors R that implement Lorentz transformations of any object X via the sandwich product $transform(X, R) = R \wedge X \wedge \widetilde{R}$ based on the geometric product, in complete analogy to the rotations in EGA.

Geometric algebra seems complex and a little overwhelming at first. Especially so, when new objects beyond vectors and several specific products are introduced. Expressions tend to quickly become large for fully occupied multivectors. However, it is important to note that it is rarely required to use

full multivectors in the respective algebra. For most applications it is sufficient to use a blade or k-vector exclusively, which represents only one grade of the multivector, e.g. a vector or a bivector, which can be represented with very few coefficients. This reduces the effort in terms of memory and computational requirements significantly – typically reduced to a level that is required anyway to solve the underlying geometrical question at hand.

1.2.2 The complement

The complement operation used in the library is defined in [Lengyel, 2024b]. It is uniquely determined with respect to the outer product (*not* with respect to the geometric product, as commonly used for dualization by many authors working with GA). This ensures consistent signs of expressions, operators, complements and duals. The left complement $\underline{u}$ and the right complement $\overline{u}$ are defined for a blade u as

$$\underline{u} \wedge u = I_n \qquad (\texttt{l\_cmpl(u)}) \tag{17a}$$
$$u \wedge \overline{u} = I_n \qquad (\texttt{r\_cmpl(u)}) \tag{17b}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n$ as the pseudoscalar for an n-dimensional algebra, and the wedge symbol ($\wedge$) as the operator symbol of the outer product. The effect of the complement is to turn basis elements not contained in the object u into the constituting basis elements of the complement of the object $\underline{u}$ or $\overline{u}$. Connecting the object and its complement with the wedge products creates an object that fills the available space in the sense of requiring all basis elements of the space to represent it. The full space itself is represented by the pseudoscalar, since the pseudoscalar I_n is formed by the product of all n basis vectors of the space. An implication of the definition is that the complement ($\underline{u}$ or $\overline{u}$) is orthogonal to u, since we exclusively use orthogonal basis vectors to model our space.

The complement is defined as given above in equations (17) for any basis element. It is extended to full multivectors A and B by linearity as

$$\overline{sA} = s\overline{A}, \quad \overline{A+B} = \overline{A}+\overline{B} \tag{18a}$$
$$\underline{sA} = s\underline{A}, \quad \underline{A+B} = \underline{A}+\underline{B} \tag{18b}$$

where s is a scalar. The left and right complements fulfill

$$\overline{\underline{u}} = u \tag{19}$$

thus the left and right complement operations are inverses to each other. This is valid independent of the sequence of application, independent of the

grade of the argument a, and independent of the dimension of the space modelled in the algebra. In spaces of odd dimension, where we only have one complement, applying the complement twice of course returns the original object as well.

However, applying the same complement operation several times in spaces of even dimension does not ensure that you get back the original object. The result is dependent on the grade of the basis vector and the dimension n of the space. When applying the same complement twice we get

$$\overline{\overline{u}} = \underline{\underline{u}} = (-1)^{gr(u)(n-gr(u))}u \tag{20}$$

i.e. this involves grade-dependent sign changes. So it is favorable to use left and right complements as inverses to each other.

Based on the given complement definition it is possible to define regressive products using rules analog to de Morgan's rules in Boolean algebra as follows (definitions can be found in [Lengyel, 2024b] and in [Browne, 2012]):

$$a \vee b \equiv \overline{\underline{a} \wedge \underline{b}} \tag{21a}$$
$$a \vee b \equiv \underline{\overline{a} \wedge \overline{b}} \tag{21b}$$

The same complement is applied to both arguments first, the resulting expressions are combined with the wedge product, and the inverse complement operation is used to transform the outcome into the final result, the regressive product. The inverted wedge symbol $(\vee)$ is used for the regressive wedge product. Both definitions (21a) and (21b) are equivalent. Equation (21a) is used to derive expressions for regressive products in `ga/ga_prdxpr`.

There are some additional formulas for complements which are useful for working with GA expressions and thus are provided here for reference. Taking the right and left complements of (21a) and (21b) we end up with

$$\overline{a \vee b} = \overline{\underline{\overline{a} \wedge \overline{b}}} = \overline{a} \wedge \overline{b} \tag{22a}$$
$$\underline{a \vee b} = \underline{\overline{\underline{a} \wedge \underline{b}}} = \underline{a} \wedge \underline{b} \tag{22b}$$

Substituting $a = \underline{u}$ and $b = \underline{v}$ into (22a) and $a = \overline{u}$ and $b = \overline{v}$ into (22b) and exchanging the left- and right-hand sides of the equations we end up with

$$u \wedge v = \overline{\underline{u} \vee \underline{v}} \tag{23a}$$
$$u \wedge v = \underline{\overline{u} \vee \overline{v}} \tag{23b}$$

which shows that the outer and the regressive outer product can be expressed by each other.

The complement operation is used to define a unique dualization operation that works for cases with non-degenerate as well as for cases with degenerate metric, like in projective geometric algebra PGA. This is the main advantage compared to the usual procedure of dualization by multiplying the object with the pseudoscalar (or its inverse) using the geometric product, since the latter breaks down for spaces with degenerate metrics. For an overview on all complements in EGA see subsection 1.2.8, in PGA see subsection 1.2.9, and in STA see subsection 1.2.12.

1.2.3 The metric

The metric is defined by the dot product of the basis vectors as

$$\mathfrak{g}_{ij} \equiv \boldsymbol{e}_i \cdot \boldsymbol{e}_j \tag{24}$$

It is a square matrix that is the identity matrix I_n for EGA, since all basis vectors square to $\boldsymbol{e}_i \cdot \boldsymbol{e}_i = +1$, and $\boldsymbol{e}_i \cdot \boldsymbol{e}_j = 0$ for $i \neq j$. It is invertible in this case. However, for PGA one basis vector squares to zero, and thus there is one zero on the main diagonal of the corresponding matrix of the metric, which is therefore not invertible (its determinant is zero). It is called degenerate for that reason.

The extended metric $\mathbf{G}$ is a so-called exomorphism matrix that fulfills

$$\mathbf{G}(A \wedge B) = \mathbf{G}(A) \wedge \mathbf{G}(B) \tag{25}$$

for any multivector A and B (see [Lengyel, 2024b], pp. 61ff.). This exomorphism is defined with respect to the wedge product. The wedge product itself is defined independent of any metric. However, this property defines an extended metric, and the resulting extended metric can in turn be used to define a dot product for components of any grade of a multivector. The dot product $dot(a, b) = a^T \mathbf{G} b$ is defined as a metric product (see eq. (50) further below).

Regarding $\mathbf{G}(u) = \mathbf{G}u$ as a unary operation on u we can define the corresponding regressive operation by first taking the complement of the argument and then applying the opposed complement on the result. This defines the regressive metric $\mathbb{G}$ (named antimetric by Lengyel). It is defined as

$$\mathbb{G}u \equiv \underline{\mathbf{G}\overline{u}} = \overline{\mathbf{G}\underline{u}} \tag{26}$$

$\mathbf{G}$ and $\mathbb{G}$ are related by

$$\mathbf{G}\mathbb{G} = det(\mathfrak{g})I_n \tag{27}$$

Thus, if all basis vectors square to ± 1, we have $\mathbf{G}\mathbb{G} = \pm I_n$, and both are invertible. However, if one basis vector squares to zero, we get $\mathbf{G}\mathbb{G} = 0$, which means that none of them is invertible, i.e. we have a degenerate metric.

For $\mathbb{G}$ there is the relation

$$\mathbb{G}(A \vee B) = \mathbb{G}(A) \vee \mathbb{G}(B) \tag{28}$$

that is complementary to equation (25). It is an exomorphism defined with respect to the regressive wedge product.

In projective geometric algebra the product $\mathbf{G}u$ defines the *bulk* of an object u. It is defined as

$$u_\bullet = \mathbf{G}u \qquad (\texttt{bulk(u)}) \tag{29}$$

The bulk only contains components that *do not* contain the projective dimension ($\boldsymbol{e}_3$ for $2D$ PGA or $\boldsymbol{e}_4$ for $3D$ PGA) as a factor. The *bulk* $u_\bullet$ *contains positional information* on u relative to the origin. Zero bulk ($u_\bullet = 0$) means that u contains the origin.

The product $\mathbb{G}u$ defines the *weight* of an object u. It is defined as

$$u_\circ = \mathbb{G}u \qquad (\texttt{weight(u)}) \tag{30}$$

The weight selects components that *do* indeed contain the projective dimension ($\boldsymbol{e}_3$ for $2D$ PGA or $\boldsymbol{e}_4$ for $3D$ PGA) as a factor. The *weight* $u_\circ$ *contains directional information* on u, i.e. its attitude and direction independent of its position. Zero weight ($u_\circ = 0$) means that u is contained in the horizon, i.e. it is infinitely far away.

This effective split for each object into bulk and weight in projective algebras can be written as

$$u = u_\bullet + u_\circ \tag{31}$$

This split is also relevant for the corresponding norms

$$\|u\| = \|u\|_\bullet + \|u\|_\circ = \sqrt{A \cdot A} + \sqrt{A \circ A} \tag{32}$$

where the left-hand side is called the *geometric norm* and the right-hand side consists of the sums of the *bulk* and *weight norms*. The *bulk norm* returns a scalar value of the corresponding algebra, whereas the *weight norm* provides

a pseudoscalar value of the respective algebra. The dot product (symbol '$\cdot$') and its regressive counterpart (symbol '$\circ$') are defined in tables 5 and 9 in subsection 1.2.9.

Consequently, the geometric norm is an object consisting of two distinct parts, a scalar and a pseudoscalar of the respective algebra, defined here as a *dual number* in the corresponding algebra. When we apply *unitization* to geometric objects we divide the object in such a way that the magnitude of the weight norm becomes a value of one. Both, bulk and weight norms, have a positive argument under the square root due to the definitions of the inner product and the regressive inner product.

1.2.4 The dual

In *EGA* the dual operation (marked by a filled star $\star$) is identical with the complement (reason: the Euclidean metric is the identity matrix; this changes for *PGA* as shown below). Therefore, in $2D$ *EGA* for the left and right dual operations are defined as

$$u^\star = \overline{u} \qquad \text{(right dual: } \texttt{r\_dual(u)}\text{)} \tag{33}$$

$$u_\star = \underline{u} \qquad \text{(left dual: } \texttt{l\_dual(u)}\text{)} \tag{34}$$

while in $3D$ *EGA*, an odd-dimensional algebra, where no distinction between left and right operations exists, we get

$$u^\star = \overline{u} \qquad \text{(dual: } \texttt{dual(u)}\text{)} \tag{35}$$

where the overline in $\overline{u}$ is the unique complement operation.

Now for *PGA*: Using the metric and the right complement, the *(right) bulk dual* $A^\star$ is defined as

$$A^\star = \overline{\mathbf{G}A} = \overline{A_\bullet} \qquad \text{(}\texttt{r\_bulk\_dual(u)}\text{)} \tag{36}$$

where A is an arbitrary multivector, $\mathbf{G}$ the extended metric and $\mathbf{G}A$ a matrix-vector-product between them. $A^\star$ is also called the Hodge dual. The multiplication with the metric happens before the application of the complement operation.

There is also a corresponding *(left) bulk dual* $A_\star$ operation using the metric and the left complement. It is defined as

$$A_\star = \underline{\mathbf{G}A} = \underline{A_\bullet} \qquad \text{(}\texttt{l\_bulk\_dual(u)}\text{)} \tag{37}$$

Where the left and right operations only differ for $3D\ PGA$, while for $2D\ PGA$, a three-dimensional and thus an odd-dimensional algebra, there is no difference between the left and right operations.

The Hodge star (symbolized by a filled star $\star$) is used as dualization operator in both cases. The lower or upper position symbolizes the left and right application. The difference between the Hodge dual and the complement is in both cases that multiplying the multivector with the extended metric happens *before* taking the corresponding complement. However, this does not lead to different results in spaces where the metric is the identity matrix. It only yields different results in spaces with a more involved metric, e.g. in projective spaces with a non-invertible (degenerate) metric as in PGA.

The effect of applying the metric before taking the complement is equivalent to applying the complement operation on the bulk part of the object u exclusively, and not on the full object u. Of course, the dualization can be applied to the weight part as well. This operation (symbolized by an unfilled star "$\bigstar$") uses the regressive extended metric and is defined for the right weight dual $A^{\bigstar}$ using the right complement as

$$A^{\bigstar} = \overline{\mathbb{G}A} = \overline{A_{\circ}} \tag{38}$$

for an arbitrary multivector A. The left weight dual $A_{\bigstar}$ is defined as

$$A_{\bigstar} = \underline{\mathbb{G}A} = \underline{A_{\circ}} \tag{39}$$

using the left complement on the weight part of A.

Relations to the (regressive) geometric product are given by

$$A^{\star} = \widetilde{A} \barwedge I_n = \widetilde{A} \barwedge \mathbb{1} \tag{40a}$$
$$A_{\star} = I_n \barwedge \widetilde{A} = \mathbb{1} \barwedge \widetilde{A} \tag{40b}$$
$$A^{\bigstar} = \underset{\sim}{A} \vee \mathbf{1} \tag{40c}$$
$$A_{\bigstar} = \mathbf{1} \vee \underset{\sim}{A} \tag{40d}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n = \mathbb{1}$ as the pseudoscalar of the n-dimensional space. $\widetilde{A}$ is the reversion operation, and $\underset{\sim}{A}$ is the regressive reversion operation applied to A (e.g. with $A = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2$ the reverse is $\widetilde{A} = \boldsymbol{e}_2 \wedge \boldsymbol{e}_1$), and "$\barwedge$" and "$\vee$" are the symbols for the geometric and regressive geometric products. As the complement, the reverse is defined for any basis element. It is extended

to full multivectors A and B by linearity as

$$\widetilde{sA} = s\widetilde{A}, \quad \widetilde{A+B} = \widetilde{A}+\widetilde{B}, \quad \widetilde{A \wedge B} = \widetilde{B} \wedge \widetilde{A} \tag{41a}$$

$$\underset{\sim}{sA} = s\underset{\sim}{A}, \quad \underset{\sim}{A+B} = \underset{\sim}{A}+\underset{\sim}{B}, \quad \underset{\sim}{A \vee B} = \underset{\sim}{B} \vee \underset{\sim}{A} \tag{41b}$$

$$\widetilde{\widetilde{A}} = A, \quad \text{and} \quad \underset{\sim}{\underset{\sim}{A}} = A \tag{41c}$$

where s is a scalar. The reverse and regressive reverse are their own inverses, i.e. applying the same reverse twice returns the original object.

For an overview on all bulk and weight duals in PGA see tables 7 and 11 in subsection 1.2.9. The reverses can be found in tables 8 and 12 in that subsection as well. The corresponding left and right (metric) duals and the reverse for $STA4DS$ are given in tables 16 and 17 in subsection 1.2.12; since $STA4DS$ is non-degenerate there is no bulk/weight split.

Following identities link the expansions and contractions and thus the outer and inner products:

$$(A \wedge B)^\star = A^\star \vee B^\star \tag{42a}$$

$$A \wedge B^\star = (A \cdot B) \wedge \mathbb{1}, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{42b}$$

$$A \wedge B^* = A \circ B, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{42c}$$

$$(A \vee B)^* = A^* \wedge B^* \tag{42d}$$

$$A \vee B^\star = A \cdot B, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{42e}$$

$$A \vee B^* = (A \circ B) \vee \mathbf{1}, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{42f}$$

The influence of the metric for the dualization operation can be seen in its effect for the different algebras which also depends on the dimension of the modelled space:

- $EGA2D$: $A_\star$ = `l_dual(A)`, $A^\star$ = `r_dual(A)`
- $EGA3D$: $A_\star$ = `dual(A)`, $A^\star$ = `dual(A)`
- $PGA2DP$: $A_\star$ = `bulk_dual(A)`, $A^\star$ = `bulk_dual(A)`
- $PGA2DP$: A_* = `weight_dual(A)`, A^* = `weight_dual(A)`
- $PGA3DP$: $A_\star$ = `l_bulk_dual(A)`, $A^\star$ = `r_bulk_dual(A)`
- $PGA3DP$: A_* = `l_weight_dual(A)`, A^* = `r_weight_dual(A)`
- $STA4DS$: $A_\star$ = `l_dual(A)`, $A^\star$ = `r_dual(A)`

In even-dimensional spaces ($EGA2D$, $PGA3DP$, $STA4DS$) we have to distinguish left and right operations, while in odd-dimensional spaces ($EGA3D$, $PGA2DP$) we don't, since they produce the same result. The "left" operations are typically used on the left-hand side of the binary operation and the "right" operations on the right-hand side of the product operator. The only deviation from that pattern occurs for the definition of the regressive operations, as defined in (21a) or (21b).

In a two-dimensional algebra, like $EGA2D$, the effect of the right dual of a vector u is to turn the vector by +90°, i.e. into the orientation of $\boldsymbol{e}_{12}$, while the left dual turns the vector by −90°, i.e. into the direction against the orientation of $\boldsymbol{e}_{12}$. This statement is valid for both right- and left-handed coordinate systems.

Conjugations: There are grade-selective operations which can be useful for handling grade-dependent signs. These operations are defined for each algebra, but are shown here for the example of a four-dimensional algebra.

Grade involution of an arbitrary multivector M for its grade r-component is defined as $\widehat{M_r} = (-1)^r M_r$, effectively inverting the sign of odd-grade blades (function $gr\_inv()$):

$$\widehat{M} = \langle M\rangle_0 - \langle M\rangle_1 + \langle M\rangle_2 - \langle M\rangle_3 + \langle M\rangle_4 \tag{43}$$

Reversion of the component M_r is defined by the pattern $\widetilde{M_r} = (-1)^{\left(\frac{r(r-1)}{2}\right)} M_r$ (function $rev()$) resulting in

$$\widetilde{M} = \langle M\rangle_0 + \langle M\rangle_1 - \langle M\rangle_2 - \langle M\rangle_3 + \langle M\rangle_4 \tag{44}$$

While the Clifford conjugation of the grade r multivector-component M_r is defined by $M_r^* = (-1)^{\frac{r(r+1)}{2}} M_r$ (function $conj()$). It is effectively a composition of reversion and grade involution ($M^* = \widehat{\widetilde{M}} = \widetilde{\widehat{M}}$) resulting in

$$M^* = \langle M\rangle_0 - \langle M\rangle_1 - \langle M\rangle_2 + \langle M\rangle_3 + \langle M\rangle_4 \tag{45}$$

Before we go on to the geometric product, let's return to eqn. (13) for a moment, which provided the definition of a plane as trivector in projective space. We are going to use this example to show what can be done geometrically using the definitions provided above:

The trivector defining a plane can be created in multiple ways, e.g. either by wedge products between three unitized projective points P, Q, R that span the plane as $pl = P \wedge Q \wedge R$ as their common subspace, or alternatively as wedge products between a unitized projective point P and two direction vectors u and v that span the plane as a bivector such that $pl = P \wedge u \wedge v$. In the latter case, the plane pl can be interpreted as bound bivector as suggested by [Browne, 2012]. For this case it is also easy to shift the plane to contain the unitized point $P + x$ instead of P. This can be achieved easily by applying the rules for the wedge product as $pl_{shifted} = P \wedge u \wedge v + x \wedge u \wedge v = (P + x) \wedge u \wedge v$.

The attitude of the plane, i.e. its orientation in space, is given by a bivector that can be calculated using the attitude function $att(pl)$. It is defined for $3D$ projective space as $att(u) = u \vee \overline{\boldsymbol{e}_4}$ and as such can be interpreted as the intersection of the plane with the horizon $H_{3dp} = \overline{\boldsymbol{e}_4}$. It yields the result $att(pl) = t_x \boldsymbol{e}_{23} + t_y \boldsymbol{e}_{31} + t_z \boldsymbol{e}_{12}$.

The definition of a plane as given in eqn. (13) implies that the normal direction n of the plane pl can be recovered from its trivector definition by applying the left weight dual to the plane, such that $n = pl_{\bigstar}$. The left weight dual operation was introduced in eqn. (39). It can also be found in table 11.

The reverse is also true, i.e. if the plane normal n is known, the trivector representing the plane can be directly found by applying the right bulk dual operation to the normal n, such that $pl_O = n^{\star}$. The right bulk dual operation was defined in eqn. (36), and can be found in table 11. This operation yields a plane that contains the origin and has the required orientation with n as normal. The signed distance to the origin could now be given as coefficient t_w to the bulk component $\boldsymbol{e}_{312}$ in case the plane needs to be shifted parallel.

As demonstrated, this enables quite a number of relevant geometric operations by applying rather simple operations to the modelled objects in $3D$ PGA.

1.2.5 Geometric products

Now let's move on in more detail to the geometric products. The multiplicative identity of the geometric product and the wedge product is the scalar basis element $\mathbf{1}$:

$$\mathbf{1} \reflectbox{\wedge} a = a \reflectbox{\wedge} \mathbf{1} = a \tag{46a}$$

$$\mathbf{1} \wedge a = a \wedge \mathbf{1} = a \tag{46b}$$

The multiplicative identity of the regressive geometric product and the regressive wedge product is the pseudoscalar basis element $\mathbb{1} = I_n$ of the respective algebra:

$$\mathbb{1} \vee a = a \vee \mathbb{1} = a \tag{47a}$$
$$\mathbb{1} \vee a = a \vee \mathbb{1} = a \tag{47b}$$

The geometric product combines the dimensions that are present in an object representation, e.g. for $PGA2DP$:

$$\begin{aligned} \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 &= 1 \\ \boldsymbol{e}_2 \wedge \boldsymbol{e}_2 &= 1 \\ \boldsymbol{e}_3 \wedge \boldsymbol{e}_3 &= 0 \end{aligned} \tag{48}$$

The regressive geometric product combines the dimensions that are <u>not</u> present in an object representation, e.g. for $PGA2DP$:

$$\begin{aligned} \overline{\boldsymbol{e}_1} \vee \overline{\boldsymbol{e}_1} &= \mathbb{1} \\ \overline{\boldsymbol{e}_2} \vee \overline{\boldsymbol{e}_2} &= \mathbb{1} \\ \overline{\boldsymbol{e}_3} \vee \overline{\boldsymbol{e}_3} &= 0 \end{aligned} \tag{49}$$

where eqns. (49) can be shown to be equivalent to eqns. (48) by taking the left complement of both sides and applying the complement definition (21a) to it.

The inner product between multivectors A and B is defined as

$$A \cdot B = A^T \mathbf{G} B \tag{50}$$

with $\mathbf{G}$ as the extended metric and using standard matrix multiplication on the right-hand side. It satisfies the identity

$$A \cdot B = \langle A \wedge \tilde{B} \rangle_0 = \langle B \wedge \tilde{A} \rangle_0 \tag{51}$$

where $\wedge$ stands for the geometric product within the grade projection operator $\langle\rangle_k$ for grade k.

Using the various notations used in literature, the contraction is defined as

$$A \rfloor B = A \ll B = A_\star \vee B \quad \text{(left contracton)} \tag{52a}$$
$$B \lfloor A = B \gg A = B \vee A^\star \quad \text{(right contracton)} \tag{52b}$$

with $\star$ denoting the Hodge dual which is formed by the respective metric and complement operation (in spaces of even dimension the left or right complement respectively, and in spaces of odd dimension the metric and complement function regardless of the side of the operand). For blades A and B, they satisfy

$$A \rfloor B = A \ll B = \langle B \barwedge \tilde{A} \rangle_{gr(B)-gr(A)} \tag{53a}$$

$$B \lfloor A = B \gg A = \langle \tilde{A} \barwedge B \rangle_{gr(B)-gr(A)} \tag{53b}$$

and fulfill the requirement that $A \rfloor B = A \lfloor B = A \cdot B$ whenever A and B have the same grade. In this case the contractions reduce to the inner product.

The geometric product between a vector v and a blade B is defined in terms of interior and exterior products, i.e. the right contraction "$\lfloor$", the left contraction "$\rfloor$" and wedge product "$\wedge$" as follows:

$$v \barwedge B = B \lfloor v + v \wedge B \tag{54a}$$

$$B \barwedge v = v \rfloor B + B \wedge v \tag{54b}$$

or sometimes expressed using the right shift "$\gg$" or left shift "$\ll$" operators for the contractions (as implemented in the ga library[5])

$$v \barwedge B = B \gg v + v \wedge B \tag{55a}$$

$$B \barwedge v = v \ll B + B \wedge v \tag{55b}$$

or sometimes by exclusively using the operations of the exterior algebra (wedge product "$\wedge$", bulk dual "$\star$" and regressive wedge product "$\vee$")

$$v \barwedge B = B \vee v^\star + v \wedge B \tag{56a}$$

$$B \barwedge v = v_\star \vee B + B \wedge v \tag{56b}$$

as an alternative. Each one is equivalent.

Be careful, when using the equivalent formulas like the right-hand side of equations (56) when deriving regressive products. Resulting formulas may not be valid! The equivalent formulas for the regressive products must be derived directly from the regressive product table. To give one example, here is the corresponding formula for the regressive geometric product for the case of a regressive product between a vector v and a bivector B:

$$v \barvee B = -B_\star \wedge v + \mathrm{rcmt}(v, B) \tag{57a}$$

$$B \barvee v = -v \wedge B^\star + \mathrm{rcmt}(B, v) \tag{57b}$$

[5]Due to low C++ operator precedence, always make sure to put operator expressions using the shift operators in parentheses! Otherwise, you might get unexpected results.

It is obviously not the result one would get from creating the regressive versions directly from the right-hand side of equations (56). The pre-conditions for the validity of the equivalence between the geometric product and the corresponding right-hand sides might change when taking the complements.

For arguments of equal grade the contractions reduce to the dot product, so that one can write specifically for two vectors u and v:

$$u \barwedge v = v \gg u + u \wedge v = u \cdot v + u \wedge v \tag{58}$$

Every vector v can be decomposed into parts $v_{\parallel}$ parallel to a k-blade B and $v_{\perp}$ perpendicular to B, such that $v = v_{\parallel} + v_{\perp}$. For equation (55a): $B \gg v$ is a $(k-1)$-blade. If $v_{\parallel} \neq 0$ then $B \gg v$ represents a subspace of B. $v \wedge B$ is a $(k+1)$-blade. If $v_{\perp} \neq 0$ then $v \wedge B$ represents $span(v, B)$.

So we get for two vectors u and v, and a bivector B, representing a plane in $3D\ EGA$:

$$v = v_{\parallel} + v_{\perp} \tag{59a}$$

$$(v \barwedge u) \barwedge u^{-1} = \underbrace{(v \cdot u) \barwedge u^{-1}}_{v_{\parallel}=\text{project\_onto(v,u)}} + \underbrace{(v \wedge u) \barwedge u^{-1}}_{v_{\perp}=\text{reject\_from(v,u)}} \tag{59b}$$

$$(v \barwedge B) \barwedge B^{-1} = \underbrace{(B \gg v) \barwedge B^{-1}}_{v_{\parallel}=\text{project\_onto(v,B)}} + \underbrace{(v \wedge B) \barwedge B^{-1}}_{v_{\perp}=\text{reject\_from(v,B)}} \tag{59c}$$

While for $3D\ PGA$ the projection and rejection are directly derived from the orthogonal projection as defined by [Lengyel, 2024b], p. 99f.:

$$\text{project\_onto}(a, b) = \text{ortho\_proj3dp}(a, b) = b \vee (a \wedge b^{\star}) \tag{60}$$

where a could be a point and b could either be a line (a bivector) or plane (a trivector), or a could be a line and b a plane. The rejection is then calculated as $\text{reject\_from}(a, b) = b - \text{project\_onto}(a, b)$.

1.2.6 STA metric, complements, and duality

In this subsection we document what was learned while implementing STA on top of the machinery that was already available from implementing EGA and PGA.

The space-time algebra $STA4DS = \mathbb{G}(1, 3, 0)$ uses the Minkowski signature $(\boldsymbol{g}_1^2, \boldsymbol{g}_2^2, \boldsymbol{g}_3^2, \boldsymbol{g}_4^2) = (-1, -1, -1, +1)$ (the three space-like directions $\boldsymbol{g}_1$, $\boldsymbol{g}_2$, $\boldsymbol{g}_3$

square to −1, the time-like direction $\boldsymbol{g}_4$ squares to +1). The non-Euclidean vector metric introduces a few specific requirements that do not show up in *EGA* (identity metric) or *PGA* (degenerate metric), and that are worth recording carefully.

Extended metric: The only metric input to an algebra is the *vector metric* $\mathfrak{g}$, i.e. the signature itself. The extended metric $\mathbf{G}$ lifts $\mathfrak{g}$ to every grade as the unique wedge exomorphism (25), seeded on the grade-1 basis vectors:

$$\mathbf{G}(\boldsymbol{g}_i) = \mathfrak{g}_{ii} \tag{61a}$$

$$\mathbf{G}(a \wedge b) = \mathbf{G}(a) \wedge \mathbf{G}(b) \quad \Longrightarrow \quad \mathbf{G}(e_S) = \prod_{i \in S} \mathfrak{g}_{ii} \tag{61b}$$

where e_S is the basis blade spanned by the index set S. Write this product $P(e_S) = \prod_{i \in S} \mathfrak{g}_{ii}$. For example, in *STA4DS* the bivector $\boldsymbol{g}_{14}$ has $P(\boldsymbol{g}_{14}) = \mathfrak{g}_{11}\,\mathfrak{g}_{44} = (-1)(+1) = -1$, while $\boldsymbol{g}_{23}$ has $P(\boldsymbol{g}_{23}) = \mathfrak{g}_{22}\,\mathfrak{g}_{33} = (-1)(-1) = +1$. The scalar maps to 1, the pseudoscalar to $det(\mathfrak{g})$, and there is exactly one such inner product on the whole exterior algebra derived from the requirement of an exomorphism.

Note that *the extended metric is independent of how a blade squares under the geometric product.* The extended metric is a property of the *wedge* structure seeded by the signature, not of the geometric product. This is visible in *PGA*: its extended-metric diagonal carries the exomorphism values 1/0 even though its bivectors square to −1 under the geometric product. Physical *characteristics* (space-like/time-like) do not belong in the extended metric: causal character lives in the geometric square (see below), not in the metric. Finally, the extended metric *defines* the dot product and hence *nrm_sq*() at every grade through $\boldsymbol{g}_S \cdot \boldsymbol{g}_S = \mathbf{G}(\boldsymbol{g}_S) = P$. This scalar P is the blade's *reverse-norm*, the scalar part of the blade times its *reverse* $\langle \widetilde{A} \wedge A \rangle_0$.

Metric and regressive metric: Following Lengyel, the metric carries two exomorphisms: the metric $\mathbf{G}$ is a wedge exomorphism, while the regressive metric $\mathbb{G}$ is a regressive wedge exomorphism (28) with $\mathbb{G}(e_S) = \prod_{i \notin S} \mathfrak{g}_{ii}$ (the complementary product). For *every* algebra they are related by the ordinary matrix product $\mathbf{G}\mathbb{G} = det(\mathfrak{g})\, I_n$. For *STA4DS* we have $det(\mathfrak{g}) = (-1)(-1)(-1)(+1) = -1$, and since every basis blade squares to ±1 we get $\mathbf{G}^2 = I_n$, so the relation collapses to $\mathbb{G} = -\mathbf{G}$: the regressive metric is the negated metric. The determinant is signature-parity invariant ($det = (-1)^q$ with q the number of negative directions), so the physically equivalent signatures $(-,-,-,+)$ and $(+,+,+,-)$ both give −1.

Geometric square B^2: The *geometric square* of a blade is $B^2 = \langle u \wedge u \rangle_0$, the blade times itself under the geometric product. It relates to the metric value P only by the reversion sign $\sigma(k) = (-1)^{k(k-1)/2}$ (with k the grade):

$$B^2(e_S) = \sigma(k)\, P(e_S) \tag{62}$$

so $B^2 = P$ at grades $k \equiv 0, 1 \pmod 4$ and $B^2 = -P$ at grades $2, 3$. Two worked examples in $STA4DS$ ($\boldsymbol{g}_1^2 = \boldsymbol{g}_2^2 = \boldsymbol{g}_3^2 = -1$, $\boldsymbol{g}_4^2 = +1$) are

$$\boldsymbol{g}_{14}: \quad P = \boldsymbol{g}_1 \cdot \boldsymbol{g}_4 = -1, \qquad B^2 = \sigma(2)\, P = +1 \;=\; \boldsymbol{g}_{14} \wedge \boldsymbol{g}_{14} \tag{63a}$$

$$\boldsymbol{g}_{23}: \quad P = \boldsymbol{g}_2 \cdot \boldsymbol{g}_3 = +1, \qquad B^2 = \sigma(2)\, P = -1 \;=\; \boldsymbol{g}_{23} \wedge \boldsymbol{g}_{23} \tag{63b}$$

The metric uses the *reverse*-norm, which removes that $\sigma(k)$: the reverse of a grade-k blade is $\widetilde{e}_S = \sigma(k)\, e_S$, so

$$nrm\_sq(e_S) = \langle \widetilde{e}_S \wedge e_S \rangle_0 = \sigma(k)\, \langle e_S \wedge e_S \rangle_0 = \sigma(k)\, B^2 = \sigma(k)^2\, P = P \tag{64}$$

because $\sigma(k)^2 = 1$. A blade thus has two natural "squares" that differ exactly by the reversion sign: the geometric square $B^2 = \sigma(k)\, P$ (blade $\wedge$ blade) and the reverse-norm P (reverse $\wedge$ blade). The reverse-norm behaves like a norm; the dot product, the contraction for equal grades, and $nrm\_sq()$ all use P. The geometric square B^2 is the physically meaningful quantity: a bivector is a *boost* plane when $B^2 > 0$ and a *rotation* plane when $B^2 < 0$, and the rotor exponential splits on its sign ($\cosh / \sinh$ vs. $\cos / \sin$). Causal character — `is_timelike`, `is_spacelike`, `is_lightlike` — therefore reads B^2, *not* $nrm\_sq()$. Keeping B^2 and the metric value P apart avoids encoding "time-like" or "space-like" into the metric. Both squares are tabulated for every basis blade in table 14 below.

Dualization: There are two commonly used ways to dualize a blade or multivector A:

1. *complement after metric* (this library, after Lengyel): $A^\star = \overline{\mathbf{G}A}$ — multiply by the extended metric, then take the purely combinatorial complement. This is the (bulk) dual already defined in (36) and (37).

2. *pseudoscalar multiplication*: $A \wedge I_n$ and $A \wedge I_n^{-1}$.

In a non-degenerate algebra the two relate by reversion, $A^\star = \widetilde{A} \wedge I_n$ (cf. (40a)). So plain $A \wedge I_n$ equals $A^\star$ where $\sigma(k) = +1$ (grades $0, 1, 4$) and $-A^\star$ at grades $2, 3$. Pseudoscalar multiplication needs I_n to be invertible; in $STA4DS$, $I_n \wedge I_n = -1$, so $I_n^{-1} = -I_n$. In a *degenerate* algebra (PGA, and any projective algebra) a null vector sits inside I_n, so $I_n \wedge I_n = 0$ and I_n has no inverse — the scheme is structurally impossible there. The complement-after-metric definition survives because I_n never appears in it; it is the definition used throughout the library for all algebras.

Handedness (left vs. right dual): $STA4DS$ is four-dimensional (even), so the left and right complements/duals differ on the odd grades and agree on the even grades:

$$A_\star = A^\star \ \text{(grades } 0,2,4), \qquad A_\star = -A^\star \ \text{(grades } 1,3) \tag{65}$$

The non-metric complements are exact mutual inverses for any signature, $\underline{(\overline{A})} = \overline{(\underline{A})} = A$, as already stated in (19). Both complements and duals are listed for every basis blade in tables 15 and 16.

Dual round trip: Composing the two metric duals yields a clean global sign, the determinant:

$$(A^\star)_\star = (A_\star)^\star = det(\mathfrak{g})\, A \tag{66}$$

For a Euclidean algebra ($det = +1$) this is the identity — the left dual undoes the right dual. For $STA4DS$ ($det = -1$) it is $-A$: the left dual is the inverse only up to the sign det. The exact inverse applies the metric on the *dual* side instead of the primal side. Since $\mathbf{G}^2 = I_n$, the inverse of $A^\star = \overline{\mathbf{G}A}$ is the *un-dual* (Lengyel: antidual), one per handedness:

$$r\_undual(D) = \underline{\overline{(\underline{D})^\star}} \quad \text{inverts } A^\star\text{: } r\_undual(A^\star) = A \tag{67a}$$

$$l\_undual(D) = \overline{\underline{(\overline{D})_\star}} \quad \text{inverts } A_\star\text{: } l\_undual(A_\star) = A \tag{67b}$$

They coincide on the even grades and differ by a sign on the odd grades. Equivalently $r\_undual = det \cdot (\,\cdot\,)_\star$ and, symmetrically, $l\_undual = det \cdot (\,\cdot\,)^\star$, i.e. $= -(\,\cdot\,)_\star$ and $-(\,\cdot\,)^\star$ respectively for $STA4DS$. The convention-clean statement is therefore "each un-dual undoes the dual of the same handedness": for Euclidean signatures the right un-dual *is* the left dual (and the left un-dual the right dual). The following table summarizes the regime per algebra:

algebra	$det(\mathfrak{g})$	r_undual is	l_undual is
$EGA2D/EGA3D$ (Eucl.)	$+1$	the left dual $(\,\cdot\,)_\star$	the right dual $(\,\cdot\,)^\star$
$STA4DS$ (Minkowski)	-1	the antidual $(-(\,\cdot\,)_\star)$	the antidual $(-(\,\cdot\,)^\star)$
PGA (degenerate)	0	no inverse; bulk/weight split (see below)	

There is also a round trip valid for *every* algebra including PGA: the metric-free complement pair $\underline{(\overline{A})} = \overline{(\underline{A})} = A$, which carries no metric structure.

Signature duality: Because $\mathbb{G} = -\mathbf{G}$, the regressive metric negates every vector square: $\mathbf{G}$ realises the signature $(-,-,-,+)$ while $\mathbb{G}$ realises $(+,+,+,-)$. Each signature is an exomorphism of exactly *one* product — $\mathbf{G}$ of the wedge, $\mathbb{G}$ of the regressive wedge. So, relative to the library's wedge convention, the "space" is the signature whose metric respects the wedge product, $(-,-,-,+)$, and its "dual space" is $(+,+,+,-)$. The whole structure is symmetric under the exchange ($\wedge \leftrightarrow \vee$, $\mathbf{G} \leftrightarrow \mathbb{G}$, space $\leftrightarrow$ dual space): taking the regressive wedge as the primary product (working in the dual algebra) swaps the roles. This is why the two signatures are physically equivalent: each is "the space" for one of the two products, the other being its dual space. Both formulations are actively used to describe phenomena in spacetime research.

Bridge to a projective STA: The statement $\mathbf{G}\mathbb{G} = det(\mathfrak{g})\, I_n$ is universal; the determinant decides the regime. For $STA4DS$ (non-degenerate, $det = -1 \neq 0$) we get $\mathbb{G} = -\mathbf{G}$, a global rescaling with no split. For a potential *projective* STA (degenerate, e.g. $\mathbb{G}(1,3,1)$ with $det = 0$) we would get $\mathbf{G}\mathbb{G} = 0$, so $\mathbf{G}$ and $\mathbb{G}$ have disjoint support — exactly the PGA bulk/weight split, with $A^\star_\bullet = \overline{\mathbf{G}A}$ and $A^\star_\circ = \overline{\mathbb{G}A}$. Pseudoscalar multiplication becomes impossible (no I_n^{-1}), but complement-after-metric can still be used and simply splits in two, as in the case of PGA. The dual is always complement-after-metric; the only question is whether $\mathbf{G}$ and $\mathbb{G}$ are proportional (non-degenerate) or supported on complementary subspaces (degenerate).

Dependency map: The single root is the vector signature $\mathfrak{g}$ (plus the basis and its canonical order). Two independent branches grow from it, and the preceding paragraphs depend on keeping them apart:

- **Combinatorial/signature-direct** (*not* via the extended metric): the wedge product "$\wedge$" (orientation only), the complement $\underline{(\,\cdot\,)}/\overline{(\,\cdot\,)}$ (metric-blind), the regressive wedge product "$\vee$", and the geometric product "$\barwedge$" (which uses the signature directly via $\boldsymbol{g}_i \barwedge \boldsymbol{g}_i = \mathfrak{g}_{ii}$).
- **Extended-metric branch** ($\mathbf{G}$ = wedge exomorphism, seeded by $\mathfrak{g}$): the extended metric and regressive metric ($\mathbf{G}\mathbb{G} = det\, I_n$), from which follow the dot product/$nrm\_sq()$ (the reverse-norm), the left and right duals ($A^\star = \overline{\mathbf{G}A}$), and in turn the contractions, the expansions, and the dual round trip/un-dual.
- **Geometric-square branch** (causal/rotor physics, *not* the metric): the geometric square $B^2 = \langle u \barwedge u \rangle_0 = \sigma(k)\, P$, from which follow causal character (sign(B^2)), the rotor exp / sqrt branch, and the transform $R \barwedge X \barwedge \widetilde{R}$.

What moves when something changes: flipping the signature $(-,-,-,+) \leftrightarrow (+,+,+,-)$ moves both metric and geometric-square branches (but the causal-character labels are signature-parity invariant); adding a null direction (projective STA, $det = 0$) changes only the extended-metric branch (the dual splits into bulk/weight) and leaves the geometric-square branch untouched; changing the complement convention moves the complements, the dual, the regressive wedge product "$\vee$", and the contractions.

1.2.7 Transwedge products

[Lengyel, 2025] has shown that the geometric product can be derived using operations from Grassmann algebra exclusively. These are the wedge product, the complement and the duality operations. *This demonstrates that the geometric product is not a fundamental operation in its own right, but can be derived from more fundamental operations defined in Grassmann algebra.*

For that purpose Lengyel introduced the transwedge product of order k by following definition:

$$a \underset{k}{\Lsh} b = \sum_{c \in \mathcal{B}_k} (\underline{c} \vee a) \wedge (b \vee c^\star) \tag{68}$$

where $\mathcal{B}_k$ is the set of all basis elements of order k. For $EGA3D$ it means for example $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3\}$, $\mathcal{B}_2 = \{\boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}\}$, and $\mathcal{B}_3 = \{\boldsymbol{e}_{123}\}$. From these expressions the geometric product can be formed by

$$a \wedge b = \sum_{k=0}^{n} (-1)^{k(k-1)/2} a \underset{k}{\Lsh} b \tag{69}$$

For $k = 0$ this yields the outer product (wedge product). For higher values of k other products are generated which sum up to the full geometric product. Thus, the transwedge products helps to decompose the geometric product. The most interesting for applications are the wedge product resulting for $k = 0$ and the transwedge product of order $k = 1$.

The regressive wedge product in equation (68) can be replaced by the wedge product using the complement-based definitions given in equations (19) to (23b). That means for example that equation (68) can be transformed to the equivalent expression

$$a \underset{k}{\Lsh} b = \sum_{c \in \mathcal{B}_k} (\underline{c \wedge \overline{a}}) \wedge (\underline{\overline{b} \wedge \overline{c^\star}}) \tag{70}$$

expressing it by using the wedge product exclusively.

Since every product can be assigned a corresponding regressive product in analogy to definition (21a) or (21b), the transwedge product (68) also has a regressive form:

$$a \underset{k}{\curlyvee} b \equiv \overline{\underline{a} \underset{k}{\curlywedge} \underline{b}} = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c} \vee \underline{a}) \wedge (\underline{b} \vee c^\star)} \tag{71}$$

which in turn can be expressed in terms of the wedge product exclusively as:

$$a \underset{k}{\curlyvee} b = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c} \wedge \underline{a}) \wedge (\underline{b} \wedge \overline{c^\star})} \tag{72}$$

Equations (70) and (72) are used for calculation of the product tables of the (regressive) transwedge products of order k and the resulting (regressive) geometric products in `ga/ga_prdxpr`.

The transwedge product is very useful to show that the geometric product can be derived from primitives of Grassmann algebra. When (regressive) transwedge product expressions are calculated, it can be shown that each transwedge product and the resulting coefficient expressions are equivalent to specific combinations of wedge, commutator or dot products, and their regressive counterparts. This was done in `ga/src_prdxpr/ga_pga2dp_ops_products.hpp` for the (regressive) transwedge product of order $k = 1$ and as well in `ga/src_prdxpr/ga_pga3dp_ops_products.hpp` to show the equivalence.

1.2.8 Complements, duals, and reverses in *EGA*

Left and right complements defined as $\underline{u} \wedge u = u \wedge \overline{u} = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge ... \wedge \boldsymbol{e}_n = I_n$. Complements are independent of the metric. They just depend on presence or absence of a basis vector in a blade. The complement provides the missing basis vectors required to fill the complete space with the wedge product, i.e. to fill the pseudoscalar I_n of the respective algebra.

Left and right (metric) duals are defined in a similar way, but the complement operation is done *after* the basis vector was multiplied with the metric $\mathbf{G}$ of the space in regular matric multiplication. The left dual is defined as $(\underline{\mathbf{G}u}) \wedge u = u_\star \wedge u = I_n$ and the right dual is defined as $u \wedge (\overline{\mathbf{G}u}) = u \wedge u^\star = I_n$.

Complements and duals are indistinguishable in *EGA* since the metric is the identity matrix. However, in *PGA* they differ since the metric selects certain basis vectors that are no longer present after the matrix multiplication with the metric (see subsection 1.2.9).

In spaces of odd dimension left and right complements deliver the same results and thus are replaced by the complement or dual operation regardless whether we consider multiplying from the left or right side with the wedge product. Therefore, no left complement and left dual are given in the table for 3*D EGA* below.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_{12} = \mathbb{1}$
$\overline{\boldsymbol{u}}$, $\boldsymbol{u}^\star$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_2$	$-\boldsymbol{e}_1$	$\mathbf{1}$
$\underline{\boldsymbol{u}}$, $\boldsymbol{u}_\star$	$\boldsymbol{e}_{12}$	$-\boldsymbol{e}_2$	$\boldsymbol{e}_1$	$\mathbf{1}$

Table 1: Right and left complements, as well as right and left duals in 2*D EGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_{12} = \mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$-\boldsymbol{e}_{12}$

Table 2: Reverse in 2*D EGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{123} = \mathbb{1}$
$\overline{\boldsymbol{u}}$, $\boldsymbol{u}^\star$	$\boldsymbol{e}_{123}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\mathbf{1}$

Table 3: Complements and duals in 3*D EGA* for every basis element. Complements and duals are the same in *EGA* since the metric is the identity matrix.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{123} = \mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{123}$

Table 4: Reverse in 3*D EGA* for every basis element.

1.2.9 Dot products, complements, duals, and reverses in *PGA*

Note: *All* regressive dot products (symbol "$\circ$") and *weight duals of scalar* **1** (symbols "*", "$_\star$") get a pseudoscalar zero for statical type correctness in C++ (also written here as 0, but with actual type `pscalar2dp` ($\mathbb{1} = \boldsymbol{e}_{321}$) or `pscalar3dp` ($\mathbb{1} = \boldsymbol{e}_{1234}$)).

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{321} = \mathbb{1}$
$\boldsymbol{u} \cdot \boldsymbol{u}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	$\mathbf{1}$	0
$\boldsymbol{u} \circ \boldsymbol{u}$	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	$\mathbb{1}$

Table 5: Dot and regressive dot products in $2D$ *PGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{321} = \mathbb{1}$
$\overline{\boldsymbol{u}}$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{32}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_2$	$\boldsymbol{e}_1$	$-\boldsymbol{e}_3$	$\mathbf{1}$

Table 6: Complements in $2D$ *PGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{321} = \mathbb{1}$
$\boldsymbol{u}^\star$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{32}$	$-\boldsymbol{e}_{31}$	0	0	0	$-\boldsymbol{e}_3$	0
$\boldsymbol{u}^*$	0	0	0	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_2$	$\boldsymbol{e}_1$	0	$\mathbf{1}$

Table 7: Bulk duals and weight duals for every basis element (=complement applied to bulk and weight).

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{321} = \mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{32}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{321}$
$\underset{\sim}{\boldsymbol{u}}$	$\mathbf{-1}$	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{321}$

Table 8: Reverse and regressive reverse in $2D$ *PGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{1234} = \mathbb{1}$
$\boldsymbol{u} \cdot \boldsymbol{u}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	0	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	$\mathbf{1}$	0
$\boldsymbol{u} \circ \boldsymbol{u}$	0	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	$\mathbb{1}$

Table 9: Dot and regressive dot products in $3D$ *PGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{1234} = \mathbb{1}$
$\overline{\boldsymbol{u}}$	$\boldsymbol{e}_{1234}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	$-\boldsymbol{e}_4$	$\mathbf{1}$
$\underset{\sim}{\boldsymbol{u}}$	$\boldsymbol{e}_{1234}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	$-\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\mathbf{1}$

Table 10: Right and left complements in $3D$ *PGA* for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{1234} = \mathbb{1}$
$\boldsymbol{u}^\star$	$\boldsymbol{e}_{1234}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	0	0	0	0	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	0	0	0	$-\boldsymbol{e}_4$	0
$\boldsymbol{u}_\star$	$\boldsymbol{e}_{1234}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	0	0	0	0	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	0	0	0	$\boldsymbol{e}_4$	0
$\boldsymbol{u}^*$	0	0	0	0	$\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	0	0	0	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	0	$\mathbf{1}$
$\boldsymbol{u}_*$	0	0	0	0	$-\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	0	0	0	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	0	$\mathbf{1}$

Table 11: Right/left bulk duals and right/left weight duals in $3D$ *PGA* for every basis element (=complement applied to bulk and weight).

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{1234} = \mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	$-\boldsymbol{e}_{321}$	$\boldsymbol{e}_{1234}$
$\underset{\sim}{\boldsymbol{u}}$	$\mathbf{1}$	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	$-\boldsymbol{e}_4$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\boldsymbol{e}_{1234}$

Table 12: Reverse and regressive reverse in $3D$ *PGA* for every basis element.

1.2.10 Solving basic geometric tasks in *PGA*

To demonstrate the benefit of using the *PGA* approach vs. the classical vector algebra approach let us look into very basic recurring geometric tasks that would partially involve solving system of equations in the classical approach. They come as simple product expressions in *PGA*.

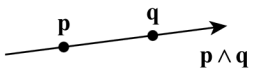
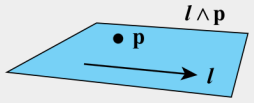
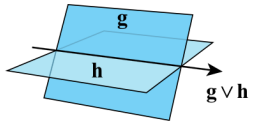
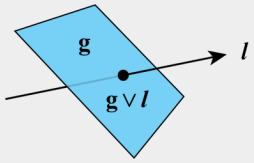
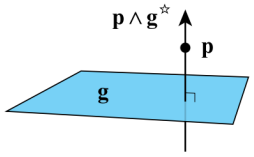
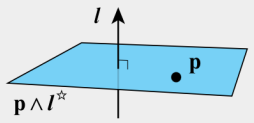
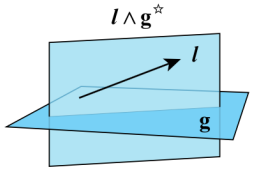
Operation	Illustration
$\mathbf{p} \wedge \mathbf{q}$ Line containing points $\mathbf{p}$ and $\mathbf{q}$.	
$\boldsymbol{l} \wedge \mathbf{p}$ Plane containing line $\boldsymbol{l}$ and point $\mathbf{p}$.	
$\mathbf{g} \vee \mathbf{h}$ Line where planes $\mathbf{g}$ and $\mathbf{h}$ intersect.	
$\mathbf{g} \vee \boldsymbol{l}$ Point where plane $\mathbf{g}$ and line $\boldsymbol{l}$ intersect.	
$\mathbf{p} \wedge \mathbf{g}^{\star}$ Line containing point $\mathbf{p}$ and orthogonal to plane $\mathbf{g}$.	
$\mathbf{p} \wedge \boldsymbol{l}^{\star}$ Plane containing point $\mathbf{p}$ and orthogonal to line $\boldsymbol{l}$.	
$\boldsymbol{l} \wedge \mathbf{g}^{\star}$ Plane containing line $\boldsymbol{l}$ and orthogonal to plane $\mathbf{g}$.	

Figure 1: Basic tasks addressed with *PGA* by using product expressions without need to solve equations. (Source: https://terathon.com/blog/transwedge-product.html, Eric Lengyel, 2025.)

1.2.11 Comparison of approaches in different algebras

Application		**Description**
Create line $\boldsymbol{l}$ from points P, Q	VA	$P = (p_x, p_y, p_z)$ $\boldsymbol{l}: \vec{x}(t) = (Q - P) \cdot t + P$ (explicit), $ax + by = c$ (implict)
	EGA	$P = (p_x, p_y, p_z)$, a vector represents a $1D$ subspace $\boldsymbol{l}: x(t) = P \cdot t$ (explicit, line through origin and P) $\boldsymbol{l}: x(t) = (Q - P) \cdot t + P$ (explicit, line in direction $Q - P$ through P) $\boldsymbol{l}: (x - P) \wedge (Q - P) = 0$ (implicit, line in direction $Q - P$ through P)
	PGA	$P = (p_x, p_y, p_z, 1)$ $\boldsymbol{l} = P \wedge Q$ (line through points P and Q, bivector)
Create plane $\boldsymbol{T}$ from points P, Q, R	VA	$\vec{u} = Q - P$, $\vec{v} = R - P$ $\boldsymbol{T}: x(s,t) = \vec{u} \cdot s + \vec{v} \cdot t + P$
	EGA	$u = Q - P$, $v = R - P$, $B = u \wedge v$ $\boldsymbol{T} = P \wedge Q$ (plane through O, P, Q; bivector) $\boldsymbol{T}: (x - P) \wedge B = 0$ (plane through P, Q, R; bivector)
	PGA	$\boldsymbol{T} = P \wedge Q \wedge R$ (plane through P, Q, R, trivector)
Intersect two lines $\boldsymbol{l_1}$, $\boldsymbol{l_2}$	VA	$\boldsymbol{l_1}: \vec{x}(s) = \vec{u} \cdot s + P$, $\boldsymbol{l_2}: \vec{y}(t) = \vec{v} \cdot t + Q$, solve system in $2D$ for s_i and t_i: $\vec{u} \cdot s + P = \vec{v} \cdot t + Q$ intersection point $I = \vec{x}(s_i) = \vec{y}(t_i)$
	EGA	see VA, # of solutions: 1, none or infinitely many
	PGA	Intersect arbitrary lines l_1, l_2 in a common plane intersection point I: $I = l_1 \vee l_2$ (projective point)
Intersect two planes $\boldsymbol{T_1}$, $\boldsymbol{T_2}$	VA	$x(s,t) = \vec{u} \cdot s + \vec{v} \cdot t + P$, $y(m,n) = \vec{m} \cdot p + \vec{n} \cdot q + Q$ solve system in $3D$: $\vec{u} \cdot s + \vec{v} \cdot t + P = \vec{m} \cdot p + \vec{n} \cdot q + Q$
	EGA	see VA
	PGA	Intersect arbitrary planes T_1, T_2 intersection line I: $I = T_1 \vee T_2$ (projective line, bivector)

Table 13: Comparison of common geometric construction tasks across three approaches. **VA** = Vector Algebra (classical approach using vectors and parametric equations); **EGA** = Euclidean Geometric Algebra; **PGA** = Projective (Euclidean) Geometric Algebra.

PGA can define geometric primitives like lines or planes directly from points. It does not require solving systems of equations for simple geometrical tasks, like the intersection of geometric objects. They can be solved easily by using a suitable product expression. Another key advantage is that the intersections, e.g. of two lines or a line and a plane always have a solution in projective geometric algebra. There is no need to distinguish different cases, as required in *VA* or *EGA*, where one might get either none, one or infinitely many solutions depending on the specific geometric case. The intersection point might be at infinity, but in projective algebra it is always well-defined.

1.2.12 Dot products, complements, duals, and reverses in *STA*

The tables below collect the per-basis-element values for the space-time algebra $STA4DS = \mathbb{G}(1,3,0)$. The Minkowski signature is $(\boldsymbol{g}_1^2, \boldsymbol{g}_2^2, \boldsymbol{g}_3^2, \boldsymbol{g}_4^2) = (-1,-1,-1,+1)$. They are the space-time analogue of the EGA and PGA tables above; the underlying concepts (extended metric as wedge exomorphism, the reverse-norm P versus the geometric square B^2, left/right complements and duals) were developed in subsection 1.2.6. $STA4DS$ is four-dimensional (even), so left and right complements/duals are distinguished; it is non-degenerate ($det(\mathfrak{g}) = -1$), so — unlike PGA — there is *no* bulk/weight split, and every basis blade squares to ±1.

Table 14 lists two distinct "squares" side by side: the dot product $\boldsymbol{u} \cdot \boldsymbol{u} = \mathbf{G}(\boldsymbol{u}) = P$ (the extended-metric reverse-norm, equal to $nrm\_sq(\boldsymbol{u})$), and the geometric square $\boldsymbol{u} \wedge \boldsymbol{u} = B^2 = \sigma(k)\, P$. They agree at grades $0, 1, 4$ and differ by sign at grades $2, 3$ — the distinction that drives causal character and the rotor exponential (see eqn. (62)).

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$\boldsymbol{g}_4$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$\boldsymbol{g}_{23}$	$\boldsymbol{g}_{31}$	$\boldsymbol{g}_{12}$	$\boldsymbol{g}_{234}$	$\boldsymbol{g}_{314}$	$\boldsymbol{g}_{124}$	$\boldsymbol{g}_{123}$	$\boldsymbol{g}_{1234} = \mathbb{1}$
$\boldsymbol{u} \cdot \boldsymbol{u}$	1	−1	−1	−1	1	−1	−1	−1	1	1	1	1	1	1	−1	−1
$\boldsymbol{u} \wedge \boldsymbol{u}$	1	−1	−1	−1	1	1	1	1	−1	−1	−1	−1	−1	−1	1	−1

Table 14: Reverse-norm $\boldsymbol{u} \cdot \boldsymbol{u} = P = nrm\_sq(\boldsymbol{u})$ (extended metric) and geometric square $\boldsymbol{u} \wedge \boldsymbol{u} = B^2 = \sigma(k)\, P$ in $STA4DS$ for every basis element. They differ by sign at grades 2 and 3.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$\boldsymbol{g}_4$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$\boldsymbol{g}_{23}$	$\boldsymbol{g}_{31}$	$\boldsymbol{g}_{12}$	$\boldsymbol{g}_{234}$	$\boldsymbol{g}_{314}$	$\boldsymbol{g}_{124}$	$\boldsymbol{g}_{123}$	$\boldsymbol{g}_{1234} = \mathbb{1}$
$\overline{\boldsymbol{u}}$	$\boldsymbol{g}_{1234}$	$\boldsymbol{g}_{234}$	$\boldsymbol{g}_{314}$	$\boldsymbol{g}_{124}$	$-\boldsymbol{g}_{123}$	$\boldsymbol{g}_{23}$	$\boldsymbol{g}_{31}$	$\boldsymbol{g}_{12}$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$-\boldsymbol{g}_1$	$-\boldsymbol{g}_2$	$-\boldsymbol{g}_3$	$\boldsymbol{g}_4$	1
$\underline{\boldsymbol{u}}$	$\boldsymbol{g}_{1234}$	$-\boldsymbol{g}_{234}$	$-\boldsymbol{g}_{314}$	$-\boldsymbol{g}_{124}$	$\boldsymbol{g}_{123}$	$\boldsymbol{g}_{23}$	$\boldsymbol{g}_{31}$	$\boldsymbol{g}_{12}$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$-\boldsymbol{g}_4$	1

Table 15: Right and left complements in $STA4DS$ for every basis element. Complements are independent of the metric.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$\boldsymbol{g}_4$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$\boldsymbol{g}_{23}$	$\boldsymbol{g}_{31}$	$\boldsymbol{g}_{12}$	$\boldsymbol{g}_{234}$	$\boldsymbol{g}_{314}$	$\boldsymbol{g}_{124}$	$\boldsymbol{g}_{123}$	$\boldsymbol{g}_{1234} = \mathbb{1}$
$\boldsymbol{u}^\star$	$\boldsymbol{g}_{1234}$	$-\boldsymbol{g}_{234}$	$-\boldsymbol{g}_{314}$	$-\boldsymbol{g}_{124}$	$-\boldsymbol{g}_{123}$	$-\boldsymbol{g}_{23}$	$-\boldsymbol{g}_{31}$	$-\boldsymbol{g}_{12}$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$-\boldsymbol{g}_1$	$-\boldsymbol{g}_2$	$-\boldsymbol{g}_3$	$-\boldsymbol{g}_4$	−1
$\boldsymbol{u}_\star$	$\boldsymbol{g}_{1234}$	$\boldsymbol{g}_{234}$	$\boldsymbol{g}_{314}$	$\boldsymbol{g}_{124}$	$\boldsymbol{g}_{123}$	$-\boldsymbol{g}_{23}$	$-\boldsymbol{g}_{31}$	$-\boldsymbol{g}_{12}$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$\boldsymbol{g}_4$	−1

Table 16: Right and left (metric) duals in $STA4DS$ for every basis element ($\boldsymbol{u}^\star = \overline{\mathbf{G}\boldsymbol{u}}$, $\boldsymbol{u}_\star = \underline{\mathbf{G}\boldsymbol{u}}$). Unlike in EGA they differ from the complements, since the Minkowski metric flips signs.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$\boldsymbol{g}_4$	$\boldsymbol{g}_{14}$	$\boldsymbol{g}_{24}$	$\boldsymbol{g}_{34}$	$\boldsymbol{g}_{23}$	$\boldsymbol{g}_{31}$	$\boldsymbol{g}_{12}$	$\boldsymbol{g}_{234}$	$\boldsymbol{g}_{314}$	$\boldsymbol{g}_{124}$	$\boldsymbol{g}_{123}$	$\boldsymbol{g}_{1234} = \mathbb{1}$
$\widetilde{\boldsymbol{u}}$	1	$\boldsymbol{g}_1$	$\boldsymbol{g}_2$	$\boldsymbol{g}_3$	$\boldsymbol{g}_4$	$-\boldsymbol{g}_{14}$	$-\boldsymbol{g}_{24}$	$-\boldsymbol{g}_{34}$	$-\boldsymbol{g}_{23}$	$-\boldsymbol{g}_{31}$	$-\boldsymbol{g}_{12}$	$-\boldsymbol{g}_{234}$	$-\boldsymbol{g}_{314}$	$-\boldsymbol{g}_{124}$	$-\boldsymbol{g}_{123}$	$\boldsymbol{g}_{1234}$

Table 17: Reverse in $STA4DS$ for every basis element. The reverse sign is the grade-only factor $\sigma(k) = (-1)^{k(k-1)/2}$, independent of the metric.

1.3 Modelling motion

For translational motion in physical contexts we typically use the following naming convention for scalars or vectors:

$$\begin{aligned} s(t) &= s \\ \dot{s}(t) &= v(t) = \dot{s} = v \\ \ddot{s}(t) &= \dot{v}(t) = a(t) = \ddot{s} = \dot{v} = a \end{aligned}$$

while for rotational motion we typically use:

$$\begin{aligned} \phi(t) &= \phi \\ \dot{\phi}(t) &= \omega(t) = \dot{\phi} = \omega \\ \ddot{\phi}(t) &= \dot{\omega}(t) = \ddot{\phi} = \dot{\omega} \end{aligned}$$

Especially in *PGA*, we will be able to combine the translational and rotational aspects in one physical variable. Thus, we will use a convention of using capital letters for physical quantities to distinguish them from the classical convention, especially so for bivectors and even-grade multivectors that do not occur in classical descriptions. We will use in *PGA*:

$B(t) = B$	(bivector)
$\dot{B}(t) = \Omega(t) = \dot{B} = \Omega$	(rate of change of bivector, "bivector speed")
$\ddot{B}(t) = \dot{\Omega}(t) = \ddot{B} = \dot{\Omega}$	(rate of change of bivector speed)

In order to model motion in *PGA* we need to work with the exponential function applying it in the context of the geometric product (symbol "$\wedge$") and the regressive geometric product (symbol "$\vee$").

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \cdots \tag{73}$$

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \frac{x^{10}}{10!} + \frac{x^{12}}{12!} - \frac{x^{14}}{14!} + \cdots \tag{74}$$

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \frac{x^{11}}{11!} + \frac{x^{13}}{13!} - \frac{x^{15}}{15!} + \cdots \tag{75}$$

If we substitute $x = i\phi$ with i as the imaginary unit ($i^2 = -1$), we get Euler's formula:

$$\begin{aligned}
\exp(i\phi) &= \sum_{n=0}^{\infty} \frac{(i\phi)^n}{n!} \\
&= 1 + i\phi + \frac{(i\phi)^2}{2!} + \frac{(i\phi)^3}{3!} + \frac{(i\phi)^4}{4!} + \frac{(i\phi)^5}{5!} + \frac{(i\phi)^6}{6!} + \frac{(i\phi)^7}{7!} + \cdots \qquad (76) \\
&= 1 + i\phi - \frac{\phi^2}{2!} - i\frac{\phi^3}{3!} + \frac{\phi^4}{4!} + i\frac{\phi^5}{5!} - \frac{\phi^6}{6!} - i\frac{\phi^7}{7!} + \cdots \\
&= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots + i(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots) \qquad (77)
\end{aligned}$$

$$\exp(i\phi) = \cos(\phi) + i\sin(\phi) \qquad (78)$$

Applying this for geometric algebra we have to distinguish between calculating the powers in the Taylor series using the geometric product (in EGA) and using the regressive geometric product (in PGA). For the rotation case we assume a unitized bivector B and a scalar ϕ. Based on the geometric product we know that $B \wedge B = -\mathbf{1}$, and for its regressive counterpart we know that $B \vee B = -\mathbb{1}$. Thus, B plays a role equivalent to i in both cases. In addition, taking any value to the power of 0 results in the neutral element of the product, which is $\mathbf{1}$ for the geometric product, and $\mathbb{1}$ for the regressive geometric product. Taking the powers then works like this:

$$\begin{aligned}
&\cdots \\
\frac{(B\phi)^2_{\wedge}}{2!} &= \frac{1}{2!}(B\phi \wedge B\phi) = \frac{\phi^2}{2!}(B \wedge B) = \frac{\phi^2}{2!}(-\mathbf{1}) = -\frac{\phi^2}{2!} && (79a) \\
\frac{(B\phi)^3_{\wedge}}{3!} &= \frac{\phi^3}{3!}(B \wedge B \wedge B) = \frac{\phi^3}{3!}(-\mathbf{1} \wedge B) = -\frac{\phi^3}{3!}B && (79b) \\
\frac{(B\phi)^4_{\wedge}}{4!} &= \frac{\phi^4}{4!}(B \wedge B \wedge B \wedge B) = \frac{\phi^4}{4!}(-\mathbf{1} \wedge -\mathbf{1}) = \frac{\phi^4}{4!} && (79c) \\
&\cdots \\
\frac{(B\phi)^2_{\vee}}{2!} &= \frac{1}{2!}(B\phi \vee B\phi) = \frac{\phi^2}{2!}(B \vee B) = \frac{\phi^2}{2!}(-\mathbb{1}) = -\frac{\phi^2}{2!}\mathbb{1} && (79d) \\
\frac{(B\phi)^3_{\vee}}{3!} &= \frac{\phi^3}{3!}B \vee B \vee B = \frac{\phi^3}{3!}(-\mathbb{1} \vee B) = -\frac{\phi^3}{3!}B && (79e) \\
\frac{(B\phi)^4_{\vee}}{4!} &= \frac{\phi^4}{4!}B \vee B \vee B \vee B = \frac{\phi^4}{4!}(-\mathbb{1} \vee -\mathbb{1}) = \frac{\phi^4}{4!}\mathbb{1} && (79f) \\
&\cdots
\end{aligned}$$

Thus, the result of exponentiating a bivector B with respect to the geometric

product is (for EGA):

$$\begin{aligned}\exp_{\wedge}(B\phi) &= \sum_{n=0}^{\infty} \frac{(B\phi)^n_{\wedge}}{n!} \\ &= 1 + B\phi + \frac{(B\phi)^2_{\wedge}}{2!} + \frac{(B\phi)^3_{\wedge}}{3!} + \frac{(B\phi)^4_{\wedge}}{4!} + \frac{(B\phi)^5_{\wedge}}{5!} + \frac{(B\phi)^6_{\wedge}}{6!} + \cdots \\ &= 1 + B\phi - \frac{\phi^2}{2!} - B\frac{\phi^3}{3!} + \frac{\phi^4}{4!} + B\frac{\phi^5}{5!} - \frac{\phi^6}{6!} - B\frac{\phi^7}{7!} + \cdots \\ &= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots + B(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots)\end{aligned} \tag{80}$$

$$\exp_{\wedge}(B\phi) = \cos(\phi) + \sin(\phi)B \tag{81}$$

The last step of moving B after the brackets is allowed, since scalar factors commute with any object using the regular product. The result structurally is an even-grade multivector with a scalar and a bivector component. We get the correct sign and angle of rotation if we replace ϕ by $-\frac{1}{2}\phi$, which was not considered above to keep the formulas as simple as possible. The substitution yields the final form of a rotor R in EGA with B as unitized bivector and ϕ the rotation angle:

$$\begin{aligned}R = \exp_{\wedge}(-\frac{\phi}{2}B) &= \cos(-\frac{\phi}{2}) + \sin(-\frac{\phi}{2})B \\ R &= \cos(\frac{\phi}{2}) - \sin(\frac{\phi}{2})B\end{aligned} \tag{82}$$

For the case using the regressive geometric product it becomes (for PGA):

$$\begin{aligned}\exp_{\vee}(B\phi) &= \sum_{n=0}^{\infty} \frac{(B\phi)^n_{\vee}}{n!} \\ &= \mathbb{1} + B\phi + \frac{(B\phi)^2_{\vee}}{2!} + \frac{(B\phi)^3_{\vee}}{3!} + \frac{(B\phi)^4_{\vee}}{4!} + \frac{(B\phi)^5_{\vee}}{5!} + \frac{(B\phi)^6_{\vee}}{6!} + \cdots \\ &= \mathbb{1} + B\phi - \mathbb{1}\frac{\phi^2}{2!} - B\frac{\phi^3}{3!} + \mathbb{1}\frac{\phi^4}{4!} + B\frac{\phi^5}{5!} - \mathbb{1}\frac{\phi^6}{6!} - B\frac{\phi^7}{7!} + \cdots \\ &= \mathbb{1}(1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots) + B(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots)\end{aligned} \tag{83}$$

$$\exp_{\vee}(B\phi) = \cos(\phi)\mathbb{1} + \sin(\phi)B \tag{84}$$

The last step of moving $\mathbb{1}$ and B after the brackets is allowed, since scalar factors, like the results of the trigonometric functions, commute with any object using the regular product. The resulting motor structurally is an even-grade

multivector with a bivector and a pseudoscalar of grade 4 for $3D\ PGA$.

To create a rotor that generates a rotation of angle ϕ we use

$$M = \exp_{\vee}\left(\frac{\phi}{2}B\right) \tag{85}$$

where the factor $\frac{1}{2}$ is needed to create a resulting angle ϕ through the double-sided sandwich product. Typically, in applications the angle ϕ is directly contained as a scalar factor in the bivector B while the factor $\frac{1}{2}$ is kept explicit.

In $2D\ PGA$ we have the same expression, but the argument of the exponential to create a motor for rotation is not a bivector B, but a unitized point P. Here $P \vee P = -\mathbb{1}$ is valid, and equation (84) becomes $\exp_{\vee}(P\phi) = \cos(\phi)\mathbb{1} + \sin(\phi)P$ instead, which leads to a motor M that structurally is an odd-grade multivector with a vector and a pseudoscalar of grade 3. The resulting rotation direction of the motor depends on the chosen pseudoscalar. The pseudoscalar for $PGA2DP$ was chosen as $\boldsymbol{e}_{321}$ in order to make signs of rotation angles (and thus turning directions) consistent between $2D$ and $3D$ PGA. For the actual motor we get

$$M = \exp_{\vee}\left(\frac{\phi}{2}P\right) \tag{86}$$

in this case.

To model a translation $\delta = (\delta_x, \delta_y, \delta_z, 0)$ in $3D\ PGA$ we use a special bivector $B = B_{tra}$ that does not square to $-\mathbb{1}$, but instead squares to zero. That makes sure that the expansion of the Taylor series of the exponential function $\exp_{\vee}(B_{tra})$ only contains the first two terms. The bivector is resulting from two consecutive reflections across parallel planes perpendicular to δ with the distance $\frac{1}{2}\|\delta\|_{\bullet}$ relative to each other. They intersect in a line at infinity that is used as "rotation axis". This however yields a translation, since the axis is infinitely far away. The derivation of the formula for the bivector will be done later in this chapter (see eqn. (126)). We use $B_{tra} = \delta^{\star} \vee \overline{\boldsymbol{e}_4} = \delta^{\star} \vee H_{3dp} = att(\delta^{\star})$. This yields $B_{tra} = (\delta_x \boldsymbol{e}_{23} + \delta_y \boldsymbol{e}_{31} + \delta_z \boldsymbol{e}_{12})$. The resulting motor is $M = \exp_{\vee}(\frac{1}{2}B_{tra}) = \mathbb{1} + \frac{1}{2}B_{tra} = \mathbb{1} + \frac{1}{2}(0, 0, 0, \delta_x, \delta_y, \delta_z)$. The motor M is an even-grade multivector containing a pseudoscalar component, which is a quadvector in $3D\ PGA$, and a bivector component.

To model a translation $\delta = (\delta_x, \delta_y, 0)$ in $2D\ PGA$ we need a special vector as argument of the exponential. It represents a direction to a point at infinity.

The "rotation" around that infinitely far away point generates a translation (actually it is resulting from two consecutive reflections across parallel lines perpendicular to δ with the distance $\frac{1}{2}\|\delta\|_{\bullet}$ relative to each other). The derivation of the special vector will be done later in this section (see eqn. (133)). The argument of the exponential $\exp_{\vee}$ for a translation is the direction vector $P = \delta^{\star} \vee \overline{\boldsymbol{e}_3} = \delta^{\star} \vee H_{2dp} = att(\delta^{\star}) = (-\delta_y \boldsymbol{e}_1 + \delta_x \boldsymbol{e}_2)$. P points in a direction perpendicular to the translation δ towards infinity. The main difference of this special point P compared to the one used for the rotation above is that $P_z = 0$. This results in $P \vee P = 0$ which causes the expansion of the exponential to only contain the first two terms: $M = \exp_{\vee}(\frac{1}{2}P) = \mathbb{1} + \frac{1}{2}P = \mathbb{1} + \frac{1}{2}(-\delta_y, \delta_x, 0)$. The motor M is an odd-grade multivector containing a pseudoscalar, which is a trivector in $2D$ PGA, and a vector component, i.e. it is structurally the same as for the rotation case in $2D$ PGA above.

Eqn. (81) allows us to model rotations in EGA, where the bivector B is the complement of the axis of rotation and $\theta = -2\phi$ is the angle of the rotation around that axis. If we directly want to specify the angle of rotation θ we need to substitute ϕ by $-\frac{\phi}{2}$ in eqn. (81). The rotated object X can then be obtained from the original object X_0 by:

$$X = R \wedge X_0 \wedge \widetilde{R} = \exp_{\wedge}(-\frac{\phi}{2}B) \wedge X_0 \wedge \exp_{\wedge}(\frac{\phi}{2}B) \quad (\text{for } EGA) \tag{87}$$

If we want to model combined translations and rotations in PGA, we need to look into another extension of the exponential function. We follow the approach defined by [Lengyel, 2024b] in chapter 3.2 and 3.6.2, where it is shown that a general motor in PGA can be expressed as a screw motion, i.e. a rotation around a unitized line $\boldsymbol{l}$ by an angle ϕ and a translation along $\boldsymbol{l}$ by a displacement δ as

$$\begin{aligned} M = \exp_{\vee}\left[(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee \boldsymbol{l}\right] &= \cos_{\vee}(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) + \sin_{\vee}(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee \boldsymbol{l} \\ &= -\frac{\delta}{2}\mathbf{1}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) + \left(\frac{\delta}{2}\mathbf{1}\cos(\frac{\phi}{2}) + \mathbb{1}\sin(\frac{\phi}{2})\right) \vee \boldsymbol{l} \\ M &= -\frac{\delta}{2}\mathbf{1}\sin(\frac{\phi}{2}) - \frac{\delta}{2}\boldsymbol{l}^{\star}\cos(\frac{\phi}{2}) + \boldsymbol{l}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) \end{aligned} \tag{88}$$

where M structurally is an even-grade multivector with scalar, bivector and pseudoscalar parts in $3D$ PGA.

For pure rotational motion with $\delta = 0$ and $\phi \neq 0$ we get

$$M_{rot} = \boldsymbol{l}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) \tag{89}$$

while for pure translational motion with $\delta \neq 0$ and $\phi = 0$ we get

$$M_{tra} = -\frac{\delta}{2}\boldsymbol{l}^{\star} + \mathbb{1} \tag{90}$$

The argument in the exponential function in eqn. (88) for all cases results in a bivector. This can be derived from inspecting the product table for the regressive geometric product. However, this time we only have a common factor of $\frac{1}{2}$, and not $-\frac{1}{2}$ as in EGA as required argument of the exponential function. This will impact a sign for the transformation in the regressive case for PGA compared to the corresponding transformations in EGA. The resulting transformation is then obtained from

$$X = M \vee X_0 \vee \underset{\sim}{M} \quad (\text{for } PGA) \tag{91}$$

We can think of the screw motion used to model motion in $3D$ PGA as a combination of a rotation by an angle ϕ around the unitized line $\boldsymbol{l}$ and by a translation from a displacement δ along that same line $\boldsymbol{l}$. The latter part results from a rotation around a line at infinity given by $\boldsymbol{l}^{\star}$. Thus, the bivector representing the line $\boldsymbol{l}$ can always be split into the parts creating the rotation (movement around that line) and the parts creating the translation (movement along that line).

1.3.1 Motion in *EGA*

In order to dive deeper into modelling of motions we first start with the *EGA* case. It is identical to the derivation shown in [Dorst and De Keninck, 2023] for their plane-based approach. This will help to better recognize the differences to the *PGA* approach used here.

Before going into more detail, we need to prepare some formulas we later need for our derivations:

For *EGA* the rotor will now be written as M instead of R to make it easy to compare to our *PGA*-derivations using motors M later on. M as a rotor in *EGA* is an orthonormal transformation and thus satisfies

$$M \wedge \widetilde{M} = \widetilde{M} \wedge M = 1 \tag{92}$$

(remember: $M^{-1} = \widetilde{M}$), and therefore

$$\frac{d}{dt}(M \wedge \widetilde{M}) = \dot{M} \wedge \widetilde{M} + M \wedge \dot{\widetilde{M}} = 0 \tag{93}$$

It follows that $M \wedge \dot{\widetilde{M}} = -\dot{M} \wedge \widetilde{M}$, which results in

$$\dot{\widetilde{M}} = -\widetilde{M} \wedge \dot{M} \wedge \widetilde{M} \tag{94}$$

Starting from $\widetilde{M} \wedge M = 1$ instead yields the same result for $\dot{\widetilde{M}}$.

In order to model rigid body mechanics we consider two coordinate frames:

- A *world frame* (index w) that is normally chosen to be an inertial frame.
- A *body frame* (index b) that is attached to a rigid body. Typically, at its center of gravity, but this is not strictly required. Rigid body's internal forces are in equilibrium and thus need not be considered explicitly. It is sufficient to describe the translational motion of the body's center of gravity and its rotational motion with respect to the center of gravity. The resulting motion of the body frame relative to the world frame then fully covers rigid body motion (kinematics and dynamics), which depends on distribution of the mass of the body as well as on external forces and moments acting on the body.
- To describe the motion we use a time dependent motor $M = M(t)$. At time $t = 0$ we call it $M_0 = M(t = 0)$. M_0 brings an element X of the body to its position X_0 at time $t = 0$.

World frame: From eqn. (82) we know the form of a general rotor in EGA and use the ansatz for a time dependent rotor $M(t)$ after the object X was brought into its initial position by M_0 as $X_0 = M_0 \wedge X \wedge \widetilde{M_0}$. If there is no difference between world and body frames at $t = 0$ then $M_0 = 1$:

$$M(t) = \exp_{\wedge}(-\frac{1}{2}B_w(t)) \wedge M_0 \tag{95}$$

with $B_w(t)$ as a time-dependent bivector that models the rotational plane and the turning angle. The derivative of M with respect to time t is

$$\dot{M}(t) = -\frac{1}{2}\dot{B}_w(t) \wedge \exp_{\wedge}(-\frac{1}{2}B_w(t)) \wedge M_0 = -\frac{1}{2}\dot{B}_w(t) \wedge M(t) \tag{96}$$

where we use that the exponential of the bivector and the bivector rate commute *(tbd: show why this is the case)*. Using $\Omega_w = \dot{B}_w(t)$ as the bivector rate of change in the *world frame*, we get

$$\dot{M} = -\frac{1}{2}\Omega_w \wedge M \tag{97}$$

By multiplying this from the right with "$\wedge \widetilde{M}$" this can be transformed to

$$\Omega_w = -2\dot{M} \wedge \widetilde{M} \tag{98}$$

Body frame: Here we first move the object in the *body frame* and then transform it into the in the world frame by the transformation M_0. In this case eqn. (95) becomes

$$M(t) = M_0 \wedge \exp_{\wedge}(-\frac{1}{2}B_b(t)) \tag{99}$$

which in turn results in

$$\dot{M}(t) = -\frac{1}{2}M_0 \wedge \exp_{\wedge}(-\frac{1}{2}B_b(t)) \wedge \dot{B}_b(t) = -\frac{1}{2}M(t) \wedge \dot{B}_b(t) \tag{100}$$

while eqn. (97) for the bivector rate of change in the *body frame* becomes

$$\dot{M} = -\frac{1}{2}M \wedge \Omega_b \tag{101}$$

using $\Omega_b = \dot{B}_b(t)$ as the bivector rate of change in the *body frame*. This can be transformed by multiplying it from the left by "$\widetilde{M} \wedge$" to obtain

$$\Omega_b = -2\widetilde{M} \wedge \dot{M} \tag{102}$$

From equations (97) and (101) we can see how world and body frame rates are connected to each other: $\Omega_w \wedge M = M \wedge \Omega_b$, which can be transformed into

$$\Omega_w = M \wedge \Omega_b \wedge \widetilde{M} \tag{103a}$$
$$\Omega_b = \widetilde{M} \wedge \Omega_w \wedge M \tag{103b}$$

Now we derive the rate of change of a point $X(t)$, i.e. its speed $\dot{X}(t)$, in the *world frame*, first assuming that $X(t=0) = X_0 = const.$, i.e. as a constant element that is transformed to become X by a time dependent motor $M(t)$. During the derivation we will use the associativity of the geometric product and the neutral element 1 of the geometric product. We can write

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}X(t) = \frac{d}{dt}(M \wedge X_0 \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_0 \wedge \widetilde{M} + M \wedge X_0 \wedge \dot{\widetilde{M}} \\
&\overset{(94)}{=} \dot{M} \wedge 1 \wedge X_0 \wedge \widetilde{M} - M \wedge X_0 \wedge (\widetilde{M} \wedge \dot{M} \wedge \widetilde{M}) \\
&\overset{(92)}{=} \dot{M} \wedge (\widetilde{M} \wedge M) \wedge X_0 \wedge \widetilde{M} - (M \wedge X_0 \wedge \widetilde{M}) \wedge \dot{M} \wedge \widetilde{M} \\
&= (\dot{M} \wedge \widetilde{M}) \wedge (M \wedge X_0 \wedge \widetilde{M}) - (M \wedge X_0 \wedge \widetilde{M}) \wedge (\dot{M} \wedge \widetilde{M})
\end{aligned}$$

From eqn. (98) we can conclude that $-\frac{1}{2}\Omega_w(t) = \dot{M} \wedge \widetilde{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= (-\frac{1}{2}\Omega_w(t)) \wedge X(t) - X(t) \wedge (-\frac{1}{2}\Omega_w(t)) \\
&= -\frac{1}{2}(\Omega_w \wedge X - X \wedge \Omega_w) \\
&= -\mathrm{cmt}(\Omega_w, X) \\
\\
\dot{X}_w &= \mathrm{cmt}(X_w, \Omega_w)
\end{aligned} \tag{104}$$

with $\mathrm{cmt}(X_w, \Omega_w)$ as the commutator product between X_w and Ω_w.

So, if the bivector rate of change Ω_w is known, the rate of change of X_w, which is $\dot{X}_w$, can be calculated from eqn. (104) using the commutator product.

It can be shown that $\dot{M} \wedge \widetilde{M}$ is always a bivector: From equations (98) and (93), and the linearity of the involved operations, we get

$$\Omega_w = -2\dot{M} \wedge \widetilde{M} = 2M \wedge \dot{\widetilde{M}} = 2\widetilde{(\dot{M} \wedge \widetilde{M})} = -\widetilde{\Omega}_w \tag{105}$$

Considering the grade-dependent sign-reversal of the reverse in $3D$ EGA, Ω_w must either be a bivector or a trivector. Since $\dot{M}$ and $\widetilde{M}$ are even-grade

multivectors, their geometric product results in an even-grade multivector as well, which does not have a trivector part. Thus, Ω_w and therefore $\dot{M} \wedge \widetilde{M}$ must indeed be a bivector.

This also assures, that the grade of X and $\dot{X}$ are identical. If X is a vector then $\dot{X}$ is a vector as well. This statement is true for any grade of X as long as Ω_w is a bivector.

If X_0 is not a constant element, but a function of time, which means that X_0 is a time dependent element in the *body frame* $X_0 = X_b(t)$, we have to add another term to the derivative in eqn. (104) as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \wedge X_b \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_b \wedge \widetilde{M} + M \wedge X_b \wedge \dot{\widetilde{M}} + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= \mathrm{cmt}(M \wedge X_b \wedge \widetilde{M}, \Omega_w) + M \wedge \dot{X}_b \wedge \widetilde{M} \\
\dot{X}_w &= \mathrm{cmt}(X_w, \Omega_w) + (\dot{X}_b)_w \qquad (106)
\end{aligned}$$

Eqn. (106) is to be considered in the world frame. The same $\dot{X}$ derivation can be carried out entirely in the *body frame*, starting from the body-frame ansatz eqn. (99) and the corresponding rate eqns. (101) and (102) — note the swapped order of M and Ω_b relative to the world frame — in place of their world-frame counterparts. The algebra is identical and yields exactly the same results as eqns. (104) and (106), with Ω_w replaced by Ω_b, so we do not reproduce it here — in the same spirit as the body-frame derivation we skip for *PGA* below.

In *EGA* rotation as a Euclidean isometry can be described with sandwich products based on the geometric product. The corresponding sandwich product is available in the library either by using the geometric product and the grade operator to extract the grade of the results directly, or by using the function `rotate(X,R)` which takes the object X and the rotor R as arguments and returns the rotated object.

1.3.2 Motion in *PGA*

Now we are looking into *PGA*. Lengyel has shown in [Lengyel, 2024b] that Euclidean isometries (covering translations as well as rotations) can be described with sandwich products based on the regressive geometric product in *PGA*. If we do that, we can stay in the space $\mathbb{G}(3,0,1)$ and do not need to switch to its dual space in order to implement these orthonormal transformations as already described above.

Here we repeat the derivation of the time-derivative $\dot{X}$ of a projective point $X(t)$ for the approach of *PGA* used here: We will use the regressive geometric product, and $\mathbb{1}$ as its neutral element. The derivation is then completely analogous to the *EGA* case with the given substitutions.

M as a motor in *PGA* is an orthonormal transformation and satisfies

$$M \vee \underset{\sim}{M} = \underset{\sim}{M} \vee M = \mathbb{1} \tag{107}$$

as the defining equation for a motor in *PGA* (remember: $M^{-1} = \underset{\sim}{M}$), leads to

$$\frac{d}{dt}(M \vee \underset{\sim}{M}) = \dot{M} \vee \underset{\sim}{M} + M \vee \dot{\underset{\sim}{M}} = 0 \tag{108}$$

It follows that $M \vee \dot{\underset{\sim}{M}} = -\dot{M} \vee \underset{\sim}{M}$, which results in

$$\dot{\underset{\sim}{M}} = -\underset{\sim}{M} \vee \dot{M} \vee \underset{\sim}{M} \tag{109}$$

Starting from $\underset{\sim}{M} \vee M = \mathbb{1}$ instead yields the same result for $\dot{\underset{\sim}{M}}$.

However, we still have to take additional aspects into account:

First, Lengyel has shown that a motor in $3D$ *PGA* can be created from the exponential in eqn. (88) as $M = \exp_{\vee}\left[(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee \boldsymbol{l}\right]$ where it is easy to show that the regressive geometric product of the line $\boldsymbol{l}$, a bivector, with the scalar part yields a bivector and the same is true for the regressive geometric product between the pseudoscalar and the line. Thus, it is sufficient to consider the exponential function of a bivector argument for the derivation for $3D$ *PGA*. The resulting motor M will be an even-grade multivector in this case.

Second, for $2D$ *PGA* the motor is derived from $M = \exp_{\vee}\left[\frac{\phi}{2}P\right]$ with the unitized point P, a vector, as an argument. The resulting motor M will be an odd-grade multivector. We need to distinguish these two cases later on. However, since the equations are structurally similar (an exponential

function with either a vector or a bivector as argument), we only do the derivation for the $3D$ case once and then will interpret the results for the $2D$ case separately afterwards.

World frame: From eqn. (88) we know the form of a general motor in PGA and use the ansatz for a time dependent motor $M(t)$ after the object X was brought into its initial position by M_0 as $X_0 = M \vee X \vee \underset{\sim}{M}$. If there is no difference between world and body frames at $t = 0$ then $M_0 = \mathbb{1}$:

$$M(t) = \exp_\vee(\frac{1}{2}B_w(t)) \vee M_0 \tag{110}$$

with $B_w(t)$ as a time-dependent bivector that models the rotational plane and the turning angle. The derivative of M with respect to time t is

$$\dot{M}(t) = \frac{1}{2}\dot{B}_w(t) \vee \exp_\vee(\frac{1}{2}B_w(t)) \vee M_0 = \frac{1}{2}\dot{B}_w(t) \vee M(t) \tag{111}$$

where we use that the exponential of the bivector and the bivector rate commute *(tbd: show why this is the case)*. Using $\Omega_w = \Omega_w(t) = \dot{B}_w(t)$ as the bivector rate of change in the *world frame*, we get

$$\dot{M} = \frac{1}{2}\Omega_w \vee M \tag{112}$$

By multiplying this from the right with "$\vee \underset{\sim}{M}$" this can be transformed to

$$\Omega_w = 2\dot{M} \vee \underset{\sim}{M} \tag{113}$$

Body frame: Here we first move the object in the *body frame* and then transform it into the in the world frame by the transformation M_0. In this case eqn. (110) becomes

$$M(t) = M_0 \vee \exp_\vee(\frac{1}{2}B_b(t)) \tag{114}$$

which in turn results in

$$\dot{M}_b(t) = \frac{1}{2}M_0 \vee \exp_\vee(\frac{1}{2}B_b(t)) \vee \dot{B}_b(t) = \frac{1}{2}M(t) \vee \dot{B}_b(t) \tag{115}$$

while eqn. (112) for the bivector rate of change in the *body frame* becomes

$$\dot{M} = \frac{1}{2}M \vee \Omega_b \tag{116}$$

using $\Omega_b = \Omega_b(t) = \dot{B}_b(t)$ as the bivector rate of change in the *body frame*. This can be transformed by multiplying it from the left by "$\underset{\sim}{M} \vee$" to obtain

$$\Omega_b = 2\underset{\sim}{M} \vee \dot{M} \tag{117}$$

From equations (112) and (116) we can see how world and body frame rates are connected to each other: $\Omega_w \vee M = M \vee \Omega_b$, which can be transformed into

$$\Omega_w = M \vee \Omega_b \vee \underset{\sim}{M} \tag{118a}$$
$$\Omega_b = \underset{\sim}{M} \vee \Omega_w \vee M \tag{118b}$$

Now we derive the rate of change of a point $X(t)$, i.e. its speed $\dot{X}(t)$, in the *world frame*, first assuming that $X(t=0) = X_0 = const.$, i.e. as a constant element that is transformed by a time dependent motor $M(t)$. During the derivation we will use the associativity of the regressive geometric product and the neutral element $\mathbb{1}$ of the regressive geometric product. We can write

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}X(t) = \frac{d}{dt}(M \vee X_0 \vee \underset{\sim}{M}) \\
&= \dot{M} \vee X_0 \vee \underset{\sim}{M} + M \vee X_0 \vee \dot{\underset{\sim}{M}} \\
&\overset{(109)}{=} \dot{M} \vee \mathbb{1} \vee X_0 \vee \underset{\sim}{M} - M \vee X_0 \vee (\underset{\sim}{M} \vee \dot{M} \vee \underset{\sim}{M}) \\
&\overset{(107)}{=} \dot{M} \vee (\underset{\sim}{M} \vee M) \vee X_0 \vee \underset{\sim}{M} - (M \vee X_0 \vee \underset{\sim}{M}) \vee \dot{M} \vee \underset{\sim}{M} \\
&= (\dot{M} \vee \underset{\sim}{M}) \vee (M \vee X_0 \vee \underset{\sim}{M}) - (M \vee X_0 \vee \underset{\sim}{M}) \vee (\dot{M} \vee \underset{\sim}{M})
\end{aligned}$$

From eqn. (113) we can conclude that $\frac{1}{2}\Omega_w(t) = \dot{M} \vee \underset{\sim}{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= (\frac{1}{2}\Omega_w(t)) \vee X_w(t) - X_w(t) \vee (\frac{1}{2}\Omega_w(t)) \\
&= \frac{1}{2}(\Omega_w \vee X_w - X_w \vee \Omega_w)
\end{aligned}$$

$$\dot{X}_w = \text{rcmt}(\Omega_w, X_w) = -\text{rcmt}(X_w, \Omega_w) \tag{119}$$

with $\text{rcmt}(\Omega_w, X_w)$ as the regressive commutator product between Ω_w and X_w. The resulting minus sign in eqn. (119) compared to eqn. (104) is the only difference between PGA and EGA. This is caused by the different ansatz for the motors comparing equations (82) and (88).

It can be shown that $\dot{M} \vee \underset{\sim}{M}$ is either always vector or a bivector: From equations (113) and (108), and the linearity of the involved operations, we get

$$\Omega_w = 2\dot{M} \vee \underset{\sim}{M} = -2M \vee \dot{\underset{\sim}{M}} = -2(\underset{\sim}{\dot{M} \vee \underset{\sim}{M}}) = -\underset{\sim}{\Omega}_w \tag{120}$$

Considering the grade-dependent sign-reversal of the regressive reverse in $3D$ PGA, Ω_w must either be a vector or a bivector (see table 12 at the end of section 1.2). Since $\dot{M}$ and $\underset{\sim}{M}$ are even-grade multivectors, their regressive geometric product results in an even-grade multivector as well, which does not have a vector part. Thus, Ω_w and $\dot{M} \vee \underset{\sim}{M}$ must indeed be a bivector in $3D$ PGA.

However, in $2D$ PGA due to the grade-dependent sign-reversal of the regressive reverse, Ω_w must either be a scalar or a vector (see table 8 at the end of section 1.2). Since $\dot{M}$ and $\underset{\sim}{M}$ are odd-grade multivectors, their regressive geometric product results in an odd-grade multivector as well, which does not have a scalar part. Thus, Ω_w and $\dot{M} \vee \underset{\sim}{M}$ must indeed be a vector in $2D$ PGA.

If X_0 is not a constant element, but a function of time, which means that X_0 is a time dependent element in the *body frame* $X_0 = X_b(t)$, we have to add another term to the derivative in eqn. (119) as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \vee X_b \vee \underset{\sim}{M}) \\
&= \dot{M} \vee X_b \vee \underset{\sim}{M} + M \vee X_b \vee \dot{\underset{\sim}{M}} + M \vee \dot{X}_b \vee \underset{\sim}{M} \\
&= -\mathrm{rcmt}(M \vee X_b \vee \underset{\sim}{M}, \Omega_w) + M \vee \dot{X}_b \vee \underset{\sim}{M} \\
\dot{X}_w &= -\mathrm{rcmt}(X_w, \Omega_w) + (\dot{X}_b)_w = \mathrm{rcmt}(\Omega_w, X_w) + (\dot{X}_b)_w \qquad (121)
\end{aligned}$$

Eqn. (121) is to be considered in the world frame. However, we can also do our derivations in a completely equivalent way using the corresponding bivector rates and rotors of the *body frame*.

The derivation in the *body frame* does not bring new insights, so we don't reproduce that here for PGA again.

1.3.3 Motor generators in *PGA*

The final step before we move on to modelling mechanics is to show how points are moved in *PGA* using the approach described above.

In $3D$ *PGA* every bivector $B = B(s, \phi)$ or every bivector rate $\dot{B} = \Omega = \Omega(v, \omega)$ has its basis components

$$\Omega_{3D} = \underbrace{\Omega_{vx}\boldsymbol{e}_{41} + \Omega_{vy}\boldsymbol{e}_{42} + \Omega_{vz}\boldsymbol{e}_{43}}_{\text{weight}(\Omega)=\Omega_\circ} + \underbrace{\Omega_{mx}\boldsymbol{e}_{23} + \Omega_{my}\boldsymbol{e}_{31} + \Omega_{mz}\boldsymbol{e}_{12}}_{\text{bulk}(\Omega)=\Omega_\bullet} \tag{122}$$

where the *weight* $\Omega_\circ$ *carries rotational information* and the *bulk* $\Omega_\bullet$ *carries translational information.*

In general, the weight only contains components that *do* contain a factor of $\boldsymbol{e}_4$. It has *directional* information on Ω. If there is zero weight, the object is contained in the horizon (H_{3dp}). The bulk only has components that *do not* contain a factor of $\boldsymbol{e}_4$. It has *positional* information on Ω. If there is zero bulk, the object contains the origin (O_{3dp}).

We can either model a pure translation, which will only be visible in the bulk part of Ω, with the weight part being zero in that case, or we can model a pure rotation around a fixed (non-ideal) line $\boldsymbol{l}$. In that case the positional information of the fixed line of rotation becomes encoded in the bulk part of Ω and the directional information on the rotation speed vector ω is encoded in the weight part of Ω.

A mixture of a translation and a rotation would either be equivalent to a rotation around another fixed line shifted by the translational part or, in case of translation along the axis of rotation, a screw motion around and along that axis. It is sufficient to look at pure translation and pure rotation around a fixed line as separate cases. This separation is also important for defining the exponential function with respect to the regressive geometric product that converts the bivector B to the respective motor M: $M(t) = \exp_{\vee}(\frac{1}{2}B(t)) = \exp_{\vee}(\frac{1}{2}\Omega t)$.

The total bivector rate Ω can be split into

$$\Omega = \Omega_\circ + \Omega_\bullet = \Omega_{rot} + \Omega_{tra} \tag{123}$$

where eqn. (123) is called the "Euclidean split" of the bivector rate Ω because it separates the rotational and translational aspects of a moving object.

A pure rotation around an axis $\vec{\omega} = \omega$ can be modelled by a fixed (unitized) line that is scaled by the angular speed of rotation, i.e. by the bulk norm $\|\omega\|_\bullet$. If we have an arbitrary rotation axis we can either wedge a unitized point $Q = O_{3dp} + q = \boldsymbol{e}_4 + q$ that is contained in the rotational axis with the rotational speed ω, or create a unitized line $\hat{\omega}$ in the direction of the rotation vector ω that is later scaled with the rotational speed such that the bivector Ω_{rot} becomes

$$\begin{aligned}
\Omega_{rot} &= Q \wedge \omega = (O_{3dp} + q) \wedge \omega \\
&= \boldsymbol{e}_4 \wedge \omega + q \wedge \omega \\
&= \|\omega\|_\bullet (\boldsymbol{e}_4 \wedge \hat{\omega} + q \wedge \hat{\omega}) \\
\Omega_{rot} &= \|\omega\|_\bullet \begin{pmatrix} \hat{\omega}_x \\ \hat{\omega}_y \\ \hat{\omega}_z \\ q_y\hat{\omega}_z - q_z\hat{\omega}_y \\ q_z\hat{\omega}_x - q_x\hat{\omega}_z \\ q_x\hat{\omega}_y - q_y\hat{\omega}_x \end{pmatrix}
\end{aligned} \tag{124}$$

where $\hat{\omega}$ is the bulk normalized rotation vector $\hat{\omega} = \omega/\|\omega\|_\bullet$. Thus, Ω_{rot} directly represents the normalized (non-ideal) axis of rotation. It is a line of fixed points not changing due to the rotation, scaled with the angular speed of rotation $\|\omega\|_\bullet$.

Of course, Ω_{rot} could also be expressed directly by the plane of rotation pl (a trivector) when the magnitude of the plane encodes the angular speed $\|\omega\|_\bullet$. Considering that the left weight dual of the plane $pl_\star$ provides the normal vector of the plane including its magnitude, eqn. (124) can be written in the equivalent form directly in terms of the plane of rotation pl

$$\Omega_{rot} = Q \wedge pl_\star \tag{125}$$

For modelling translational speed $\vec{v} = v$ in $3D$ we need a bivector rate Ω_{tra} that does not contain a factor of $\boldsymbol{e}_4$. This can be achieved by encoding the speed of translation in the normal vector n_v of a plane and then intersecting this plane with the horizon $\overline{\boldsymbol{e}_4} = \overline{O_{3dp}} = H_{3dp}$. The plane with the corresponding attitude can be encoded by using the right bulk dual $v^\star$ to model the plane (a trivector) with the bulk norm $\|v\|_\bullet$ as magnitude. It contains the origin O_{3dp} and results in a bivector rate Ω_{tra} of the translation as

$$\Omega_{tra} = v^\star \vee \overline{\boldsymbol{e}_4} = v^\star \vee H_{3dp} = att(v^\star) = (0, 0, 0, v_x, v_y, v_z) \tag{126}$$

For the case of screw motion we have the special case that the translational motion is directed in the same direction as the rotation axis, i.e. we have a

special Ω_{tra}

$$\Omega_{tra,screw} = \frac{\delta}{\|\omega\|_\bullet} att(\omega^\star) = \delta\,(0,0,0,\hat{\omega}_x,\hat{\omega}_y,\hat{\omega}_z) \tag{127}$$

where $\hat{\omega}$ is the bulk normalized vector ω, which has magnitude one. This vector is scaled with the translation distance δ resulting in Ω_{screw} of

$$\Omega_{screw} = \Omega_{rot} + \Omega_{tra,screw} = \|\omega\|_\bullet \begin{pmatrix} \hat{\omega}_x \\ \hat{\omega}_y \\ \hat{\omega}_z \\ q_y\hat{\omega}_z - q_z\hat{\omega}_y \\ q_z\hat{\omega}_x - q_x\hat{\omega}_z \\ q_x\hat{\omega}_y - q_y\hat{\omega}_x \end{pmatrix} + \delta \begin{pmatrix} 0 \\ 0 \\ 0 \\ \hat{\omega}_x \\ \hat{\omega}_y \\ \hat{\omega}_z \end{pmatrix} \tag{128}$$

The components of Ω_{rot}, Ω_{tra} and Ω_{screw} are relevant for the exponential function that transforms our bivector, representing the intended motion, into a motor that transforms points accordingly.

Using for example eqn. (119) and the Euclidean split as defined in eqn. (123) we can write

$$\begin{aligned} \dot{X}_w &= \mathrm{rcmt}(\Omega_w, X_w) = \dot{x}_w \\ &= \mathrm{rcmt}(\Omega_{rot} + \Omega_{tra}, X_w) \\ \dot{x}_w &= \mathrm{rcmt}(\Omega_{rot}, X_w) + \mathrm{rcmt}(\Omega_{tra}, X_w) && \text{(129a)} \\ \dot{x}_w &= v_{rot} + v_{tra} && \text{(129b)} \end{aligned}$$

to calculate the combined speed $\dot{x}_w$ from rotational and translational effects at a position X_w.

In $2D$ PGA the equations are very similar, but the rate of change is not a rate of change bivector, but is it a rate of change vector, since the rotation is not around a fixed axis, as in $3D$, but it is around a fixed point. The translation is characterized by a $2D$ vector, whereas the rotation rate can be characterized by a single scalar value and a fixed point Q around with the rotation occurs. To show that we still get comparable equations to the $3D$ case, we will also use $\Omega = \Omega(v, \omega)$ for the rate of change. In $2D$ PGA has its basis components

$$\Omega_{2D} = \underbrace{\Omega_x \boldsymbol{e}_1 + \Omega_y \boldsymbol{e}_2}_{\text{bulk}(\Omega)=\Omega_\bullet} + \underbrace{\Omega_z \boldsymbol{e}_3}_{\text{weight}(\Omega)=\Omega_\circ} \tag{130}$$

where the *weight* $\Omega_\circ$ *carries rotational information* and the *bulk* $\Omega_\bullet$ *carries translational information*. In general, the weight only contains components that *do* contain a factor of $\boldsymbol{e}_3$ (which here takes a role equivalent to $\boldsymbol{e}_4$ in the $3D$ case). The bulk only has components that *do not* contain a factor of $\boldsymbol{e}_3$.

The total vector rate of change Ω can be split into

$$\Omega = \Omega_\bullet + \Omega_\circ = \Omega_{tra} + \Omega_{rot} \tag{131}$$

where eqn. (131) is called the "Euclidean split" of the vector rate of change Ω separating the translational and rotational aspects of a moving object.

A simple rotation in $2D$ with an angular speed ω (a simple scalar in this case) around the unitized point $Q = O_{2dp} + q = \boldsymbol{e}_3 + q$ can be modelled by multiplying the point Q with ω using the wedge product to generate the vector Ω_{rot}.

$$\Omega_{rot} = Q \wedge \omega = \boldsymbol{e}_3 \wedge \omega + q \wedge \omega = (0, 0, 1)\,\omega + (q_x, q_y, 0)\,\omega = (q_x, q_y, 1)\,\omega \tag{132}$$

For modelling translational speed $\vec{v} = v$ in $2D$ we need a vector rate of change Ω_{tra} that does not contain a factor of $\boldsymbol{e}_3$. To show this we use exactly the same equations as in the $3D$ case by encoding the speed of translation in the normal vector n_v of a line and then intersecting this line with the horizon $\overline{\boldsymbol{e}_3} = \overline{O_{2dp}} = H_{2dp}$. The double reflection on parallel lines that effectively creates the translation, necessitates the need for encoding the translation speed in a vector pointing normal to the translation direction. The line with the corresponding attitude can be encoded by using the right bulk dual $v^\star$ to model the line (a bivector) with the bulk norm $\|v\|_\bullet$ as magnitude. The vector rate of change Ω_{tra} of a pure translation is then given as

$$\Omega_{tra} = v^\star \vee \overline{\boldsymbol{e}_3} = v^\star \vee H_{3dp} = att(v^\star) = (-v_y, v_x, 0) \tag{133}$$

where the direction of the movement is encoded as the normal pointing towards the point at infinity which is the“rotation center at infinity” of the modelled translation.

We can either model a pure translation, which will only be visible in the bulk part of Ω, with the weight part being zero in that case, or we can model a pure rotation around a fixed (non-ideal) point Q. In that case the positional information of the fixed point becomes encoded in the bulk part of Ω and the rotational speed ω is encoded in the weight part of Ω. A mixture of a translation and a rotation would be equivalent to a rotation around another fixed point shifted by the translational part. So it is sufficient to look at pure translation and pure rotation around a fixed point as separate cases.

$$\Omega_{2D} = \Omega_{rot} + \Omega_{tra} = \omega \begin{pmatrix} q_x \\ q_y \\ 1 \end{pmatrix} + \begin{pmatrix} -v_y \\ v_x \\ 0 \end{pmatrix} \tag{134}$$

This separation is also important for defining the exponential function with respect to the regressive geometric product that converts the bivector B to the respective motor M: $M(t) = \exp_{\vee}(\frac{1}{2}B(t)) = \exp_{\vee}(\frac{1}{2}\Omega t)$ that enables us to actually transform points according to the vector rates of change as defined above.

Caution: The additive split $\Omega = \Omega_{rot} + \Omega_{tra}$ is a statement about *generators* — the velocity level, where rates add and the components are independent. Finite motions do not add. One must not read this split as a recipe for building a combined finite motion by adding generators (equivalently, by assuming the two motors commute); that is valid only in one special case. The next section makes this precise (Sec. 1.3.4).

1.3.4 Independent generators, coupled finite displacements

The motion generator Ω_b lives in the *Lie algebra* $\mathfrak{se}(2)$ ($2D$) or $\mathfrak{se}(3)$ ($3D$). The motor M itself lives in the corresponding *Lie group* $SE(2)$ or $SE(3)$. These two levels have fundamentally different structure. The Lie algebra $\mathfrak{se}(2)$ is three-dimensional with basis generators:

Generator	`vec2dp` encoding	Effect
Translation in x	$\{0,\ 1,\ 0\}$	body drifts in $+x$
Translation in y	$\{-1,\ 0,\ 0\}$	body drifts in $+y$
Rotation about origin	$\{0,\ 0,\ 1\}$	body spins about world origin

Table 18: Basis generators of $\mathfrak{se}(2)$ in $PGA2DP$ encoding.

A general instantaneous velocity $\Omega_b = \{-v_y,\ v_x,\ \omega\}$ means simultaneously: translate at (v_x, v_y) and rotate at ω – with no coupling between the three components. Integrating the velocity over an infinitesimal step dt gives

$$B_b \leftarrow B_b + \Omega_b \cdot dt, \tag{135}$$

so all three components of B_b accumulate independently.

When computing a *finite* displacement via $M = \exp_\vee(\frac{1}{2}B_b)$, the structure of the $2D$ plane imposes a hard constraint:

> **Chasles' theorem ($2D$):** Any rigid body displacement in $2D$ is either a pure rotation about some center, or a pure translation. There is no screw motion.

Consequently, for any B_b with $B_b.z \neq 0$, the motor $\exp_\vee(\frac{1}{2}B_b)$ is always a pure rotation – the translational components merely determine the rotation center:

$$\text{pivot} = \left(\frac{B_b.x}{B_b.z},\ \frac{B_b.y}{B_b.z}\right). \tag{136}$$

The "translation" encoded in $B_b.x$ and $B_b.y$ does not produce an independent translational displacement; it shifts the rotation center.

At the velocity level (Lie algebra) the components are independent. At the displacement level (Lie group) they are not.

In $3D$ the Lie algebra $\mathfrak{se}(3)$ is six-dimensional: three components for the rotation axis direction, three for the translation velocity. Chasles' theorem for $3D$ states:

> **Chasles' theorem ($3D$):** Any rigid body displacement in $3D$ is a *screw motion*: a rotation about an axis combined with a translation *along* that same axis (helical motion).

Exponentiating a combined $3D$ generator therefore yields a screw – rotation and translation are simultaneously present in the finite displacement. The rotation axis direction and the translational pitch are independently adjustable because there exists a direction (along the axis) that is geometrically orthogonal to the plane of rotation. In $2D$ no such direction exists: the axis is perpendicular to the entire plane, so simultaneous rotation and translation always collapse to rotation about a different center.

Lie algebra level	*PGA2DP*	*PGA3DP*
Dimension of $\mathfrak{se}$	3	6
Velocity generator Ω_b	$(-v_y,\ v_x,\ \omega)$	$(v_x,\ v_y,\ v_z,\ \omega_x,\ \omega_y,\ \omega_z)$
Components independent?	yes	yes

Table 19: Velocity generators at the Lie algebra level.

Lie group level	*PGA2DP*	*PGA3DP*
Result of $\exp_\vee(\frac{1}{2}B)$	pure rotation	screw motion
Finite pivot / screw axis	point $\left(\frac{B.x}{B.z},\ \frac{B.y}{B.z}\right)$	line (Plücker coordinates)
Role of "translation" in B	shifts rotation center	adds pitch along axis

Table 20: Finite displacements at the Lie group level.

The accumulated generator B_b (and B_w) is a Lie algebra element – its components are additive and independent *as generators*. The pivot decoded from B_b as $(B_b.x/B_b.z,\ B_b.y/B_b.z)$ is meaningful as the instantaneous rotation center of the *finite* displacement $\exp_\vee(\frac{1}{2}B_b)$, not of the instantaneous velocity Ω_b. When $B_b = B_{b,0} + \Omega_b \cdot t$ (uniform motion from initial condition $B_{b,0}$):

- If $B_{b,0} = \mathbf{0}$: the decoded pivot equals the body-frame pivot of Ω_b for all t, i.e. $(B_b.x/B_b.z,\ B_b.y/B_b.z) = (\Omega_b.x/\Omega_b.z,\ \Omega_b.y/\Omega_b.z) = Q_b$ – constant and geometrically meaningful.
- If $B_{b,0} \neq \mathbf{0}$ with $B_{b,0}.z = 0$ (a translational initial offset): the decoded pivot starts at infinity (pure translation at $t = 0$) and converges asymptotically to Q_b as $t \to \infty$. This makes the decoded pivot hard to in-

terpret and should be avoided in visualizations intended to explain the encoding.

Design rule: use $B_{b,0} = \mathbf{0}$ (body starts at the M_0 reference pose) so that the accumulated generator retains a clean geometric meaning.

Composing motions — multiply motors, do not add generators: The same caveat governs how separate motions *compose*. A finite motion is applied by its motor $M = \exp_{\vee}(\frac{1}{2}B)$, and two successive motions compose by the regressive geometric product (motor multiplication), which is order-dependent:

$$M_2 \vee M_1 \neq M_1 \vee M_2 \tag{137a}$$

$$\text{and} \quad \exp_{\vee}\big(\tfrac{1}{2}(A+B)\big) \neq \exp_{\vee}\big(\tfrac{1}{2}A\big) \vee \exp_{\vee}\big(\tfrac{1}{2}B\big) \tag{137b}$$

$$\text{unless} \quad \operatorname{rcmt}(A,B) = 0 \tag{137c}$$

Adding generators equals multiplying motors only when the generators commute under the regressive product. For a pure translation and a pure rotation this happens *exclusively* in the $3D$ screw case (translation *along* the rotation axis); in $2D$ a nonzero rotation and a translation never commute (Chasles: the result is always a pure rotation about a shifted center). Consequently, writing $B = B_{rot} + B_{tra}$ and exponentiating yields the *single* screw — or, in $2D$, the single rotation about a shifted pivot — generated by that combined twist; it is *not* the composition "translate, then rotate". To realize that composition, multiply the two motors in the intended order, $M = M_{rot} \vee M_{tra}$.

1.3.5 Motion in STA

Motion in $STA4DS$ is the Lorentz transformation of a space-time object X, applied by the same geometric-product rotor sandwich as in EGA — not by the regressive motors of PGA:

$$X' = R \wedge X \wedge \widetilde{R}, \qquad R \wedge \widetilde{R} = 1. \tag{138}$$

The even-grade part of the algebra forms the group of rotors R, each built as the exponential of a bivector generator. What the Minkowski signature adds to the EGA picture is a second kind of generator.

A boost is the space-time counterpart of a spatial rotation: a rotation mixes two space directions, a boost mixes one space direction with time. It relates the measurements of two observers in uniform relative motion — the passage

from a rest frame to one moving at constant velocity. The character of a rotor follows from the geometric square of its generating plane: a space-like plane ($B^2 < 0$) exponentiates with cos and sin (a rotation, periodic in its angle), a time-like plane ($B^2 > 0$) with cosh and sinh (a boost, unbounded in its *rapidity* φ). A rotation by θ in $\boldsymbol{g}_{12}$ and a boost by φ in $\boldsymbol{g}_{14}$ therefore read

$$R_{rot} = \cos\tfrac{\theta}{2} - \sin\tfrac{\theta}{2}\,\boldsymbol{g}_{12} = \exp_{\wedge}(-\tfrac{1}{2}\,\theta\,\boldsymbol{g}_{12}), \tag{139}$$

$$R_{boost} = \cosh\tfrac{\varphi}{2} + \sinh\tfrac{\varphi}{2}\,\boldsymbol{g}_{14} = \exp_{\wedge}(+\tfrac{1}{2}\,\varphi\,\boldsymbol{g}_{14}) \tag{140}$$

(the rotors built by `get_rotor` and `get_boost`). Note the opposite half-angle signs — $-\frac{1}{2}$ for the rotation, $+\frac{1}{2}$ for the boost: each is fixed independently so that a positive parameter produces a positive-sense transform (a right-handed rotation by $+\theta$, a boost with $\gamma = \cosh\varphi$), and the two are not meant to agree. The relative speed of the boost is $\beta = \tanh\varphi$ (with $c = 1$), so no boost attains $\beta = 1$, and collinear rapidities add as $\varphi_1 + \varphi_2$, just as rotation angles do. The mutual tilt of the time and space axes that a boost produces is the origin of time dilation and length contraction.

A general Lorentz transformation is one boost and one rotation combined, carried by the two commuting simple bivectors of the logarithm $\log(R) = b_{boost} + b_{rot}$ (one time-like, one space-like). This invariant decomposition is what `exp`, `log` and `sqrt` use for a non-simple rotor: exponentiate each plane on its own branch and multiply, $R = \exp_{\wedge}(b_{boost}) \wedge \exp_{\wedge}(b_{rot})$; invert that to recover the two generators; halve both to take the square root.

1.4 Modelling mechanics in PGA

The key idea is to represent physical entities using objects represented in the underlying algebra. They have a geometric interpretation and can be manipulated by applying algebraic operations. This is possible since the algebraic objects we use in geometric algebra, especially the blades resulting from outer products between vectors, represent subspaces of the overall space and thus allow for direct algebraic manipulation. At the same time, they represent or can be attributed to objects of geometric or physical meaning. This enables implementing geometrically and physically meaningful manipulations by applying algebraic operations on those subspaces or between them.

Euclidean geometric algebra (EGA) is limited to objects that contain the origin. Thus, it can be used to directly model vectors (i.e. directions), lines, and planes through the origin. In addition, it can model complex numbers, quaternions and thus rotations in $2D$ and $3D$ well. To realize them, it uses sandwich products based on the geometric product for calculating the transformation resulting from two consecutive reflections either across lines ($2D$ case) or across planes ($3D$ case). The resulting transformations are rotations. The advantage of using geometric algebra to model those rotations is the intuitive modelling, due to geometric meaning of the objects and the fact that the sandwich products can be used to transform different objects, like vectors, bivectors or rotation operators using the same sandwich product formulas in a unified fashion regardless of the type of object to be transformed. Despite those limitations EGA is already a very powerful tool to express physical ideas as shown by David Hestenes in [Hestenes, 1986]. In this work he reformulates classical mechanics using geometric algebra mainly with EGA as a tool. But he is also showing that even more is possible, e.g. for formulating the motion of rigid bodies, clearly pointing to PGA in his chapter 7 on rigid body mechanics, even if he does not call that PGA yet. This is already looking extremely similar to what is shown later in this section. His book is a rich source of applications of geometric algebra for many application areas of mechanics.

Projective geometric algebra (PGA) is indeed capable to model objects that do not necessarily contain the origin. Thus, it can be used to directly model points, directions, lines, and planes that may or may not contain the origin. In contrast to EGA, translations can additionally be implemented in PGA directly as orthogonal transformation using a sandwich product. The operation is realized by two consecutive reflections across parallel lines ($2D$) or planes ($3D$). It is based on the regressive form of the geometric product.

The reason for this deviation from EGA, which uses the geometric product for that purpose, is that the Euclidean isometries (rotation, translation, and screw motion) can only be performed in a space that is dual to the space we use for modelling our projective objects (for a more in depth explanation see [Lengyel, 2024b]). The use of the regressive geometric product on these objects is equivalent to first transforming the object into dual space by applying the complement operation, then using the geometric product in that space for the intended transformation and finally transforming the resulting transformed object back into the original space, by applying the inverse complement operation. Thus, the sandwich product transformation formulas in PGA only differ from the formulas used in EGA by using the regressive geometric product instead of the geometric product, but are otherwise the same.

The additional degree of freedom in PGA is possible due to embedding the Euclidean space of dimension n into a projective space of dimension $n+1$, e.g. two-dimensional Euclidean space is modelled by embedding it in a three-dimensional projective space, and three-dimensional Euclidean space is modelled in a four-dimensional projective space. In the $n+1$-dimensional projective space all algebraic objects contain the origin (exactly as in EGA), but in the modelled n-dimensional Euclidean space the objects do not need to contain the origin any longer. They can be located anywhere in the modelled space including positions at infinity. This can be realized in projective geometry, since objects of dimension $n+1$ are projected into the modelled space resulting in objects of dimension n after the projection. A projective point (which models a scalar or $0D$-object) is a vector representing a ray through the origin in the embedding space (a $1D$-object), and a $1D$-object, like a line in modelled space, is created by a bivector in the embedding projective space (a $2D$-object), and so on for higher-grade objects.

Due to its larger expressiveness PGA is suitable to model physics of mass points and rigid bodies including all isometries. For example, it can model position, velocity, acceleration, force, torque, momentum, angular momentum, etc., and thus it can be used to describe kinematics as well as dynamics of mass points and rigid bodies. The approach used here is inspired by [Dorst and De Keninck, 2022] and [Dorst and De Keninck, 2023], but it does not use the plane-based ansatz with planes as vectors, which are based on dual representations of the objects to be modelled. It rather uses the approach as described by [Lengyel, 2024b] using projective geometric algebra based on direct representations of points, lines and planes. Lengyel also gives reasons for choosing the latter approach and explains the background in more detail compared to the short indications given here.

Objects modelled in a projective space can be multiplied by a scalar value without changing the *geometric* meaning of the object in the modelled Euclidean space, i.e. its interpretation after the projection. Equality of modelled geometric objects is thus often only defined up to a scalar factor. This kind of equality up to a scalar factor is defined as congruence (see e.g. [Browne, 2012]). Therefore, it is important for modelling of physical facts that unitization is used consciously to avoid false interpretations. For example, a heavy point can be modelled by multiplying a *unitized* point with the scalar value of its mass, but of course the multiplication with m must happen after unitization in order to keep the scalar factor m in the calculation. Other geometric objects need to be unitized in order to focus on the intended geometric operation and avoid additional scaling, i.e. the rotation operators. They must be unitized before use as well. Thus, it is advisable to think through the geometric operations and their physical interpretation well to avoid surprises and to be consistent.

1.4.1 Modelling force and torque

[Browne, 2012] (see sections 4.1-4.5) is a very good source showing how to use Grassmann algebra and projective geometry in order to model forces and moments. Classical approaches based on vector algebra only model forces by providing vectors of adequate magnitude and direction to represent the force. However, the physically very meaningful "line of action", which determines which torque is created by the force, is not modelled mathematically in the classical approach at all. It rather needs to be derived from the description of the context in order to know where the force is actually acting. Thus, force and torque are not directly interlinked in the classical description, but must be modelled as separate physical effects.

This changes in *PGA*. Any force can be directly modelled by its line of action, which encodes the force *and* the torque generated by the force at the origin within one algebraic object. A force f is modelled by a line F. If f is the force vector of corresponding magnitude and direction, and Q is any unitized projective point on its line of action then F is an algebraic object representing the force f as well as the resulting line of action directly:

$$F = Q \wedge f \tag{141}$$

The resulting bivector F represents a line, a geometric object which contains all physically relevant information of the force f and the torque generated by f. For that reason [Dorst and De Keninck, 2023] call this object "forque". It can directly be used to recover the force vector and to calculate the resulting free torque caused by the force either at the origin or at any point in space.

But first, let's have a closer look how this is accomplished. We can separate F into its components. Each unitized projective point Q is the sum of the origin O and a vector q parallel to the projective plane (in $2D$) or projective space (in $3D$) pointing from O to Q, such that $Q = O + q$ ("projective point split"). Thus, F can be split into two bivector components

$$F = Q \wedge f = (O + q) \wedge f = \underbrace{O \wedge f}_{=\text{force}} + \underbrace{q \wedge f}_{=torque} = F_\circ + F_\bullet \tag{142}$$

where the first term encodes the force within the weight components of the bivector, while the second term encodes the resulting free torque relative to the origin caused by the action of f at Q – or alternatively anywhere along the line of action – within the bulk components of the bivector.

To see this even more clearly, let's look at the component equations of both terms written as bivectors for the case of $2D$ PGA (the corresponding basis bivectors are shown on the right-hand side in square brackets for reference):

$$O \wedge f = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{31} \\ \boldsymbol{e}_{32} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{143a}$$

$$q \wedge f = \begin{pmatrix} q_x \\ q_y \\ 0 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ q_x f_y - q_y f_x \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{31} \\ \boldsymbol{e}_{32} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{143b}$$

where equation (143a) shows how the force is encoded in the first two components of the bivector and equation (143b) shows how the torque is encoded in the third component of the bivector F.

For $3D$ PGA the encoding is similar, but now requires six bivector components instead of three (the corresponding basis bivectors are shown on the right-hand side in square brackets for reference again):

$$O \wedge f = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \\ 0 \\ 0 \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{41} \\ \boldsymbol{e}_{42} \\ \boldsymbol{e}_{43} \\ \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{144a}$$

$$q \wedge f = \begin{pmatrix} q_x \\ q_y \\ q_z \\ 0 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ q_y f_z - q_z f_y \\ q_z f_x - q_x f_z \\ q_x f_y - q_y f_x \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{41} \\ \boldsymbol{e}_{42} \\ \boldsymbol{e}_{43} \\ \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{144b}$$

The first three bivector components model the force while the last three components model the torque. The force vector f itself can be retrieved from F by using the attitude function $att()$:

$$f = att(F) = F \vee \overline{\boldsymbol{e}_n} = att(F_\circ) = F_\circ \vee \overline{\boldsymbol{e}_n} \tag{145}$$

with $\boldsymbol{e}_n$ denoting the basis vector of the projective dimension that squares to zero in the respective algebra ($\boldsymbol{e}_3$ for $2D$ and $\boldsymbol{e}_4$ for $3D$). The torque generated relative to the origin O is encoded in the bulk of the bivector:

$$M_O = bulk(F) = F_\bullet \tag{146}$$

while the resulting torque from the application of f at any given point R in space can be retrieved from F, either using any known point Q on F[6], or alternatively by projecting R onto F and using the perpendicular distance vector $d_\perp$ from R towards the line F in order to calculate the resulting free torque $M_F(R)$ at the position R as

$$M_F(R) = [(Q - R) \wedge f]_\bullet = [(Q - R) \wedge att(F)]_\bullet \tag{147a}$$
$$M_F(R) = [d_\perp \wedge f]_\bullet = [(F \vee (R \wedge F^*) - R) \wedge att(F)]_\bullet \tag{147b}$$

where equation (147a) is the formula with the least computational effort using a known point Q on F, while equation (147b) uses the orthogonal projection of R onto F to calculate the perpendicular distance vector $d_\perp$.

The resulting total force and torque acting on an object are given by:

$$F_{total} = \sum_i F_i \tag{148}$$

This resulting sum will be used later for connecting kinematics and dynamics.

Things to remember:

- Adding forces represented as projective lines creates resulting force lines located at the right position to get equivalent moments automatically. When two parallel forces are added the resulting force line will be located in between such that it creates a torque equivalent to the torque initially generated by both forces. It creates an equivalent force system reducing multiple forces and moments acting on a rigid body to a single resultant force and a resultant moment about a specific point while maintaining the same external effect on the rigid body.
- Thus, the sum of forces and their induced moments can be directly created by the sum of their respective lines of action, i.e. by summing the bivectors representing those lines. The resulting lines of action contain all relevant information incl. the resulting vector sum of the forces and the resulting moment values from the summation. There is no need to calculate these separately. The result will always be a resulting bivector that potentially contains a bound vector and a resulting moment which creates a torque. (see [Browne, 2012], p. 176).

[6] Which is always available from the function $sup(F)$. It provides the support point on F with the shortest distance to the origin O by orthogonally projecting the origin onto F.

- Neither a vector nor a bivector are located anywhere. They can be moved around freely. A line however represents a bound vector that is bound by its line. Thus, the vector can only be moved along its line of action without changing the geometric and physical meaning (see [Browne, 2012], p. 169).
- A free torque is modelled by a bivector with force components being zero, i.e. the resulting force vector (e.g. from summing several forces acting on an object) has no direction and zero length, but the bivector still might contain a resulting torque value.
- The torque calculated from either equation (147a) or (147b) is necessarily a free torque as can be seen from the formulas. The origin does not occur in the result anymore due to the difference of the two projective points involved.

1.4.2 Modelling linear and angular momentum

Momentum in the classical approach is split into linear momentum $\vec{p} = m\vec{v}$ and angular momentum $\vec{L} = \vec{x} \times \vec{p} = \vec{x} \times m\vec{v}$. Both are represented as vectors, but they represent entities with different physical meaning. So they cannot be combined into one object in the classical approach, e.g. by adding them.

The *PGA* approach to momentum is to define both, the linear and the angular aspects, within one algebraic entity. This is in complete analogy to what we have seen in the last subsection for force and moment resulting in its line of action modelled by a bivector.

Single point mass: The momentum P of a point mass m at a projective (unitized) position X that moves with velocity $\dot{X}$ is defined as a bivector P as

$$P = X \wedge m\dot{X} \tag{149}$$

It defines the overall momentum at position X including linear and angular momentum in the bivector $P(X)$, which can be shown by splitting the projective point X into its components $X = O + x$ ("projective point split"). In the $3D$-case with the origin $O = O_{3dp} = (0, 0, 0, 1)$ and the direction vector x, where $x = (X_x, X_y, X_z, 0)$, i.e. specifically with $X_w = 0$, which represents a direction towards a point at infinity. This results in

$$\begin{aligned} P(X) &= X \wedge m\dot{X} = (O + x) \wedge m\dot{X} \\ &= O \wedge m\dot{X} + x \wedge m\dot{X} \\ &= O \wedge m\dot{(O + x)} + x \wedge m\dot{(O + x)} \\ &= O \wedge m\dot{x} + x \wedge m\dot{x} && \text{(150a)} \\ &= \underbrace{P_\circ}_{\text{linear}} + \underbrace{P_\bullet}_{\text{angular}} && \text{(150b)} \end{aligned}$$

P defines an instantaneous momentum line where the weight part $P_\circ$ of the bivector P encodes the linear momentum and the bulk part $P_\bullet$ encodes the angular momentum with respect to the origin.[7]

[7] The "projective point split" works in this way, since the origin is a distinguishable point. This is in contrast to *EGA* or the classical approach where the origin is a null-vector, which has no lasting effect if it is added to another point. This is one reason why *PGA* can combine linear and angular aspects as separate physical effects in a single algebraic object. The other is that the required 6 degrees of freedom can be encoded in a bivector.

Recovering linear and angular momentum: The physical aspects encoded in the momentum line P can be retrieved in complete analogy to the case of force and torque in subsection 1.4.1:

The linear momentum vector p itself can be retrieved from P by using the attitude function $att()$:

$$p = mv = m\dot{x} = att(P) = P \vee \overline{\boldsymbol{e}_n} = att(P_\circ) = P_\circ \vee \overline{\boldsymbol{e}_n} \tag{151}$$

with $\boldsymbol{e}_n$ denoting the basis vector of the projective dimension that squares to zero in the respective algebra ($\boldsymbol{e}_3$ for $2D$ and $\boldsymbol{e}_4$ for $3D$).

The angular momentum L generated relative to the origin O is encoded in the bulk of the bivector:

$$L_O = x \wedge m\dot{x} = bulk(P) = P_\bullet \tag{152}$$

while the resulting angular momentum relative to any given point R in space can be retrieved from the momentum line P by

$$L(R) = \left[(X - R) \wedge m\dot{X}\right]_\bullet = \left[(X - R) \wedge att(P)\right]_\bullet \tag{153}$$

using X as a known point on P. This provides the resulting angular momentum $L(R)$ with respect to R. The momentum line encodes the resulting momentum including its linear and angular aspects relative to any arbitrarily chosen point.

The equations (151) and (152) given above permit to separate the linear and angular effects as in the classical approach. However, these aspects need not be separated at all and can be calculated together in PGA.

Using eqn. (149) and the definition $X = O + x$ with X as unitized projective point, O as the chosen origin and x as the direction vector pointing from O to X we can write

$$P_O(X) = X \wedge m\dot{X} = (O + x) \wedge m(\dot{O + x}) \tag{154}$$

Compared to eqn. (149) an index O was added to P, since the formula (154) seems to confirm that the momentum depends on the choice of the origin. Now it is important to demonstrate that this is *not* the case. We are going to demonstrate that the described physics does not depend on the chosen origin and not on the resulting coordinates used for its description.

We use a transformation of the standpoint from which we describe the physical situation without actually moving the coordinate system (we still want to use the chosen coordinate system to describe the vectors and bivectors used): We go to a virtual origin $O' = Y$, but describe the situation in the same coordinate system. This results in describing the origin as $O = Y - y$ as seen from Y. Inserting that definition into eqn. (154) and using the geometric vector identity

$$X = Y + (X - Y) = Y + (O + x - (O + y)) = Y + (x - y) \tag{155}$$

we express $P(X)$ as viewed from Y by reformulating the equation for P relative to the origin O

$$\begin{aligned} P_O(X) &= X \wedge m\dot{X} = (O + x) \wedge m(\dot{O + x}) \\ &= (Y - y + x) \wedge m(\dot{Y - y + x}) \\ &= (Y + (x - y)) \wedge m(\dot{Y + (x - y)}) = P_Y(X) \end{aligned} \tag{156}$$

where the last equation obviously is the same as eqn. (154), but here Y takes the role of O and $(x - y)$ takes the role of the vector x in the old description. The dot used for the time-derivative refers to the whole expression in parentheses. So the same physical situation was described from another viewpoint. However, the algebraic object P is unchanged, i.e. $P_O(X) = P_Y(X)$. This clearly shows that P does not depend on the chosen origin and that eqn. (154) is indeed a coordinate free definition.

Mass distributions: The total momentum of a set of mass points $m_i X_i$ is defined as the sum of all momentum blades.

$$P_{total} = \sum_i P_i \tag{157}$$

P_{total} will always be a momentum blade in $2D$ PGA, while in $3D$ PGA (a four-dimensional algebra) it could also result in a general bivector that might not be factorizable into a blade (see [Browne, 2012]). That would be the case, if $P \wedge P \neq 0\ \mathbb{1}$, i.e. when the result is a pseudoscalar that is not equal to a zero multiple of the pseudoscalar $\boldsymbol{e}_{1234}$. This test can always be used to determine whether a bivector is a blade in $3D$ PGA.

Using discrete points $X_{w,i}$ of the world frame indexed by i that can be moved by a motor $M(t) = exp_{\vee}[\frac{1}{2}B_w(t)] \vee M_0$ in the world frame as by eqn. (91) $X_{w,i} = M \vee X_{0,i} \vee \underbrace{M}$ results in a local speed of $\dot{X}_{w,i} = \mathrm{rcmt}(\Omega_w, X_{w,i})$ as shown

in eqn. (119). Therefore, we can write the total momentum P_w in the world frame as

$$\begin{aligned} P_w &= \sum_i P_i \\ &= \sum_i X_i \wedge m_i \dot{X}_i \\ &= \sum_i X_i \wedge m_i \operatorname{rcmt}(\Omega_w, X_i) \\ P_w &\equiv I_w[\Omega_w] \end{aligned} \tag{158}$$

where eqn. (158) is a linear function of Ω_w that maps the bivector rate in the world frame Ω_w to the total momentum P_w. It is a function that takes a bivector as an argument and returns a bivector (the "inertia map" I_w). It describes the influence of the mass distribution of the points on the movement via the combined Newton/Euler law

$$\dot{P}_w = F_w \tag{159}$$

(for the total force in the *world frame* $F_{total} = F_w$ see eqn. (148)). If we go from discrete mass points to a continuous mass distribution, the sum has to be replaced by an integral over $dm = \rho dV$ (see following subsections on inertia maps for more details).

The equation $\dot{P}_w = X_w \wedge m\ddot{X}_w = F_w$ cannot be integrated directly since the second derivative $\ddot{X}_w$ cannot be isolated on the left-hand side of the equation as required for direct integration. To circumvent this we introduce another variable that can be isolated, enabling us to solve the ODE. For this purpose we use the rate of change Ω (a vector in $2D$ and a bivector in $3D$). The integration of that variable will then allow for computation of position, speed, etc. as a separate step.

Rate of change: Calculating the derivative $\dot{P}_w$ in the world frame in general yields two terms

$$\dot{P}_w = \dot{I}_w[\Omega_w] + I_w[\dot{\Omega}_w] \tag{160}$$

since in the world frame the inertia $I_w[\Omega_w]$ map will change over time due to moving points as well as due to the rate of change Ω_w. Therefore, it is reasonable to calculate the inertia map in the body frame, since it is constant in that frame, if the body frame is chosen to be fixed to the modelled rigid object. This avoids the effort to additionally calculate the time derivative of the inertia map (the first term on the right-hand side above).

Body frame: If we want to express the physical problem in the body frame, we have to transform the components of eqn. (158) accordingly. Using eqn. (118b), i.e. $I_b = \underset{\sim}{M} \vee I_w \vee M$ and eqn. (118a) $\Omega_w = M \vee \Omega_b \vee \underset{\sim}{M}$ then results in

$$\begin{aligned} P_w &= I_w[\Omega_w] \\ \underbrace{\underset{\sim}{M} \vee P_w \vee M}_{P_b} &= \underbrace{\underset{\sim}{M} \vee I_w[\underset{\sim}{M} \vee \Omega_w \vee M] \vee M}_{I_b[\Omega_b]} \end{aligned}$$

which finally results in an expression for the inertia map

$$P_b = I_b[\Omega_b] \equiv \underset{\sim}{M} \vee I_w[M \vee \Omega_b \vee \underset{\sim}{M}] \vee M \tag{161}$$

The same transformation must be applied to F_w for the right-hand side of the differential equation $\dot{P}_w = F_w$. Therefore, we get in the body frame $F_b = \underset{\sim}{M} \vee F_w \vee M$, resulting in $\dot{P}_b = F_b$.

The time derivative in the body frame is simpler, as long as we restrict ourselves to rigid bodies, resulting in only one term, since the rate of change Ω_b is the only time dependent function in the body frame

$$\dot{P}_b = I_b[\dot{\Omega}_b] \tag{162}$$

otherwise $\dot{I}_b[\Omega_b]$ would have to be covered, as in eqn. (160) in the world frame.

The mapping to the classical inertia can be done when splitting Ω_b relative to the centroid of the body with a projective point split on Ω_b and on $I_b[\Omega_b]$ as shown by [Dorst and De Keninck, 2023] on page 25f and adjusted to our approach using regressive geometric products.

Reference points: Now we introduce some typically used reference points to describe kinematics via the bivector rate of change Ω and dynamics via the total momentum P:

- *Fixed body reference point* O_b used as origin of the body frame. When the body is moved by the motor M it will move in the world frame to $O_{b,w} = M \vee O_b \vee \underset{\sim}{M}$.
- *Relative locations* r_i *versus* O_b are positions in the body frame. Their projective counterpart in the body frame is $R_i = O_b + r_i$. When transformed to the world frame we get X_i in the world frame by the usual transformation $X_i = M \vee R_i \vee \underset{\sim}{M} = M \vee (O_b + r_i) \vee \underset{\sim}{M}$.

- *Center of mass* in the body frame is chosen to be located at O_b, such that $\sum_i m_i r_i = \vec{0}$. Thus, the origin of the body frame is located at the center of mass. The center of mass in the world frame C_w then is $C_w \equiv \sum_i m_i X_i / \sum_i m_i = \sum_i m_i (M \vee (O_b + r_i) \vee \underset{\sim}{M}) / \sum_i m_i = M \vee O_b \vee \underset{\sim}{M}$.

The last missing piece for dynamics is the time derivative of P_w expressed as time rate of change Ω_b of the inertia in the body system. Applying eqn. (119) in the body frame, which is valid for any rigid object moved by a motor M, we get

$$\frac{d}{dt} P_b = \frac{d}{dt} I_b[\Omega_b] = \text{rcmt}(\Omega_b, I_b[\Omega_b]) + I_b[\dot{\Omega}_b] \tag{163}$$

where the last term arises since Ω_b is also time-dependent. Solving eqn. (159), but in the body frame, we get

$$\dot{P}_b = \text{rcmt}(\Omega_b, I_b[\Omega_b]) + I_b[\dot{\Omega}_b] = F_b \tag{164}$$

resolving this for $\dot{\Omega}_b$ we get

$$I_b[\dot{\Omega}_b] = F_b - \text{rcmt}(\Omega_b, I_b[\Omega_b]) \tag{165}$$

$$\dot{\Omega}_b = I_b^{-1}\left[F_b - \text{rcmt}(\Omega_b, I_b[\Omega_b])\right] \tag{166}$$

Together with the ansatz in the body system eqn. (114) that can be used to calculate M(t) when we know Ω_b at a given time t from

$$M(t) = M_0 \vee \exp_\vee(\frac{1}{2} B_b(t)) = M_0 \vee \exp_\vee(\frac{1}{2} \Omega_b t)$$

and the rate of change in the body frame according to eqn. (116) we get

$$\dot{\Omega}_b = I_b^{-1}\left[F_b - \text{rcmt}(\Omega_b, I_b[\Omega_b])\right] \tag{167}$$

as an ODE that can be solved as initial value problem. Due to $\dot{\Omega}_b = \ddot{B}_b$ it actually is a 2^{nd} order ODE (like the classical Newton/Euler equations that can be integrated directly from the instantaneous acceleration and the angular acceleration, respectively, in order to calculate speed and position). Here we rather integrate from $\ddot{B}_b$ in order to calculate $\dot{B}_b = \Omega_b$ and B_b. Thus, we require initial values for Ω_b and B_b. The actual positions of the points of the rigid body can then be computed from eqn. (91) $X_w(t) = M(t) \vee X_0 \vee \underset{\sim}{M}(t)$. The speed of the points can be calculated either from eqn. (119) $\dot{X}_w = \text{rcmt}(\Omega_w, X_w)$ or eqn. (121) $\dot{X}_w = \text{rcmt}(\Omega_w, X_w) + (\dot{X}_b)_w$ – which one is applicable depends on whether the rigid body is fixed in the body frame or moves in the body frame. The expressions in the world frame can of course be expressed in the body frame as well using eqn. (118a) $\Omega_w = M \vee \Omega_b \vee \underset{\sim}{M}$ and $X_w = M \vee X_b \vee \underset{\sim}{M}$.

Inertia map ($2D$): For that purpose we need the inertia map $I_b[\Omega_b]$, and it's inverse to set up the right-hand side of the ODE system given in equations (167). We start with the $2D$ case where the rate of change Ω_b is a vector, as shown at the end of subsection 1.3. We assume discrete points X_i and use the wedge product $X \wedge Y$ and the regressive commutator product term $m\,\mathrm{rcmt}(\Omega_b, X)$ to express the total momentum P written in component form:

$$X \wedge Y = \begin{pmatrix} X_x \\ X_y \\ X_z \end{pmatrix} \wedge \begin{pmatrix} Y_x \\ Y_y \\ Y_z \end{pmatrix} = \begin{pmatrix} X_z Y_x - X_x Y_z \\ X_z Y_y - X_y Y_z \\ X_x Y_y - X_y Y_x \end{pmatrix} \tag{168a}$$

$$m\,\mathrm{rcmt}(\Omega_b, X) = m \begin{pmatrix} \Omega_y X_z - \Omega_z X_y \\ \Omega_z X_x - \Omega_x X_z \\ 0 \end{pmatrix} \tag{168b}$$

Now the inertia map for a point X with mass m and rate of change Ω can be expressed in the $2D$ case as

$$\begin{aligned} I_{2D}[\Omega] &= X \wedge m\,\mathrm{rcmt}(\Omega, X) \\ &= m \begin{pmatrix} X_z(\Omega_y X_z - \Omega_z X_y) \\ X_z(\Omega_z X_x - \Omega_x X_z) \\ X_x(\Omega_z X_x - \Omega_x X_z) - X_y(\Omega_y X_z - \Omega_z X_y) \end{pmatrix} \\ I_{2D}[\Omega] &= m \begin{bmatrix} 0 & X_z^2 & -X_y X_z \\ -X_z^2 & 0 & X_x X_z \\ -X_x X_z & -X_y X_z & X_x^2 + X_y^2 \end{bmatrix} \begin{bmatrix} \Omega_x \\ \Omega_y \\ \Omega_z \end{bmatrix} \end{aligned} \tag{169}$$

where eqn. (169) is linear in Ω as expected. Of course, the total inertia map needs to be summed for the contributions from each point, i.e. $I_{total} = \sum_i I_i$. Since all points of the rigid body see the same rate of change the total inertia map can be summed for all mass point contributions independent of the rate of change and the resulting total inertia map can be inverted either symbolically or numerically to get its inverse I_{2D}^{-1} as needed to set up the right-hand side of the system in eqn. (167). The inertia map in $2D$ maps the rate of change vector to the momentum bivector. The inverse inertia map than maps from a momentum bivector to the rate of change vector.

Inertia map ($3D$): Now let's do the same for the $3D$ case, where the rate of change is a bivector. The component expressions are generated from the

corresponding product tables in `ga_prdxpr` and integrated into the products in the ga library.

$$X \wedge Y = \begin{pmatrix} X_x \\ X_y \\ X_z \\ X_w \end{pmatrix} \wedge \begin{pmatrix} Y_x \\ Y_y \\ Y_z \\ Y_w \end{pmatrix} = \begin{pmatrix} X_w Y_x - X_x Y_w \\ X_w Y_y - X_y Y_w \\ X_w Y_z - X_z Y_w \\ X_y Y_z - X_z Y_y \\ X_z Y_x - X_x Y_z \\ X_x Y_y - X_y Y_x \end{pmatrix} \tag{170a}$$

$$m \operatorname{rcmt}(\Omega_b, X) = m \begin{pmatrix} \Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w \\ \Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w \\ \Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w \\ 0 \end{pmatrix} \tag{170b}$$

Now the inertia map for a point X with mass m and rate of change Ω can be expressed in the $3D$ case as

$$I_{3D}[\Omega] = X \wedge m \operatorname{rcmt}(\Omega, X) =$$

$$m \begin{pmatrix} X_w(\Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w) \\ X_w(\Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w) \\ X_w(\Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w) \\ X_y(\Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w) - X_z(\Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w) \\ X_z(\Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w) - X_x(\Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w) \\ X_x(\Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w) - X_y(\Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w) \end{pmatrix} =$$

$$m \begin{bmatrix} 0 & X_z X_w & -X_y X_w & X_w^2 & 0 & 0 \\ -X_z X_w & 0 & X_x X_w & 0 & X_w^2 & 0 \\ X_y X_w & -X_x X_w & 0 & 0 & 0 & X_w^2 \\ X_y^2 + X_z^2 & -X_x X_y & -X_x X_z & 0 & -X_z X_w & X_y X_w \\ -X_x X_y & X_x^2 + X_z^2 & -X_y X_z & X_z X_w & 0 & -X_x X_w \\ -X_x X_z & -X_y X_z & X_x^2 + X_y^2 & -X_y X_w & X_x X_w & 0 \end{bmatrix} \begin{bmatrix} \Omega_{vx} \\ \Omega_{vy} \\ \Omega_{vz} \\ \Omega_{mx} \\ \Omega_{my} \\ \Omega_{mz} \end{bmatrix} \tag{171}$$

where eqn. (171) is linear in Ω as expected. Again, the total inertia map needs to be summed for the contributions from each point, i.e. $I_{total} = \sum_i I_i$. The resulting total inertia map can be inverted numerically to get its inverse I_{3D}^{-1} as needed to set up the right-hand side of the system in eqn. (167). The inertia map in $3D$ maps the rate of change bivector to the momentum bivector. The inverse inertia map than maps from a momentum bivector to the corresponding rate of change bivector.

As shown in [Dorst and De Keninck, 2023] the inertia map is additive. That means that inertia maps of parts of a body consisting of several parts that move with the same bivector rate can be added to get the total inertia.

1.4.3 Solving the Newton-Euler equations numerically

In classical mechanics the Newton equation states that change of linear momentum $\vec{p} = m\vec{v}$ of a mass point of mass m moving with speed $\vec{v}$ is driven by an external force $\vec{f}$ acting on the mass. This results in $\dot{\vec{p}} = m\dot{\vec{v}} = m\ddot{\vec{x}}_{cm} = \vec{f}$ for translational motion in a system with constant mass m. The center of mass at position $\vec{x}_{cm}$ moves with the velocity $\vec{v}$ as a result.

The analogous equation for rotational motion, attributed to Euler, states that the temporal change of angular momentum $\vec{L} = \vec{x} \times \vec{p} = \vec{x} \times m\vec{v}$ is driven by an external angular momentum $\vec{M}$. $\vec{M}$ is given as $\vec{M} = \vec{r} \times \vec{f}$ with $\vec{r}$ as the vector pointing from the rotational axis to the point of action of the force $\vec{f}$. The equation for rotational motion $\dot{\vec{L}} = J\dot{\vec{\omega}} = \vec{M}$ for a rigid body with moment of inertia $J = \int_m r_\perp^2 dm = \int_V r_\perp^2 \rho dV$ with $r_\perp$ as the perpendicular distance of the mass point within the rigid body to the momentary rotational axis. The equation determines the temporal development of the orientation of the rigid body which freely spins with the angular speed $\|\vec{\omega}\|$ around the momentary rotation axis defined by $\vec{\omega}$ as a result.

Due to the same structure the equations of a freely moving body can be written as one system of equations that can be solved as uncoupled equations (e.g. covering cases with either translation or rotation exclusively within the identical structure). In the $2D$ case the translational part requires two degrees of freedom (DoF) and the rotational part needs one DoF resulting in a total of three DoF. It can be written compactly as a system introducing the inertia matrix I as a linear function that maps the acceleration degrees of freedom to the corresponding external force and torque components[8]:

$$\underbrace{\begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & J \end{bmatrix}}_{inertia\ I} \begin{pmatrix} \ddot{x}_x \\ \ddot{x}_y \\ \ddot{\phi} \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ M \end{pmatrix} \tag{172}$$

Solving such a system requires to isolate the accelerations by multiplying the

[8] Be aware, that this does not actually couple the equations! We just solve two independent equations conveniently in parallel to show the principle here. If the movement is actually coupled, coupling terms must be added to model physics correctly. In the *PGA* formulation this coupling is already captured by the rcmt term in the body-frame equation of motion (167).

system from the left with the inverse of the inertia map I^{-1} resulting in

$$\begin{pmatrix} \ddot{x}_x \\ \ddot{x}_y \\ \ddot{\phi} \end{pmatrix} = \underbrace{\begin{bmatrix} \frac{1}{m} & 0 & 0 \\ 0 & \frac{1}{m} & 0 \\ 0 & 0 & \frac{1}{J} \end{bmatrix}}_{inverse\ inertia\ I^{-1}} \begin{pmatrix} f_x \\ f_y \\ M \end{pmatrix} \tag{173}$$

Both equations result in a system of ordinary differential equation (ODE) of second order that requires two initial conditions, one for the initial position $\vec{x}(t = 0) = \vec{x}_0$ and the initial speed $\vec{v}(t = 0) = \vec{v}_0$ (Newton equation) and one for the initial orientation $\phi(t = 0) = \phi_0$ and the initial angular speed $\omega(t = 0) = \omega_0$ (Euler equation) to be integrated. In the $2D$ case ϕ and ω are scalar values, while in the $3D$ case they also become vectors each representing three degrees of freedom.

To integrate the equations the system of 2^{nd} order must be transferred into a system of 1^{st} order. We have a 2^{nd} order ODE for $y(t)$ with $\ddot{y} = \frac{d^2}{dt^2}(y)$ of the general form

$$\ddot{\vec{y}} = f(\dot{\vec{y}}, \vec{y}, t) \tag{174}$$

The vector y has components that we index with n. We use an additional index to distinguish between the position (index 0) and the speed (index 1) of the respective DoF. With the substitution

$$y_{n,0} = y_n \qquad \rightarrow position \tag{175a}$$
$$y_{n,1} = \dot{y}_n \qquad \rightarrow speed \tag{175b}$$

this results in the following system of 1^{st} order for each component n of the vector $\vec{y}$:

$$\dot{y}_{n,0} = y_{n,1} \tag{176a}$$
$$\dot{y}_{n,1} = f(y_{n,1}, y_{n,0}, t) \tag{176b}$$

If we would have to integrate for more than one point, one could easily add an index at the beginning to extend the system accordingly. However, for now we only focus on integrating the system at one point (the center of mass) at which we want to know position and speed as well as angular orientation and angular speed as function of time. The substitution above transforms

the system in eqn. (173) into a system of 1^{st} order as follows:

$$\dot{x}_x = v_x \tag{177a}$$
$$\dot{v}_x = \frac{1}{m} f_x \tag{177b}$$
$$\dot{x}_y = v_y \tag{177c}$$
$$\dot{v}_y = \frac{1}{m} f_y \tag{177d}$$
$$\dot{\phi} = \omega \tag{177e}$$
$$\dot{\omega} = \frac{1}{J} M \tag{177f}$$

This 1^{st} order ODE system $\dot{\vec{u}} = rhs(\vec{u}, t)$ can now be solved by direct integration, using e.g. a Runge-Kutta-Scheme of 4^{th} order accuracy for precise time integration, and using the initial conditions defined above. As a result, we directly get the positions and velocities as function of time.

Classical body frame vs. *PGA* — what unifies: It is instructive to compare the classical body-frame formulation with the single *PGA* equation of motion (167), $\dot{\Omega}_b = I_b^{-1}[F_b - \mathrm{rcmt}(\Omega_b, I_b[\Omega_b])]$. Classically one carries two separate vector equations – Newton's for $\dot{\vec{v}}$ and Euler's for $\dot{\vec{\omega}}$ (the latter with the nonlinear gyroscopic term $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$) – plus a separate orientation update via a rotation matrix or quaternion. The *PGA* formulation collapses all of this into one bivector ODE plus a motor update, as summarised in Table 21.

Classical body frame	*PGA* equivalent
Two separate equations: Newton + Euler	One unified equation for $\dot{\Omega}_b$
Coupling term $\vec{\omega} \times \vec{v}$	Encoded in $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$
Gyroscopic term $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$	Also encoded in $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$
Inertia tensor $\mathbf{I}_b$ (3×3 or 6×6 matrix)	Inertia map $I_b[\cdot]$ (linear operator on bivectors)
Quaternion evolution $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \vec{\omega}$	Motor $M = \exp_\vee[\frac{1}{2}B_b]$ – orientation implicit in B_b
State: $(\vec{r}, \mathbf{q}, \vec{v}, \vec{\omega})$	State: (B_b, Ω_b) – bivector pair
Inertial-frame position from $\dot{\vec{r}} = \mathbf{R}\vec{v}$	Inertial-frame points updated via sandwich product $X(t) = M \vee X_b(t) \vee \underset{\sim}{M}$

Table 21: Classical body-frame vs. *PGA* formulation: explicit correspondence.

Where the classical formulation needs one equation for $\dot{\vec{v}}$ (3 DoF), one for $\dot{\vec{\omega}}$ (3 DoF), and a separate orientation integration, the *PGA* ODE handles the first two in a single equation over the bivector $\dot{\Omega}_b$ (6 DoF) and recovers position and orientation from a motor derived directly from Ω_b. The single term $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$ plays the combined role of both classical coupling terms ($\vec{\omega} \times \vec{v}$ and $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$). As described above, the *PGA* system is first solved for $B(t)$, $\dot{B} = \Omega(t)$ and $\ddot{B} = \dot{\Omega}(t)$; the actual points and their velocities then follow in a second step from the motor and the sandwich transformation.

Basic examples for translational and rotational motion are implemented as tests in `ga_test/src/ga_appl_mechanics_test.hpp` based on the Newton-Euler equations in both approaches, the classical approach and the approach directly formulated in *PGA*, for very simple reference cases.

Constrained motion — rotation about a fixed pivot: The body-frame *PGA* formulation extends naturally to constrained motion. A common case in $2D$ is a rigid body whose rotation is constrained so that a specific body-frame point Q_b (the pivot) remains fixed in the world – for example a plate pendulum fixed at one of its corners. Two questions are to be answered independently:

1. *In which frame are the equations expressed?* (body frame vs. world frame)

2. *About which axis does the body actually rotate?* (center of mass vs. pivot)

The Euler equation $\dot{\Omega}_b = I_{cm}^{-1}[F_b - \mathrm{rcmt}(\Omega_b, I_{cm}[\Omega_b])]$ describes a *free* rigid body rotating about its own center of mass. When the pivot is fixed, the center of mass follows a circular arc of radius r around the pivot, with a displacement of the rotation point relative to the center of mass, effectively increasing the inertia relative to that pivot point. The constraint reaction also creates centripetal acceleration to keep the center of mass moving along the arc. This is felt in the body system at the center of mass as additional centrifugal acceleration. The Steiner (parallel axis) theorem gives the corrected moment of inertia in $2D$:

$$I_{\mathrm{pivot}}[2,2] \;=\; I_{cm}[2,2] + mr^2 \tag{178}$$

so the angular acceleration about the fixed pivot is $\alpha = \tau_{\mathrm{eff}}/I_{\mathrm{pivot}}[2,2]$, *not* $\tau_{\mathrm{eff}}/I_{cm}[2,2]$. This is a correction for the physical rotation axis, not for the

choice of frame – it is required regardless of whether the equations are expressed in the body frame or the world frame.

In the *PGA* implementation the scalar Steiner correction is built into the inertia constructor via an optional pivot parameter:

- $2D$ – *pivot is a point* Q_b (grade 1, `Vec2dp`):
  `get_plate_inertia(m, w, h, Q_b)` updates only $I[2,2]$ by adding $m\,r^2$ where $r^2 = Q_{b,x}^2 + Q_{b,y}^2$ is the squared distance from the body origin to Q_b. All other entries (including off-diagonal coupling terms) remain equal to I_{body}. The default $Q_b = (0,0,1)$ (body origin) gives I_{body} with no correction.

- $3D$ – *pivot is a line* L_{pivot} (grade 2, `BiVec3dp`):
  `get_cuboid_inertia(m, w, h, d, L_pivot)` computes the foot of the perpendicular $P_{\text{foot}} = (P_x, P_y, P_z)$ from the body origin to L_{pivot} and updates only the diagonal entries of the lower-left block: $I[3,0]$ += $m(P_y^2+P_z^2)$, $I[4,1]$ += $m(P_x^2+P_z^2)$, $I[5,2]$ += $m(P_x^2+P_y^2)$. Off-diagonal products of inertia are not added. The default $L_{\text{pivot}} = 0$ (pivot through origin) gives no correction. The grade of the pivot entity – point in $2D$, line in $3D$ – matches the grade of the velocity generator Ω (see Tables 22 and 24).

After computing $\dot{\Omega}_b = I_{\text{pivot}}^{-1}[F_b - \text{rcmt}(\Omega_b, I_{\text{pivot}}[\Omega_b])]$ a constraint projection is still required to prevent numerical drift of off-constraint components during RK4 integration. In $2D$ the body-frame state is encoded as $B_b = \varphi \cdot Q_b$ and $\Omega_b = \omega \cdot Q_b$; since $Q_b.z = 1$ the scalar angular acceleration is

$$\alpha = (\dot{\Omega}_b)_z, \qquad \dot{\Omega}_b = \alpha \cdot Q_b \tag{179}$$

In $3D$ the analogous projection onto the normalized pivot line gives $\alpha = \dot{\Omega}_b \cdot \hat{L}_{\text{pivot}}$. The body-frame encoding $B_b = \varphi \cdot Q_b$ carries a key geometric property: the world image of Q_b is stationary for all φ. From the body-frame motor formula (114), $M(t) = M_0 \vee R(\varphi)$ with the pure pivot rotation $R(\varphi) = \exp_\vee(\frac{1}{2}B_b(t)) = \exp_\vee(\frac{1}{2}\varphi Q_b)$ and the constant base motor $M_0 = M(t{=}0)$. A finite point is invariant under a rotation about itself, so $R(\varphi) \vee Q_b \vee \underbrace{R(\varphi)} = Q_b$, and the world image of the pivot reduces to

$$M(t) \vee Q_b \vee \underbrace{M(t)} \;=\; M_0 \vee Q_b \vee \underbrace{M_0} \;=\; Q_{b,\text{world}} = \text{const} \tag{180}$$

No explicit constraint enforcement is required for the *position* of the pivot – it is preserved exactly for all φ by the motor algebra. Only the velocity direction (eqn. (179)) must be enforced at each integration step.

1.4.4 Structure of inertia and inertia maps

The inertia map $I[\Omega]$ derived in the preceding subsections is a linear map that takes the velocity twist Ω of a rigid body and returns its momentum wrench $P = I[\Omega]$. Its matrix representation – shown in eqns. (169) and (171) – encodes mass, scalar moment of inertia, and the full classical inertia tensor in a single algebraic object. However, the classical quantities do not sit where standard intuition might suggest: mass does not appear on the diagonal, and moments of inertia do not appear in the block connecting forces to accelerations of the same type.

The placement of each entry is determined by two ingredients: the choice of basis for the input and output of the inertia map, and the complement structure of the respective algebra. The right complement $\overline{(\cdot)}$ maps each basis element to its complement in the algebra. In PGA the translational and rotational generators live in dual subspaces – ideal lines are dual to real lines – so the mass contribution, which couples velocity to momentum across dual subspaces, always lands in an off-diagonal block. The following paragraphs explain this structure first for $2D$, then for $3D$, and concludes with a unified comparison.

PGA2DP — **structure of the 2D inertia map:** In $2D$ PGA the velocity twist Ω is a vector and the momentum wrench P is a bivector, so the inertia map crosses two different grades:

$$I_{2D}[\Omega] : \text{Vec2dp} \to \text{BiVec2dp} \tag{181}$$

The basis elements and their physical roles are listed in Table 22.

Index	Vec2dp input (velocity twist Ω)	BiVec2dp output (momentum wrench P)
0	$\boldsymbol{e}_1$ – x-velocity	$\boldsymbol{e}_{31}$ – x-momentum
1	$\boldsymbol{e}_2$ – y-velocity	$\boldsymbol{e}_{32}$ – y-momentum
2	$\boldsymbol{e}_3$ – angular velocity	$\boldsymbol{e}_{12}$ – angular momentum

Table 22: Basis correspondence for the $2D$ PGA inertia map.

For a unitized point (X_x, X_y, X_z) with projective coordinate $X_z = 1$ and mass m, the inertia matrix from eqn. (169) simplifies to

$$I_{2D} = m \begin{bmatrix} 0 & 1 & -X_y \\ -1 & 0 & X_x \\ -X_x & -X_y & X_x^2 + X_y^2 \end{bmatrix} \tag{182}$$

where rows correspond to output components ($\boldsymbol{e}_{31}$, $\boldsymbol{e}_{32}$, $\boldsymbol{e}_{12}$) and columns to input components (Ω_x, Ω_y, Ω_z).

Why mass is off-diagonal: The mass contribution to the inertia map takes the form $I_{\text{mass}}(\Omega) = -m \cdot \overline{\Omega}$, where $\overline{(\cdot)}$ denotes the complement in $PGA2DP$. The complement of the translational basis vectors crosses indices:

$$\overline{\boldsymbol{e}_1} = \boldsymbol{e}_{32}, \qquad \overline{\boldsymbol{e}_2} = -\boldsymbol{e}_{31} \tag{183}$$

Applying $I_{\text{mass}} = -m \cdot \overline{(\cdot)}$ gives the following contributions:

- Ω_x (component along $\boldsymbol{e}_1$, column 0) maps to $-m \cdot \boldsymbol{e}_{32}$, landing at output row $1 \to I[1,0] = -m$.
- Ω_y (component along $\boldsymbol{e}_2$, column 1) maps to $-m \cdot (-\boldsymbol{e}_{31}) = +m \cdot \boldsymbol{e}_{31}$, landing at output row $0 \to I[0,1] = +m$.

The diagonal entries $I[0,0]$ and $I[1,1]$ are zero because the complement of $\boldsymbol{e}_1$ is $\boldsymbol{e}_{32}$ (not $\boldsymbol{e}_{31}$) and the complement of $\boldsymbol{e}_2$ is $-\boldsymbol{e}_{31}$ (not $\boldsymbol{e}_{32}$): there is no self-coupling. This is a direct consequence of the degenerate metric $\boldsymbol{e}_3^2 = 0$.

The resulting mass block is antisymmetric with $I[0,1] = +m$ and $I[1,0] = -m$. The sign structure is consistent with the kinematic encoding: the motor-generator Ω for a pure translation with classical velocity (v_x, v_y) is encoded in $2D$ PGA as $\Omega = (-v_y, v_x, 0)$. The two sign flips cancel and the classical linear momentum $p_x = mv_x$, $p_y = mv_y$ is correctly recovered. In any other context – position, force, acceleration – $\boldsymbol{e}_1$ always encodes the x-component and $\boldsymbol{e}_2$ the y-component.

Moment of inertia and cross-coupling terms: The entry $I[2,2] = m(X_x^2 + X_y^2) = mr^2$ maps angular velocity ($\boldsymbol{e}_3$, index 2) to angular momentum ($\boldsymbol{e}_{12}$, index 2), recovering the classical scalar moment of inertia $J = mr^2$ for a point mass at distance r from the rotation center. The complement of $\boldsymbol{e}_3$ is $\overline{\boldsymbol{e}_3} = -\boldsymbol{e}_{12}$, which connects the angular velocity generator directly to the angular momentum carrier; the sign is absorbed into the definition so that $J > 0$. The entries $I[0,2] = -mX_y$ and $I[1,2] = +mX_x$ couple angular velocity to linear momentum, encoding the classical relation $\vec{L}_{\text{linear}} = m(\vec{\omega} \times \vec{r})$ for a body rotating at offset (X_x, X_y) from the rotation center. These entries, together with their transposes $I[2,0]$ and $I[2,1]$, vanish when the point is at the origin.

Body-frame inertia matrix: Integrating the single-point formula over a rigid body with center of mass at the origin (body frame) gives:

$$I_{\text{body}} = \begin{bmatrix} 0 & m & 0 \\ -m & 0 & 0 \\ 0 & 0 & J \end{bmatrix} \tag{184}$$

where $J = \int_m (x^2 + y^2)\, dm$ is the classical scalar moment of inertia about the center of mass. The antisymmetric mass block (upper-left 2×2) and the moment of inertia entry (lower-right 1×1) are cleanly separated: translational and rotational degrees of freedom decouple at the center of mass, exactly as in classical mechanics.

General frame with center of mass at (c_x, c_y): When the chosen reference point does not coincide with the center of mass, the integrated inertia matrix is:

$$I = \begin{bmatrix} 0 & m & -mc_y \\ -m & 0 & mc_x \\ -mc_x & -mc_y & J_{\text{cm}} + m(c_x^2 + c_y^2) \end{bmatrix} \tag{185}$$

where J_{cm} is the scalar moment of inertia about the center of mass and $r^2 = c_x^2 + c_y^2$ is the squared distance from the center of mass to the reference point. The matrix decomposes into the following sub-blocks:

- *Upper-left* 2×2 $[0, m;\ -m, 0]$: the antisymmetric mass block. It is *frame-invariant* – its values are unchanged regardless of where the center of mass is located.
- *Upper-right* 2×1 $[-mc_y;\ mc_x]$: angular velocity → linear momentum coupling. Encodes $\vec{L}_{\text{linear}} = m(\vec{\omega} \times \vec{r}_{\text{cm}})$. Zero only when the center of mass coincides with the reference point.
- *Lower-left* 1×2 $[-mc_x,\ -mc_y]$: translational velocity → angular momentum coupling. Transpose of the upper-right block (up to sign convention).
- *Lower-right* 1×1: $J_{\text{cm}} + m(c_x^2 + c_y^2)$ – the classical parallel axis theorem applied to the scalar moment of inertia.

Uniform rectangular plate: As a verification, integrating over a uniform plate of width W (along $\boldsymbol{e}_1$), height H (along $\boldsymbol{e}_2$), total mass M, centered at

the origin gives:

$$I_{\text{plate}} = M \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & \dfrac{W^2 + H^2}{12} \end{bmatrix} \tag{186}$$

The entry $I[2,2] = M(W^2 + H^2)/12$ is the standard formula for the moment of inertia of a uniform rectangle about its centroid axis. For a discrete endpoint-inclusive grid of $n \times n$ equally spaced point masses, the relative error in $I[2,2]$ compared to the continuous value is $2/(n-1)$, independent of plate dimensions, converging from above as $n \to \infty$.

Table 23 summarizes the classical inertial quantities and their locations in the $2D$ PGA inertia matrix.

Classical quantity	$2D$ PGA entry	Condition
Total mass m	$I[0,1] = +m$, $I[1,0] = -m$	Always
Scalar moment J about CoM	$I[2,2] = J$	CoM at origin
CoM offset (c_x, c_y)	$I[0,2] = -mc_y$, $I[1,2] = +mc_x$ (and transposed)	General frame
Parallel axis correction	$I[2,2] = J_{\text{cm}} + m(c_x^2 + c_y^2)$	General frame

Table 23: Classical inertial quantities and their location in the $2D$ PGA 3×3 inertia matrix.

$3D$ PGA — **structure of the 3D inertia map:** In $3D$ PGA both the velocity twist Ω and the momentum wrench P are bivectors, so the inertia map acts within the same grade:

$$I_{3D}[\Omega] : \text{BiVec3dp} \to \text{BiVec3dp} \tag{187}$$

The six basis bivectors split into two groups: *ideal lines* ($\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, containing the projective direction $\boldsymbol{e}_4$ which squares to zero) carrying translational information, and *real lines* ($\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$) of the Euclidean subspace carrying rotational information. Their physical roles are listed in Table 24.

Idx	BiVec3dp input (velocity twist Ω)	BiVec3dp output (momentum wrench P)
0	$\boldsymbol{e}_{41}$ – ideal line, x-velocity	$\boldsymbol{e}_{41}$ – linear momentum x
1	$\boldsymbol{e}_{42}$ – ideal line, y-velocity	$\boldsymbol{e}_{42}$ – linear momentum y
2	$\boldsymbol{e}_{43}$ – ideal line, z-velocity	$\boldsymbol{e}_{43}$ – linear momentum z
3	$\boldsymbol{e}_{23}$ – real line, angular velocity about $\boldsymbol{e}_1$	$\boldsymbol{e}_{23}$ – angular momentum about $\boldsymbol{e}_1$
4	$\boldsymbol{e}_{31}$ – real line, angular velocity about $\boldsymbol{e}_2$	$\boldsymbol{e}_{31}$ – angular momentum about $\boldsymbol{e}_2$
5	$\boldsymbol{e}_{12}$ – real line, angular velocity about $\boldsymbol{e}_3$	$\boldsymbol{e}_{12}$ – angular momentum about $\boldsymbol{e}_3$

Table 24: Basis correspondence for the $3D$ PGA inertia map.

The 6×6 inertia matrix from eqn. (171) for a unitized point with $X_w = 1$ is:

$$I_{3D} = m \begin{bmatrix} 0 & X_z & -X_y & 1 & 0 & 0 \\ -X_z & 0 & X_x & 0 & 1 & 0 \\ X_y & -X_x & 0 & 0 & 0 & 1 \\ X_y^2 + X_z^2 & -X_x X_y & -X_x X_z & 0 & -X_z & X_y \\ -X_x X_y & X_x^2 + X_z^2 & -X_y X_z & X_z & 0 & -X_x \\ -X_x X_z & -X_y X_z & X_x^2 + X_y^2 & -X_y & X_x & 0 \end{bmatrix} \tag{188}$$

where columns correspond to $(\Omega_{vx}, \Omega_{vy}, \Omega_{vz}, \Omega_{mx}, \Omega_{my}, \Omega_{mz})$ and rows to the same ordering for P.

Block structure and the complement in 3D: The 6×6 matrix decomposes into four 3×3 blocks with distinct physical roles:

- *Upper-left [0:3, 0:3]*: translational velocity → linear momentum. For a point at the origin this block is zero. For a point displaced to (X_x, X_y, X_z) it becomes the antisymmetric cross-product matrix $[X\times]$, encoding the linear momentum contribution due to the off-axis position.

- *Upper-right [0:3, 3:6]*: angular velocity → linear momentum. For a unitized point ($X_w = 1$) this block equals $m\mathbf{I}_3$ (mass times the 3×3 identity). This is the $3D$ analogue of the off-diagonal mass entries in the $2D$ case.

- *Lower-left [3:6, 0:3]*: translational velocity → angular momentum. This block carries the classical moments and products of inertia.

- *Lower-right [3:6, 3:6]*: angular velocity → angular momentum. Zero for a point at the origin; becomes antisymmetric for a displaced center of mass.

The placement of mass in the upper-right block follows from the complement in $3D$ *PGA*, which maps real lines to ideal lines:

$$\overline{\boldsymbol{e}_{23}} = \boldsymbol{e}_{41}, \quad \overline{\boldsymbol{e}_{31}} = \boldsymbol{e}_{42}, \quad \overline{\boldsymbol{e}_{12}} = \boldsymbol{e}_{43} \tag{189}$$

Applying $I_{\text{mass}} = -m \cdot \overline{(\cdot)}$ maps each angular velocity component to the corresponding linear momentum component, placing m on the diagonal of the upper-right block: $I[0,3] = I[1,4] = I[2,5] = m$. The diagonal entries of the upper-left and lower-right blocks are zero by the same degenerate-metric argument as in $2D$.

Body-frame inertia matrix: Integrating the single-point formula over a rigid body with center of mass at the origin gives:

$$I_{\text{body}} = \begin{bmatrix} 0 & 0 & 0 & m & 0 & 0 \\ 0 & 0 & 0 & 0 & m & 0 \\ 0 & 0 & 0 & 0 & 0 & m \\ I_{xx} & -I_{xy} & -I_{xz} & 0 & 0 & 0 \\ -I_{xy} & I_{yy} & -I_{yz} & 0 & 0 & 0 \\ -I_{xz} & -I_{yz} & I_{zz} & 0 & 0 & 0 \end{bmatrix} \tag{190}$$

written in block form as $I_{\text{body}} = [\mathbf{0},\ m\mathbf{I}_3;\ \mathbf{J}_{cm},\ \mathbf{0}]$, where $\mathbf{J}_{cm}$ is the classical 3×3 inertia tensor about the center of mass:

$$\mathbf{J}_{cm} = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix} \tag{191}$$

with the classical integrals for the diagonal moments $I_{xx} = \int_m (y^2 + z^2)\, dm$, $I_{yy} = \int_m (x^2 + z^2)\, dm$, $I_{zz} = \int_m (x^2 + y^2)\, dm$ and the off-diagonal products of inertia $I_{xy} = \int_m xy\, dm$, $I_{xz} = \int_m xz\, dm$, $I_{yz} = \int_m yz\, dm$. Translational and rotational degrees of freedom decouple at the center of mass: the diagonal blocks are zero and all physical content sits in the off-diagonal blocks.

General frame with center of mass at $\vec{r}_{cm} = (c_x, c_y, c_z)$: In a general frame all four 3×3 blocks are non-zero:

$$I = \begin{bmatrix} 0 & mc_z & -mc_y & m & 0 & 0 \\ -mc_z & 0 & mc_x & 0 & m & 0 \\ mc_y & -mc_x & 0 & 0 & 0 & m \\ I_{xx} & -I_{xy} & -I_{xz} & 0 & -mc_z & mc_y \\ -I_{xy} & I_{yy} & -I_{yz} & mc_z & 0 & -mc_x \\ -I_{xz} & -I_{yz} & I_{zz} & -mc_y & mc_x & 0 \end{bmatrix} \tag{192}$$

where the lower-left inertia tensor entries include the classical parallel axis corrections:

$$\begin{aligned} I_{xx} &= I_{xx,cm} + m(c_y^2 + c_z^2), & I_{xy} &= I_{xy,cm} + mc_x c_y \\ I_{yy} &= I_{yy,cm} + m(c_x^2 + c_z^2), & I_{xz} &= I_{xz,cm} + mc_x c_z \\ I_{zz} &= I_{zz,cm} + m(c_x^2 + c_y^2), & I_{yz} &= I_{yz,cm} + mc_y c_z \end{aligned} \tag{193}$$

Two structural features carry over from the $2D$ case:

- *The upper-right block $m\,\mathbf{I}_3$ is frame-invariant.* It remains equal to $m\,\mathbf{I}_3$ regardless of the center-of-mass position. Mass is a dual coupling between ideal and real lines that does not depend on position. This is the $3D$ counterpart of the invariant antisymmetric mass block in $2D$.

- *The upper-left and lower-right blocks are conjugate antisymmetric matrices.* Upper-left $= m[\vec{r}_{cm}\times]^\top$ and lower-right $= m[\vec{r}_{cm}\times]$, where $[\vec{r}_{cm}\times]$ is the 3×3 antisymmetric cross-product matrix of the center-of-mass position vector. Both blocks are driven by the same offset $\vec{r}_{cm}$, reflecting that the translational-to-translational and angular-to-angular couplings induced by the offset are conjugate couplings.

This 6×6 structure is the PGA form of the classical spatial inertia matrix from rigid body dynamics [Dorst and De Keninck, 2023], expressed in the (translational, angular) velocity ordering of $3D$ PGA.

Uniform rectangular cuboid: For a uniform cuboid of width w (along $\boldsymbol{e}_1$), height h (along $\boldsymbol{e}_2$), depth d (along $\boldsymbol{e}_3$), total mass m, centered at the

origin, all cross-inertia and center-of-mass coupling terms vanish:

$$I_{\text{cuboid}} = m \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ \frac{h^2+d^2}{12} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{w^2+d^2}{12} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{w^2+h^2}{12} & 0 & 0 & 0 \end{bmatrix} \tag{194}$$

The three diagonal entries of the lower-left block are the classical rectangular moments of inertia: $I[3,0] = m(h^2+d^2)/12$ about the x-axis, $I[4,1] = m(w^2+d^2)/12$ about the y-axis, and $I[5,2] = m(w^2+h^2)/12$ about the z-axis – each determined by the two extents of the body perpendicular to the respective rotation axis, as in the classical formula.

Table 25 gives the correspondence between classical inertial quantities and their location in the $3D$ PGA inertia matrix.

Classical quantity	$3D$ PGA block	Entries (body frame, CoM at origin)
Total mass m	Upper-right diagonal ($m\mathbf{I}_3$)	$I[0,3] = I[1,4] = I[2,5] = m$
Diagonal moments I_{xx}, I_{yy}, I_{zz}	Lower-left diagonal	$I[3,0]$, $I[4,1]$, $I[5,2]$
Products of inertia I_{xy}, I_{xz}, I_{yz}	Lower-left off-diagonal	$I[3,1] = I[4,0] = -I_{xy}$, etc.
CoM offset (general frame)	All four 3×3 blocks	via parallel axis theorem

Table 25: Classical inertial quantities and their location in the $3D$ PGA 6×6 inertia matrix (body frame, center of mass at origin).

Unified view — 2D and 3D: In both $2D$ PGA and $3D$ PGA the mass contribution to the inertia map takes the same algebraic form:

$$I_{\text{mass}}(\Omega) = -m \cdot \overline{\Omega} \tag{195}$$

The complement maps each velocity generator to its momentum carrier in the dual subspace, placing mass in the off-diagonal block that connects dual

velocity and momentum spaces. The location of that block in the matrix depends on the basis ordering.

In $2D$ *PGA* the translational generators ($\boldsymbol{e}_1$, $\boldsymbol{e}_2$, indices 0–1) sit at lower indices and the rotational generator ($\boldsymbol{e}_3$, index 2) sits at the highest index. The complement maps translational inputs to translational outputs while crossing x and y, placing mass in the *upper-left* 2×2 sub-block (translational velocity → linear momentum).

In $3D$ *PGA* the translational generators ($\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, indices 0–2) are ideal lines at lower indices, while the rotational generators ($\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$, indices 3–5) are real lines at higher indices. The complement maps real-line inputs (angular velocity) to ideal-line outputs (linear momentum), placing mass in the *upper-right* 3×3 sub-block (angular velocity → linear momentum).

Table 26 gives the side-by-side comparison for the body frame (center of mass at origin).

Classical quantity	$2D$ *PGA* (3×3)	$3D$ *PGA* (6×6)
Total mass m	Upper-left 2×2: $I[0,1] = +m$, $I[1,0] = -m$	Upper-right 3×3 diag.: $I[0,3] = I[1,4] = I[2,5] = m$
Scalar/diagonal moments	Lower-right 1×1: $I[2,2] = J$	Lower-left 3×3 diag.: $I[3,0], I[4,1], I[5,2]$
Products of inertia	n/a (scalar J in $2D$)	Lower-left 3×3 off-diagonal
CoM coupling	$I[0,2], I[1,2]$ and transposed	All four 3×3 blocks non-zero
Parallel axis	$I[2,2] = J_{\text{cm}} + mr^2$	Lower-left 3×3 (3D formula)

Table 26: Unified mapping of classical inertial quantities to the *PGA* inertia matrix (body frame, center of mass at origin).

The frame-invariance of the mass block and the decoupling at the center of mass are common to both dimensions. In the body frame with the origin at the center of mass – the natural choice for integrating eqn. (167) – the coupling blocks vanish, the mass block takes its simplest form, and the moment block reduces to J (in $2D$) or $\mathbf{J}_{cm}$ (in $3D$) without any parallel axis corrections. This matches the classical result that Newton's and Euler's equations decouple at the center of mass, and it is the reason the body frame with the center of mass at the origin is used as the preferred reference frame throughout this section. Note, however, that while the inertia map I_{body} is block-diagonal at the center of mass – translational and rotational blocks decoupling in the inertia matrix itself – the equations of motion re-

tain dynamical coupling through the $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$ term. This is the *PGA* counterpart of the classical coupling terms $\vec{\omega} \times \vec{v}$ and $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$, and it arises from the rotation of the body frame, not from the inertia structure.

Pivot-corrected inertia — using the pivot as reference point: The general-frame formulae derived above – eqn. (185) in $2D$ and eqn. (192) in $3D$ – apply directly when the reference point is chosen to coincide with the pivot rather than the center of mass. In that case the center-of-mass offset vector (c_x, c_y) resp. $\vec{r}_{cm}$ points from the pivot to the center of mass, and the inertia map automatically incorporates the full Steiner correction: $I[2,2] = J_{cm} + mr^2$ in $2D$ (eqn. (178)) and the complete tensor update of eqn. (193) in $3D$.

1.4.5 Moving coordinate systems

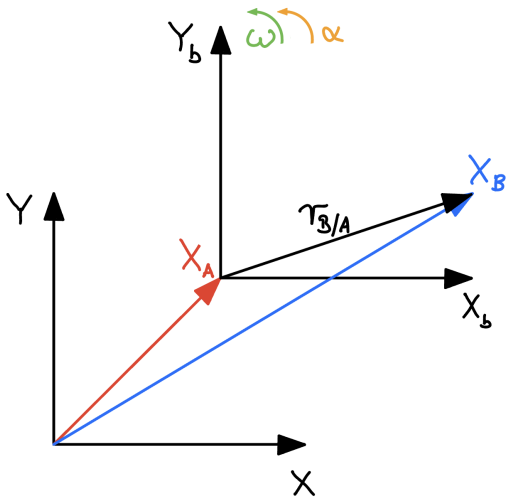


Figure 2: Moving frame: World or reference frame $\{X, Y\}$ (only $2D$ shown as examples, extends naturally to $3D$). Body frame $\{X_b, Y_b\}$. Point A might move with v_A and accelerate with a_A (not shown). Body frame turns with rotational speed ω and rotational acceleration α relative to the world frame.

When modelling motion in moving coordinate systems we look into relative motion. Typically, we use a world frame as reference frame and a body frame as frame moving relative to the world frame, as shown in fig. 2. x_A and x_B are absolute positions in the world frame.

Most of the time x_A is a reference position (e.g. the center of mass of a rigid body)[9], and we are interested in the movement of x_B relative to x_A. The relative position of B with respect to A is shown as $r_{B/A}$. If A is the center of mass of the rigid body then r are local (relative) coordinates of the points of the body in the body frame. So we define

$$r = x_B - x_A \quad \textit{(relative position)} \tag{196a}$$
$$\dot{r} = v_B - v_A \quad \textit{(relative velocity)} \tag{196b}$$
$$\ddot{r} = a_B - a_A \quad \textit{(relative acceleration)} \tag{196c}$$

as relative position, speed and acceleration.

[9]The reference position x_A can be any arbitrary point of the rigid body. However, it is often convenient to choose x_A to be located at the center of mass.

The relative position in a $3D$ frame is defined with (x, y, z) as local or relative coordinates

$$r = x\boldsymbol{e}_1 + y\boldsymbol{e}_2 + z\boldsymbol{e}_3 \tag{197}$$

The total change of the relative position has the following components

$$\dot{r} = \frac{dr}{dt} = \underbrace{\dot{x}\boldsymbol{e}_1 + \dot{y}\boldsymbol{e}_2 + \dot{z}\boldsymbol{e}_3}_{\text{changing speed: } \frac{dr}{dt} = v} + \underbrace{x\frac{d\boldsymbol{e}_1}{dt} + y\frac{d\boldsymbol{e}_2}{dt} + z\frac{d\boldsymbol{e}_3}{dt}}_{\text{changing frame: } \frac{dr}{dt} = \omega \times r} \tag{198}$$

leading to the relative speed $\dot{r}$

$$\dot{r} = \omega \times r_{B/A} + v_{B/A} \tag{199}$$

which can be transformed using eqn. (196b) resulting to the total speed at position B as

$$v_B = v_A + \omega \times r_{B/A} + v_{B/A} \tag{200}$$

We can get the relative acceleration from deriving eqn. (199) again with respect to t

$$\ddot{r} = \dot{\omega} \times r_{B/A} + \omega \times \dot{r}_{B/A} + \dot{v}_{B/A} \tag{201}$$

Using eqn. (199) directly for $\dot{r}_{B/A}$ and for $\dot{v}_{B/A} = \omega \times v_{B/A} + a_{B/A}$ this yields

$$\ddot{r} = \dot{\omega} \times r_{B/A} + \omega \times \omega \times r_{B/A} + \omega \times v_{B/A} + \omega \times v_{B/A} + a_{B/A} \tag{202}$$

when reordered for terms depending on position, relative speed and relative acceleration, we get

$$\ddot{r} = \underbrace{\omega \times \omega \times r_{B/A}}_{\text{centripetal accel. } = -|\omega|^2 \, r_{B/A}} + \underbrace{2\,\omega \times v_{B/A}}_{\text{Coriolis acceleration}} + \underbrace{\dot{\omega} \times r_{B/A}}_{\text{frame acceleration}} + \underbrace{a_{B/A}}_{\text{particle accel.}} \tag{203}$$

Using eqn. (196c) this results in the total acceleration at position B as

$$a_B = a_A + \omega \times \omega \times r_{B/A} + 2\,\omega \times v_{B/A} + \dot{\omega} \times r_{B/A} + a_{B/A} \tag{204}$$

Now we can translate the various velocity and acceleration terms that describe physics using expressions in the geometric algebra language, separately for EGA and PGA. To get there, we mainly replace the cross-product expressions by bivectors in EGA and trivectors in PGA. They are duals of the momentary rotation axis. This directly applies to $2D$ in that form.[10]

[10] The $2D$ application is done in classical vector algebra by using the z-component of the $3D$ cross-product as replacement, even if the cross-product is not defined for $2D$. In the classical approach we temporarily have to include the third dimension and use the relevant z-component exclusively. However, this temporary inclusion of an additional dimension is not required in geometric algebra, since a rotation only requires two dimensions for its representation in geometric algebra.

Term		**Description**
Definition of vectors v and points P	VA	$v = (v_x, v_y, v_z)$, $P = (p_x, p_y, p_z)$
	EGA	$v = (v_x, v_y, v_z)$, $P = (p_x, p_y, p_z)$
	PGA	$v = (v_x, v_y, v_z, 0)$, $P = (p_x, p_y, p_z, 1)$
Definition of rotation	VA	rotation axis vector $\omega = (\omega_x, \omega_y, \omega_z)$ (a direction, axis always contains origin)
	EGA	rotation bivector $\Omega_E = \omega^\star = (\Omega_x, \Omega_y, \Omega_z)$ (bivector dual to direction vector ω – defines the plane of rotation, which always contains the origin)
	PGA	rotation direction $\omega = (\omega_x, \omega_y, \omega_z, 0)$ reference point on rotation axis $Q = (q_x, q_y, q_z, 1)$ rotation bivector $\Omega_P = Q \wedge \omega$ - a line, where $\Omega_P = (\Omega_{vx}, \Omega_{vy}, \Omega_{vz}, \Omega_{mx}, \Omega_{my}, \Omega_{mz})$ (a rotation axis modelled by a bivector, with arbitrary orientation *and* location; can also be created from any unitized line that is scaled with $\lvert\omega\rvert$)
Speed of rotation at r or R	VA	evaluation of velocity at $r = (r_x, r_y, r_z)$ (r is vector perpendicular to axis; points to evaluation point) $v = \omega \times r$
	EGA	evaluation of velocity at $r = (r_x, r_y, r_z)$ (see VA) $v = \mathrm{cmt}(r, \Omega_E) = \Omega_E \gg r$
	PGA	evaluation of velocity at $R = (r_x, r_y, r_z, 1)$ $v = \mathrm{rcmt}(\Omega_P, R)$
Centripetal acceleration	VA	$a_{cp} = \omega \times \omega \times r_{B/A}$
	EGA	$a_{cp} = \mathrm{cmt}(\mathrm{cmt}(r_{B/A}, \Omega_E), \Omega_E) = \Omega_E \gg (\Omega_E \gg r_{B/A})$
	PGA	$a_{cp} = \mathrm{rcmt}(\Omega_P, \mathrm{rcmt}(\Omega_P, r_{B/A}))$
Coriolis acceleration	VA	$a_{co} = 2\,\omega \times v_{B/A}$
	EGA	$a_{co} = 2\,\mathrm{cmt}(v_{B/A}, \Omega_E) = 2(\Omega_E \gg v_{B/A})$
	PGA	$a_{co} = 2\,\mathrm{rcmt}(\Omega_P, v_{B/A})$
Frame rotational acceleration	VA	$a_{frame\,rotation} = \dot{\omega} \times r_{B/A}$
	EGA	$a_{frame\,rotation} = \mathrm{cmt}(r_{B/A}, \dot{\Omega}_E) = (\dot{\Omega}_E \gg r_{B/A})$
	PGA	$a_{frame\,rotation} = \mathrm{rcmt}(\dot{\Omega}_P, r_{B/A})$

Table 27: Description of rotation and rotation speed evaluation for **VA** = Vector Algebra (classical approach using vectors); **EGA** = Euclidean Geometric Algebra; **PGA** = Projective (Euclidean) Geometric Algebra.

1.4.6 Frame trees: static, kinematic and dynamic systems

The preceding subsections developed the mechanics of a *single* rigid body: its pose evolves by a motor $M(t)$, its momentum wrench is the inertia map $P = I[\Omega]$ (subsubsection 1.4.4), and its motion obeys the body-frame Euler equation (167). Modelling assemblies – a vehicle with wheels, a robot arm, a pendulum chain – requires composing many such bodies, each described *relative* to another. The library provides this as a strict three-layer stack of classes, all sharing one geometry, a *tree of coordinate frames*. Each layer inherits the one below and adds exactly one new kind of state:

Layer	**Adds per frame**	**Answers question**
`static_system2dp`	a pose (relative to parent)	*Where* is each frame?
`kinematic_system2dp`	a velocity + acceleration twist	*How* is it moving? (no forces)
`dynamic_system2dp`	inertia + joints + gravity	How do *forces* move it?

Table 28: The three-layer system stack. Each layer inherits the previous one and adds a single kind of per-frame state.

The layering keeps each capability additive and independently testable: the transformation algebra at the static layer, velocity and acceleration transport at the kinematic layer, and energy conservation at the dynamic layer. A user who only needs coordinate transforms never pays for the dynamics. The same three classes exist for $3D$ (`static_system3dp`, `kinematic_system3dp`, `dynamic_system3dp`); their class structure, method names and algorithms are identical – only the representations change, as summarized at the end of this subsection. The tree describes *open* chains; closed kinematic loops (parallel mechanisms) are added by a separate layer on top of it, developed in subsection 1.4.10.

1.4.7 Pose, frame and the frame tree

A *pose* is the rigid placement of one frame relative to another: an origin and an orientation (in $2D$ a position vector and a scalar angle φ; in $3D$ a position and an axis-angle vector). It is the human-readable form of a *motor* M, and the two interconvert: the motor is built from the pose as in eqn. (114), and the pose is read back by decoding the origin from $M \vee O \vee \underset{\sim}{M}$ and the orientation from the rotated $\boldsymbol{e}_1$-axis (a constrained motor logarithm, exact on round-trip).

A `static_system2dp` holds a *tree* of frames, each recording its parent, with the invariant that a frame's pose is always stored relative to its parent, never

in world coordinates. The world (root) frame is its own parent. Adding a frame without naming a parent links it to the previously added one, so a plain sequence builds a linear chain $root \to f_1 \to f_2 \to \ldots$, while naming an earlier frame branches the tree. This provides three modelling capabilities: *branching* (a body frame with two arms, each arm chain parented off the body, the body off the possibly-moving world – any sub-assembly is a subtree); *name addressing* (frames carry unique names and every operation has a by-name overload); and *reposing* (moving a frame relative to its parent drags the whole subtree along automatically, because all descendant poses are relative).[11]

Tree walk: The transform from any frame to any other is composed along the tree path. Walking *up* from the source to the lowest common ancestor (LCA) composes child-to-parent steps (the regressive reverse of each frame's pose motor); walking *down* from the LCA to the target composes parent-to-child steps; the result is the regressive product of the two segments. Special cases fall out of the same procedure: a frame to itself yields the identity (the unit pseudoscalar, neutral for $\vee$); a pure ancestor/descendant relation makes one segment empty (a forward or backward transform); a linear chain is the degenerate tree where the LCA is simply the shallower frame. Because composition is motor multiplication with $\vee$, the entire tree machinery is dimension-neutral and lifts to $3D$ unchanged.

One sandwich for all objects: The transform between two frames is a single motor M, and *any* geometric object is moved by the *same* sandwich $X' = M \vee X \vee \underset{\sim}{M}$. Classical mechanics, by contrast, provides several different rules and the modeller must determine which one applies:

- a *point* (a location) transforms *affinely* – rotate *and* translate;
- a *free vector* (a direction, a velocity) transforms *linearly* – rotate only, no translation;
- *basis vectors* transform covariantly while a vector's *coordinates* transform contravariantly (the inverse-transpose), the source of much index bookkeeping.

[11] The frame tree is exercised end-to-end in the visualization tool `ga_view`: the frame-transformation scene (`ga_view/src/active_frame_trafo`) drives a `static_system2dp` directly, and the merry-go-round scene (`ga_view/src/active_merry_go_round`) carries the tree one layer up as a rotating `kinematic_system2dp`. Assertion-checked usage is in the test cases `ga_test/src/ga_appl2dp_mechanics_test.hpp` and its $3D$ analogue `ga_appl3dp_mechanics_test.hpp`.

In *PGA* there is a single operation, and the object itself carries the information that selects the right behaviour through its *homogeneous weight* (the projective component $\boldsymbol{e}_3$ in $2D$, $\boldsymbol{e}_4$ in $3D$): a point has weight 1 and is rotated *and* translated; a direction has weight 0 (a point at infinity) and is rotated *only*, since translating a point at infinity does nothing. No contravariant/covariant distinction remains: the motor sandwich is a single conjugation that transports every grade (points, lines, directions, and – see below – twists and wrenches) correctly. The choice of transformation law is thus handled by the algebra itself.

1.4.8 Transport of motion along a kinematic chain

The preceding subsections transport the twist of a *single* body between its body and world frames (eqn. (118a), eqn. (119)). The `kinematic_system2dp` generalizes this to a *chain* of frames: each frame carries a relative velocity twist and a relative acceleration twist with respect to its parent, and the layer answers velocity and acceleration queries for any point of any frame.

Adjoint: A twist B (a vector in $2D$, a bivector in $3D$) is the instantaneous motion (Lie-algebra) with the motor M defined as $M = \exp_{\vee}(\frac{1}{2}B)$ (twists *add*, motors *multiply*; see subsection 1.3.3). Re-expressing a twist B in another frame is done by the *adjoint* of the twist, and the geometric-algebra fact that makes the implementation compact is that the adjoint *is* the very sandwich used to move a point (or any other object):

$$\mathrm{Ad}_M(B) = M \vee B \vee \underset{\sim}{M} \tag{205}$$

This is eqn. (118a) read as a twist transport rather than an object transport. So the same sandwich serves double duty – point action and twist adjoint – because both are the conjugation $M \vee (\cdot) \vee \underset{\sim}{M}$. A separate 6×6 adjoint matrix, as the classical formulation requires, is unnecessary.

Velocity composition: This step goes beyond the single-body case. In a chain, each frame k contributes a relative twist ξ_k expressed in *its own* frame. Transporting each to the world frame by the adjoint and superposing – valid because twists live in a vector space – gives the world velocity twist of frame i:

$$V_i = \sum_{k \in \mathit{path}(\mathit{root} \to i)} \mathrm{Ad}_{M_k}(\xi_k) = \sum_k M_k \vee \xi_k \vee \underset{\sim}{M_k} \tag{206}$$

They add rather than compose precisely because they are infinitesimal: finite motions do not commute and must be multiplied, but to first order simultaneous infinitesimal motions sum. Equation (206) is the geometric-algebra

form of the robotics *spatial Jacobian* – a quantity a single moving body does not exhibit.

Velocity and acceleration fields: Given a frame's world velocity twist V (and acceleration twist A), the velocity of any point X of that frame is the regressive commutator $\dot{X} = \mathrm{rcmt}(V, X)$, and its acceleration is $\mathrm{rcmt}(A, X) + \mathrm{rcmt}(V, \mathrm{rcmt}(V, X))$ – exactly the frame (Euler) and centripetal terms of subsection 1.4.5, now evaluated per chain frame. Differentiating the adjoint along the moving chain yields the recursive Newton-Euler propagation

$$V_i = V_{parent} + \mathrm{Ad}_{M_i}(\xi_i) \tag{207a}$$
$$A_i = A_{parent} + \mathrm{Ad}_{M_i}(\dot{\xi}_i) + \left[\, V_i,\ \mathrm{Ad}_{M_i}(\xi_i) \,\right] \tag{207b}$$

where the second acceleration term – the Lie bracket of twists – is the Coriolis/centrifugal coupling, measuring how far two twists fail to commute. In this primal PGA that bracket is again a native product, $[A, B] = \mathrm{rcmt}(A, B)$ (the $\mathfrak{se}(2)/\mathfrak{se}(3)$ twist bracket). The single-frame transport theorem is the content of subsection 1.4.5; the recursion over a chain is what the kinematic layer adds.
The velocity/finite distinction underlying this – twists add in the Lie algebra while motors multiply non-commutatively, equal only when the generators commute (Chasles' theorem, the screw) – is developed in subsection 1.3.4. It is also the reason the parent-to-child pose of a frame is built by *composing* a rotation and a translation motor rather than exponentiating their summed generators.

1.4.9 Articulated coupled dynamics

The earlier subsections solve a *single* free rigid body (eqn. (167)) and a *single* revolute pivot (eqn. (180), eqn. (179)). The `dynamic_system2dp` adds the coupled dynamics: *several bodies coupled by joints*, integrated together in *reduced (joint) coordinates*.[12]

Spatial pairing and energy: Each body carries the inertia map I of subsection 1.4.4 (velocity twist → momentum wrench $P = I[\Omega]$). A twist

[12]The canonical example is the double pendulum – two revolute-jointed plates swinging chaotically under gravity while conserving energy. It is visualized in `ga_view` (`ga_view/src/active_double_pendulum`, a `dynamic_system2dp` advanced via `step(dt)`) and pinned by assertion-checked tests: the planar case in `ga_test/src/ga_appl2dp_mechanics_test.hpp` and the non-parallel-axes $3D$ case in `ga_appl3dp_mechanics_test.hpp`. Both check energy conservation to RK4 tolerance and the mass-matrix-versus-kinetic-energy identity.

is paired with a wrench through the dimension-agnostic *reciprocal (spatial) product*

$$\langle \xi, P \rangle = -\operatorname{rwdg}(\xi, P), \qquad \langle \xi, I[\xi] \rangle = 2T \tag{208}$$

i.e. the inertia-map quadratic form equals twice the kinetic energy T, carrying the mass and the angular terms uniformly with no $\frac{1}{2}mv^2 + \frac{1}{2}I\omega^2$ split. This one pairing drives the kinetic energy, the joint-space mass matrix and the forward dynamics, and lifts to $3D$ by replacing the 3×3 map with the 6×6 inertia map and rwdg with the bivector reciprocal product.

Joints: A joint is the reduced coordinate linking a body to its parent: *free* (the unconstrained body), *revolute* (a 1-DOF hinge) or *prismatic* (a 1-DOF slider). Both 1-DOF joints run identical code. The body-to-parent motor, the relative twist and the Jacobian column are always

$$M(q) = M_{rest} \vee \exp_{\vee}\!\left(\tfrac{1}{2}\, q\, S_b\right), \qquad \xi = \dot{q}\, S_b, \qquad v(\cdot) = \operatorname{rcmt}(S_b, \cdot) \tag{209}$$

and only the *screw generator* S_b differs: a *finite point* Q_b for a revolute joint (so $\exp_{\vee}$ is a rotation about Q_b) versus an *ideal point*/direction for a prismatic joint (so $\exp_{\vee}$ is a translation). The earlier single-pivot result (eqn. (180)) is the special case of one revolute joint; the generic screw machinery, the prismatic case, and the parallel-axis (Steiner) inertia emerging automatically from the spatial Jacobian are what the dynamic layer adds.

Forward dynamics: In reduced coordinates the joint itself *is* the constraint, so there are no explicit constraint forces, no drift, and clean energy conservation – at the cost of a small linear solve. By virtual work over the bodies one assembles the joint-space equation of motion $M(q)\,\ddot{q} = \tau(q, \dot{q})$ with

$$M_{jk} = \sum_i \left\langle S_j^{(i)},\ I_i\!\left[\, S_k^{(i)} \,\right] \right\rangle, \qquad S_j^{(i)} = \underset{\sim}{M_i} \vee S_j \vee M_i \tag{210a}$$

$$\tau_j = \sum_i m_i \left(v_{cm,i}(S_j) \cdot g \;-\; v_{cm,i}(S_j) \cdot b_{cm,i} \right) \tag{210b}$$

where the sums run over bodies i that have joint j (respectively k) as an ancestor; S_j is the world joint screw; $S_j^{(i)}$ transports it into body i's frame, where its inertia map lives; $v_{cm,i}(S_j) = \operatorname{rcmt}(S_j, C_i)$ is the centre-of-mass velocity per unit joint rate (a column of the spatial Jacobian); $b_{cm,i}$ is the velocity-product (Coriolis/centripetal) cm-acceleration evaluated at $\ddot{q} = 0$; and g is gravity. The system is solved for $\ddot{q}$ by a small LU factorization. Each body still individually obeys the Euler equation (167); the mass matrix is its projection onto the joint coordinates, and the identity $\frac{1}{2}\,\dot{q}^{\top} M(q)\,\dot{q} = T$

ties the two together (a built-in consistency check). In $3D$ the right-hand side carries the full spatial bias wrench $I[A_{bias}] + \mathrm{rcmt}(V, I[V])$ – whose second term is the gyroscopic torque – which vanishes in the planar case to recover eqn. (210b).
The system is advanced in time by a Runge-Kutta integrator (RK4): a coupled joint chain is integrated together in its reduced coordinates, free bodies independently. Energy conservation to integrator tolerance over the motion – validated for the free body, the compound pendulum, the prismatic slider and the chaotic double pendulum, in both $2D$ and $3D$ – is the correctness metric used throughout.

1.4.10 Closed-loop and parallel mechanisms

The frame tree of subsection 1.4.6 reaches every frame by a *unique* path from the root; that uniqueness is what makes the reduced (joint) coordinates well-defined. A closed kinematic chain breaks the premise: a four-bar linkage, a parallelogram, a Stewart-Gough platform each contain a frame reached through several branches at once, so no single parent assignment describes it. Such loops are admitted without giving up the energy-clean reduced coordinates by keeping the tree as a *spanning tree* and re-closing each loop with an explicit constraint between two of its frames – *coordinate partitioning* [Wehage and Haug, 1982], not the maximal coordinates weighed in subsection 1.4.12. The open-chain assembly is reused unchanged; the loop closure is an additive layer on top of it [Featherstone, 2008].[13]

Loop-closure constraints: A loop is closed by requiring two anchor points – P_a carried by one branch, P_b by another – to coincide. In PGA coincidence is the vanishing of the join $P_a \vee P_b$, the line through the two points; realized in coordinates as the unitized point difference

$$g(q) = \hat{P}_a(q) - \hat{P}_b(q) = 0, \tag{211}$$

m scalar equations per loop ($m = 2$ in $2D$, $m = 3$ in $3D$). This is the spatial spherical (ball) joint that pins a Stewart-Gough leg to its platform, or the pin that closes a planar four-bar.

[13] The distinction is shown side by side in `ga_view` (`ga_view/src/active_open_vs_closed`): the same three-link arm under one shoulder drive, swinging freely as an open chain, and – with its hand pinned by a single coincidence constraint – re-solving its remaining joints to keep the hand planted.

Constraint Jacobian: Differentiating eqn. (211) along the motion gives $\dot{g} = G(q)\,\dot{q}$ with $G = \partial g/\partial q$. Each column of G is the relative partial velocity of the two anchors,

$$G_{cj} = \frac{\partial(P_a - P_b)}{\partial q_j} = \mathrm{rcmt}(S_j, P_a) - \mathrm{rcmt}(S_j, P_b), \tag{212}$$

each term present only when joint j lies on the path to that anchor. But $v(\cdot) = \mathrm{rcmt}(S_j, \cdot)$ is the spatial-Jacobian column already built for the mass matrix and the bias force (eqn. (210a), eqn. (207)). The loop closure introduces no new geometry: the same joint screws S_j that assemble M and τ assemble G.

Constrained equations of motion: The tree dynamics given by the equation $M(q)\,\ddot{q} = \tau$ (eqn. (210a)) gain the m constraints $g(q) = 0$. Differentiated twice they read $G\,\ddot{q} = -\dot{G}\,\dot{q}$; adjoining the constraint forces $G^{\mathsf{T}}\lambda$ gives the bordered (KKT) system solved each step,

$$\begin{bmatrix} M & G^{\mathsf{T}} \\ G & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} \tau \\ -\dot{G}\,\dot{q} \end{bmatrix}. \tag{213}$$

The Lagrange multipliers λ are the constraint wrenches – the leg forces. M and τ are reused unchanged from the open-chain assembly (eqn. (210a)–(210b)); only G and the velocity-product term $\dot{G}\,\dot{q}$ – the relative anchor acceleration evaluated at $\ddot{q} = 0$, read from the same bias pass that yields b_{cm} – are new. The one constraint, differentiated zero, once and twice, supplies the three tiers of solve: the position assembly $g(q) = 0$ by Newton (the dependent joint angles from a driven one), the velocity distribution $G\,\dot{q} = 0$, and the acceleration distribution $G\,\ddot{q} = -\dot{G}\,\dot{q}$.

Stabilisation and energy: Discretely integrated, the position and velocity constraints drift. They are restored after each step by projection onto the constraint manifold – a few Gauss-Newton corrections $q \leftarrow q - G^{+}g$ and $\dot{q} \leftarrow \dot{q} - G^{+}(G\,\dot{q})$ (the Gear-Gupta-Leimkuhler scheme [Gear et al., 1985]). Because the constraint forces $G^{\mathsf{T}}\lambda$ are orthogonal to the feasible velocities ($G\,\dot{q} = 0$) they do no work, and the projection corrects drift without injecting energy. The energy-conservation property of the reduced coordinates (subsection 1.4.12) therefore survives the loop closure – the objection that ruled out maximal coordinates, whose continuous stabilization trades energy for feasibility, does not bind on the few constraints a spanning-tree closure adds. Baumgarte stabilization [Baumgarte, 1972] (a term $2\alpha\,G\dot{q} + \beta^2 g$ on the right-hand side of eqn. (213)) is the simpler alternative, kept as a fallback.

The four-bar linkage: The planar four-bar is the spanning tree crank → coupler, with the rocker a second branch from the ground, closed by one coincidence of the coupler tip with the rocker tip: three revolute joints, two equations, one degree of freedom.[14] Driving the crank, the assembly solves the coupler and rocker angles from eqn. (211); the velocity and acceleration ratios follow from $G\,\dot{q} = 0$ and $G\,\ddot{q} = -\dot{G}\,\dot{q}$ and reproduce the classical four-bar relations. Released under gravity the single degree of freedom swings, with total energy conserved to integrator tolerance while $\|g\|$ stays bounded – the same metric the open-chain dynamics use.

The lift to $3D$: As for the open-chain tier, the step to $3D$ is a change of representation, not of method. The coincidence residual carries three components instead of two, the joint screws become lines (`twist3dp`, a bivector) and the anchor points carry the weight $\boldsymbol{e}_4$; eqn. (211)–(213) and the projection are unchanged, and the bordered solve is dimension-agnostic. A spatial loop – two arms meeting at a shared spherical joint, the kinematic core of a Stewart-Gough or delta platform[15] – then assembles, distributes its velocities, and conserves energy under the same machinery (`closed_loop_system3dp`).

1.4.11 The transition from $2D$ to $3D$

Everything above is presented for $2D$ PGA (`pga2dp`); the $3D$ stack (`pga3dp`) is the same classes, methods and algorithms, with only the representations changed:

[14] It is visualized in `ga_view` (`ga_view/src/active_four_bar`, a motor-driven `closed_loop_system2dp` whose coupler point traces the classic coupler curve) and pinned by assertion-checked tests: the planar four-bar in `ga_test/src/ga_appl2dp_mechanics_test.hpp` and a spatial two-arm loop ($3D$) in `ga_appl3dp_mechanics_test.hpp` – assembly, velocity and acceleration ratios, energy conservation across a long run, and a bounded closure error $\|g\|$.

[15] The planar analogue is realized in `ga_view` (`ga_view/src/active_planar_delta`): a 5-bar (2-RRR) parallel manipulator – two two-link arms rising from two driven shoulders, their tips pinned to a shared end-effector by one coincidence (four revolute joints, two equations, two degrees of freedom). The two shoulders are driven by phase-shifted sinusoids; each frame re-solves the elbows from the loop closure (forward kinematics), so the effector traces a Lissajous curve. The same well-conditioning is pinned in `ga_test/src/ga_appl2dp_mechanics_test.hpp`; a true $3D$ delta awaits $3D$ rendering in `ga_view`.

Concept	`pga2dp`	`pga3dp`
motor (pose)	even/odd versor `mvec2dp_u`	versor `mvec3dp_e`
pose orientation	scalar angle φ	axis-angle vector
velocity twist	a vector `Vec2dp` (grade 1)	a bivector `BiVec3dp` (grade 2)
point	vector, weight $\boldsymbol{e}_3 = 1$	vector, weight $\boldsymbol{e}_4 = 1$
joint screw	a finite/ideal point	a line / screw axis (a bivector)
inertia map	3×3	6×6
twist bracket	rcmt(vec, vec)	rcmt($bivec, bivec$)
spatial pairing	$-$rwdg($vec, bivec$)	$-$rwdg($bivec, bivec$)

Table 29: Representation changes in the $2D \to 3D$ lift of the system layer. The motor sandwich, the adjoint, the tree walk, the joint exponential, the inertia-map pairing and the Euler equation are *identical formulae* in both dimensions.

The motor sandwich, the adjoint, the tree walk, the joint exponential, the inertia-map pairing and the Euler equation $\dot{\Omega} = I^{-1}[\, W - \mathrm{rcmt}(\Omega, I[\Omega])\,]$ are identical formulae in $2D$ and $3D$, differing only in the grade the twist and wrench occupy. The $2D \to 3D$ step is thus a change of representation, not a rewrite; this follows from developing the $2D$ case fully first. The same holds for the closed-loop layer of subsection 1.4.10: the loop-closure residual gains a component and the joint screws become lines, while the constraint Jacobian and the bordered solve are unchanged.

1.4.12 Addendum: what geometric algebra contributes to rigid-body dynamics

This addendum weighs the approach against classical methods. It is not part of the modelling exposition; readers interested only in *how* the model works may skip it. It states where the PGA formulation is advantageous and where a classical method remains preferable. This library realizes the *primal* PGA ($\mathbb{G}(2,0,1)$, $\mathbb{G}(3,0,1)$, see subsection 1.4); the merits below are those of the geometric formulation as such and hold for the dual plane-based variant too. The fair baseline is not "geometric algebra versus nothing" but geometric algebra versus the standard tooling: 4×4 homogeneous $SE(3)$ matrices, or Featherstone's spatial vector algebra (six-component twists and wrenches with a hand-defined spatial cross product) [Featherstone, 2008]. Both are mature and fast; the question is whether the reorganization is worthwhile.

Properties of the geometric formulation:

1. *Pose is a motor, not a* 4×4 *matrix.* There is no orthonormality constraint to maintain and no renormalization drift; one exponential/logarithm covers rotation and translation together as a screw, and the pose evolves on the manifold exactly via $M \vee \exp_{\vee}(\frac{1}{2}\,\Omega\,dt)$.

2. *Twists, momenta and wrenches are bivectors, and the "spatial cross product" is the algebra's commutator.* A twist is a screw, the momentum wrench is the inertia-map image $P = I[\Omega]$, the Newton-Euler bias wrench is a bivector, and the spatial cross product is rcmt – a native product rather than a bespoke operator with conventional signs.

3. *The adjoint equals the point sandwich.* Transporting a twist between frames is the same operation as moving a point (eqn. (205)); the classical formulation needs a separate 6×6 adjoint matrix.

4. *A force is its line of action – a wrench.* The combined force-and-moment bivector $F = Q \wedge f$ (subsection 1.4.1) is the object the mechanics literature calls a *wrench*; moments are not a separate pseudovector built with an $r \times F$ cross product.

5. *The Euler equation is one dimension-agnostic line* (eqn. (167)), identical in $2D$ and $3D$ up to grade; the $2D \to 3D$ lift is the same formula, not a rewrite.

Coriolis coupling: The most involved part of a classical double-pendulum Lagrangian is the Coriolis/centrifugal coupling – the Christoffel symbols built from derivatives of the mass matrix. In the geometric formulation it collapses to one term, the Lie bracket of twists in the acceleration recursion (207): the Coriolis force is the geometric fact that twists do not commute. These objects are also drawable in a way six-component vectors are not: each body's instantaneous centre of rotation is a point read off its world twist, the joint screw is the joint point, and the momentum wrench is a bivector. The dynamical state is itself geometry, so the visualization is not a separate construction.

Limitations: Inertia remains a linear operator (3×3 in $2D$, 6×6 in $3D$); for an anisotropic body it does not collapse to a single multivector. The joint-space mass matrix $M(q)$ is still a small numerical solve – reducing to generalized coordinates is a standard robotics step, and the final $\ddot{q} = M^{-1}\tau$ is ordinary linear algebra. Performance is not improved either: mature spatial-vector and matrix codes are highly optimized, so the geometric formulation changes the representation, not the run time.

Reduced vs. maximal coordinates: A more GA-native formulation was deliberately not used: maximal coordinates, in which every body stays a full motor-and-twist and joints become constraint wrenches. That formulation is more uniform, but its constraint stabilization injects and removes energy, which conflicts with an energy-conservation test. The reduced (joint) coordinates chosen here make the joint itself the constraint, so there are no explicit constraint forces, no drift, and clean energy – at the cost of the small $M(q)$ solve. This retains most of the modelling value while conserving energy. The closed-loop extension (subsection 1.4.10) keeps the same choice: a reduced-coordinate spanning tree plus a few explicit loop constraints, rather than maximal coordinates throughout, so its constraint forces do no work and the energy behaviour is preserved.
The recursive Newton-Euler *algorithm* is, of course, classical and not original here. What geometric algebra contributes is the *representation and the concepts* – screws, motors, bivectors and the commutator in place of matrices, six-vectors and hand-defined operators – together with three properties the standard tooling does not provide directly: the same primitives scale unchanged from one body to a coupled tree; the state is visualizable geometry; and the $2D \to 3D$ transition is a change of representation rather than a rewrite.

1.4.13 Addendum: further reading

The concepts underlying this layer – motors, the sandwich, the adjoint, twists as a Lie algebra, the screw unification of rotation and translation, the inertia map, the Coriolis commutator – are each standard results with a primary source. The references below progress from the algebra itself, through the plane-based specialization of PGA, to rigid-body mechanics, and finally to the robotics / Lie-theory and classical viewpoints that the geometric formulation translates.
A standing caveat applies to every PGA source: this library uses the *primal* $\mathbb{G}(2,0,1)/\mathbb{G}(3,0,1)$ (Lengyel's convention – points and lines as primary objects, motions carried by the regressive product), whereas Gunn and Dorst & De Keninck use the *dual* plane-based $\mathbb{G}^*$ (planes primary, motions carried by the geometric product). The two are dual-equivalent (subsection 1.4); their *concepts* carry over directly, but their *formulae* are the dual of those used here and must be translated through the complement relation before being matched against the code.

- **Geometric algebra foundations.**

  - [Macdonald, 2010] – a gentle, rigorous first contact with the geometric product, blades and rotors.
  - [Hestenes, 1986] – the origin of doing mechanics in geometric algebra; its chapter on rigid-body mechanics already points toward *PGA*.
  - [Browne, 2012] – the exterior-algebra background for the wedge and regressive products, complements, and the force and momentum lines.
  - [Perwass, 2009] – a broad engineering-oriented treatment of geometric algebra, covering conformal geometric algebra, numerical and uncertainty aspects, and the interactive visualization tool CLUViz.

- **Plane-based *PGA* and the primal/dual choice.**
  - [Lengyel, 2024b] – the convention this library follows (rationale in [Lengyel, 2024a] and [Lengyel, 2025]).
  - [Gunn, 2020] and [Dorst and De Keninck, 2022] – the foundational and the most accessible plane-based (dual) treatments; read for structure, dualize the formulae.

- **Rigid-body mechanics in *PGA*.**
  - [Dorst and De Keninck, 2023] – the direct (dual) analogue of the dynamics developed here: momentum bivector, the "forque", the motor equation of motion.
  - [Gunn, 2011] – the most complete treatment of rigid-body kinematics and dynamics with motors and bivector twists in (dual) projective geometric algebra.

- **The Lie-theory / robotics translation** (the same groups $SE(2)/SE(3)$ as matrices).
  - [Solà et al., 2018] – the most compact modern account of the group/algebra correspondence, the adjoint and the bracket.
  - [Lynch and Park, 2017] – the clearest intuition for twists, the adjoint and the velocity/acceleration recursion.
  - [Murray et al., 1994] – the canonical thorough treatment.
  - [Selig, 2005] – bridges classical screw theory and the Lie-group view.

- **The classical baseline.**
  - [Featherstone, 2008] – spatial vector algebra: six-component twists and wrenches and the spatial cross product, the Plücker-coordinate cousin of the bivector twists and rcmt used here, and the reference for the forward-dynamics algorithm.
- **Constrained multibody dynamics** (the closed-loop layer of subsection 1.4.10).
  - [Wehage and Haug, 1982] – coordinate partitioning: the split into a reduced (spanning-tree) set and the dependent coordinates fixed by the loop constraints.
  - [Gear et al., 1985] – the projection (GGL) scheme that restores the position and velocity constraints after each step without injecting energy.
  - [Baumgarte, 1972] – the simpler constraint stabilization kept as a fallback.

1.5 Approaches with increasing expressiveness

1.5.1 Classic approach vector algebra (VA)

- Use of scalars and vectors, exclusively
- Use of the cross-product between vectors to create directions perpendicular to both vectors (a concept that can only be defined in three-dimensional space)
- Rotations are defined by a rotation axis (a concept that is only defined in three-dimensional space)[16]

1.5.2 General observations for Geometric Algebra (GA)

- The GA approach is coordinate-free!
- W.r.t. a vector, the dot product (or interior product) takes into account the parts parallel to the vector, exclusively. Thus, it is not invertible itself.
- W.r.t. a vector, the wedge product (or outer product) takes into account the parts perpendicular to the vector, exclusively. Thus, it is not invertible itself.
- Only the geometric product taking both parts into account is invertible.

1.5.3 Euclidean geometric algebra (EGA)

- Use of scalars, vectors, bivectors, ..., pseudoscalars (higher grade objects up to order n depend on the dimension n of the modelled space).
- Use of the outer product (wedge product) between vectors, which defines the space spanned by both vectors (can be defined in any dimension, applied as join operation).
- Use of the regressive outer product (regressive wedge product) which defines the space shared by both objects (represents the intersection of the spaces used by the objects, applied as meet operation).
- Modelled objects cover scalars, directions, planes (in $3D$) and the whole space directly.

[16] The last two concepts cannot be extended to higher-dimensional spaces.

- All objects modelled in the algebra contain the (non-distinguishable) origin $O = (0, 0, ..., 0)$. Objects outside the origin must be modeled by explicitly adding translations (leading to objects that cannot directly be manipulated as easily as objects that directly represent subspaces within the algebra).
- Orthogonal transformations in the algebra can directly model reflections and rotations. Rotations are defined in a plane spanned by two vectors defining the rotation.

1.5.4 Projective (Euclidean) geometric algebra (PGA)

- Like EGA, but in addition having a distinguishable origin encoded in an additional dimension $O = (0, 0, ..., 0, 1)$ (in a projective space with an additional dimension projected onto a fixed value, typically 1).
- Use of the outer and regressive outer products as join and meet operations.
- Directly models points, directions and planes (in $3D$), as well as the whole space, including objects located in the horizon, i.e. at infinity (points at infinity are modeled as directions towards those points).
- Can directly model higher grade objects by products between objects in the algebra that do not contain the origin, e.g. the wedge product between two arbitrary points defines a line.
- Orthogonal transformations in the algebra can directly model reflections, rotations and translations.

1.5.5 Space-time algebra (STA)

- Extends the Euclidean approach to space-time: $\mathbb{G}(1, 3, 0)$ with three space-like and one time-like basis vector (Minkowski signature).
- Bivectors carry both kinds of transformation generators: space-like planes generate rotations, time-like planes generate Lorentz boosts. The even-grade rotors implement both through the same sandwich product as in EGA.
- Quantities that are separate objects in $3D$ combine into single space-time objects (events and four-velocities as vectors; the electric and magnetic fields as one field bivector), and frame changes act on all of them by rotor conjugation.

- Suitable when time is part of the modelled physics itself: fields of moving sources, induction, waves, invariance arguments. For static and quasi-static problems EGA with time as an external parameter remains sufficient.

1.5.6 Choosing the algebra

The algebras are layers with distinct responsibilities rather than competing formalisms. EGA describes the *value* of a physical quantity at a point: its metric is invertible, which provides norms, angles and gradients, and with them field calculus. Its objects carry no position – the evaluation point is a parameter, not part of the object. PGA describes *located* objects – points, lines of action, planes, poses – and rigid motion; its degenerate metric intentionally gives up field calculus to gain translations as sandwich products. STA describes quantities whose physics couples space and time.

A quantity therefore belongs in EGA when its position is merely where it is evaluated (a field value, an energy density), and in PGA when its position is part of what it is (a force on its line of action, the pose of a body). The two connect through the ideal sector: an EGA direction embeds as an ideal point, an EGA bivector as an ideal line (a couple), so a local result computed in EGA is bound to its location by a single wedge product – e.g. $P \wedge f$ turns a force vector evaluated at a point into the wrench acting on that point's line of action. STA enters when time can no longer stay an external parameter.

In short: EGA answers *what* – the value of a quantity at a point. PGA answers *where* – location and rigid motion. STA answers *when* – coupling across space and time.

1.6 Glossary

Pose

- A pose is the rigid placement of one frame relative to another: an origin and an orientation (in $2D$ a position and a scalar angle φ; in $3D$ a position and an axis-angle vector).
- It is a point of the rigid-motion group, carried by a unit motor M; the origin/orientation form and the motor interconvert exactly (see 1.4.7).
- A pose is a finite placement (where a frame is), as opposed to a twist (how it is moving).

Motor and rotor

- In PGA a motor M performs a rigid motion by the sandwich product $X' = M \vee X \vee \underset{\sim}{M}$, applied identically to every object: points, lines, twists, wrenches.
- It is the exponential of a twist, $M = \exp_{\vee}(\frac{1}{2}B)$ (see eqn. (114)); motors multiply (compose) where their generators add.
- A unit motor in $3D$ PGA encodes a screw motion (rotation about and translation along an axis), so one motor covers rotation, translation and their combination.
- The EGA counterpart, built from the geometric product $\wedge$ instead of the regressive $\vee$ and acting only through the origin, is the rotor (rotations only).

Screw (Chasles' theorem)

- By Chasles' theorem every rigid motion is a screw: a rotation about an axis combined with a translation along it.
- In PGA the screw axis is a line (a bivector in $3D$); the central quantities are all screws on such an axis: the twist (a velocity screw), the wrench (a force screw), and the motor (the finite screw motion $\exp_{\vee}$ of a twist).
- A pure rotation and a pure translation are the degenerate screws of zero and infinite pitch.

Twist

- A twist B is the instantaneous motion of a frame: the Lie-algebra generator that builds its motor through $M = \exp_{\vee}(\frac{1}{2}B)$ (see eqn. (114)). Its grade is dimension-dependent: a vector in $2D$ PGA, a bivector in $3D$ PGA.
- The rate Ω is the time derivative of the generator, $\Omega = \dot{B} = dB/dt$. For a constant rate the generator grows linearly, $B(t) = \Omega\, t$, so the motor is $M(t) = \exp_{\vee}(\frac{1}{2}\Omega\, t)$; conversely the rate is recovered from a moving motor by $\Omega = 2\,\dot{M} \vee \underset{\sim}{M}$ (see eqn. (116)).
- Twists add (they live in a vector space) where motors multiply.
- A twist is a velocity screw; it induces the point velocity $\dot{X} = \mathrm{rcmt}(\Omega, X)$.

Pose vs. twist

- Pose is the group element (where a frame is, the motor M); twist is the tangent/rate (how it moves, the generator B with rate $\Omega = \dot{B}$).
- They interconvert by integration and differentiation: $M(t) = M_0 \vee \exp_{\vee}(\frac{1}{2}\int \Omega\, dt)$ and conversely $\Omega = 2\,\dot{M} \vee \underset{\sim}{M}$.
- Pose lives in the Lie group, twist in the Lie algebra (see 1.3.4); cf. the entry on infinitesimal vs. finite movements.

Lie group vs. Lie algebra

- The motors form a Lie group: the manifold of finite rigid motions, composed by $\vee$.
- The twists form its Lie algebra: the tangent space at the identity, a vector space closed under the bracket $[A, B] = \mathrm{rcmt}(A, B)$.
- The two are linked by $\exp_{\vee}$ (algebra to group) and its logarithm (group to algebra); twists add and commute to first order, motors multiply and do not (see Sec. 1.3.4).

Infinitesimal vs. finite movements

- An infinitesimal movement is a twist (a Lie-algebra generator); a finite movement is the motor it integrates to (a Lie-group element).
- Simultaneous infinitesimal motions superpose (twists add), but the finite motors they generate do not commute and must be multiplied in order.

- This is why a parent-to-child pose is built by composing a rotation and a translation motor, not by exponentiating their summed generators (see 1.3.4).

Adjoint

- The adjoint Ad_M re-expresses a twist in another frame.
- In *PGA* it is the same sandwich that moves any object, $\mathrm{Ad}_M(B) = M \vee B \vee \underset{\sim}{M}$ (see eqn. (205)).
- Point action and twist transport are thus one operation; no separate 6×6 adjoint matrix is needed (see 1.4.8).

Commutator (Lie bracket)

- The Lie bracket (commutator) of two twists measures how far they fail to commute.
- In *PGA* it is a native product, $[A, B] = \mathrm{rcmt}(A, B)$: the $\mathfrak{se}(2)/\mathfrak{se}(3)$ twist bracket.
- It supplies the Coriolis/centrifugal coupling in the Newton-Euler recursion (see eqn. (207)) and the bias term $\mathrm{rcmt}(\Omega, I[\Omega])$ in the body-frame Euler equation.

Moment and torque

- Torque is the rotational counterpart of linear force: a force applied at a distance from a reference point generates a moment (torque) about that point equal to the force times the perpendicular distance.
- The terms are often used interchangeably; in mechanics, *torque* usually refers to the turning effect in a dynamic system, *moment* in a static one. Both are calculated as force times perpendicular distance from the pivot.
- When modelling mechanics in *PGA*, force and torque do not need to be tracked separately. The line of action $F = P \wedge f$ (eqn. (141)) carries both: the force f itself and the torque it generates about any reference point. The torque at the origin is $bulk(F)$; the free moment at any other point follows from the same object F.

Wrench

- A wrench is a force screw — the bivector encoding a resultant force and the torque it induces, bound to a line of action (the "forque" of 1.4.1). It is the force-space counterpart of the velocity twist: the two pair through the spatial product $\langle \xi, P \rangle = -\operatorname{rwdg}(\xi, P)$ to give power.
- The momentum line of Sec. 1.4.2 is likewise the wrench dual to the velocity twist.
- A twist ξ pairs with a wrench $P = I[\xi]$ through the spatial product to give twice the kinetic energy (see eqn. (208)).

Momentum

- Momentum[17] characterizes the movement of an object: it has a direction (that of the velocity) and a magnitude (mass times speed). A heavier object moving at the same speed has more momentum; momentum changes when either speed or direction changes.
- In *PGA* the momentum of a body is a bivector (a momentum wrench): the image $P = I[\Omega]$ of the velocity twist Ω under the inertia map.
- It combines linear and angular momentum in one object, just as a wrench combines force and torque; the two are recovered as bulk and weight parts of P (see 1.4.2).
- Newton's second law takes the form $\dot{P} = W$ (net wrench equals rate of change of momentum wrench), from which the Euler equation follows on inverting the inertia map.

Inertia map

- The inertia map $I[\cdot]$ is the linear operator mapping a body's velocity twist Ω to its momentum wrench, $P = I[\Omega]$ (see 1.4.4).
- It is a 3×3 map (vector to bivector) in $2D$ and a 6×6 map (bivector to bivector) in $3D$, encoding mass, scalar moment of inertia and the full inertia tensor in one object; the mass contribution is $-m\,\overline{\Omega}$, the complement placing it in the dual block.
- It is additive over the parts of a body and is inverted to set up the Euler equation $\dot{\Omega} = I^{-1}[\, W - \operatorname{rcmt}(\Omega, I[\Omega]) \,]$.

Total thickness variation (TTV)

[17] German: *Impuls*, *Bewegungsgröße* — not to be confused with *Moment* (torque), the entry above.

- The total thickness variation of a wafer is the spread of its thickness over the surface, TTV = $\max t - \min t$: the difference between its thickest and thinnest points, and the principal flatness metric of a thinned wafer.
- In the surface model the thickness is the ground upper surface minus the chuck profile, a difference of two height fields; TTV is the peak-to-peak of that difference.

1.7 Links to get started

Overview to easily get started (high-level motivation and initial introductions on youtube)[18]:

- *https://www.thestrangeloop.com* (Jack Rusher, Funny intro to GA with historical perspective)
- *https://www.youtube.com/@sudgylacmoe* (sudgylacmoe, A swift intro to GA)
- *https://www.youtube.com/@Wildberger* (Norman Wildberger - Projective geometry)
- *https://www.youtube.com/@sudgylacmoe-GAoperations* (sudgylacmoe, Overview of GA ops.)
- *https://www.youtube.com/@AllAngles/GA* (All Angles - GA Intro)
- *https://www.youtube.com/@bivector/GAME23* (De Keninck and Roelfs, GAME 2023 - GA Intro)

Sources for GA and PGA as a mathematical topic:

- *https://projectivegeometricalgebra.org* (Eric Lengyel)
- *https://terathon.com/blog/poor-foundations-ga.html* (Eric Lengyel)
- *https://bivector.net* (bivector home page, Leo Dorst et.al.)
- *https://www.youtube.com/@bivector* (Youtube channel bivector)
- *https://en.wikipedia.org/wiki/Bivector* (Wikipedia Bivector)
- *https://www.youtube.com/@sudgylacmoe* (Youtube channel sudgylacmoe)
- *https://www.youtube.com/@hestenes* (Hestenes, Tutorial Geometric Calculus (relevant formulas))
- *https://en.wikipedia.org/PGA* (Plane-based GA in $R^*(3,0,1)$ instead of $R(3,0,1)$ as used here)
- *https://terathon.com/blog/dual-pga.html* (Lengyel's argumentation for using $R(3,0,1)$)

Sources for GA, PGA and STA in more specific applications:

- *https://www.youtube.com/DynamicsPGA* (Leo Dorst, Dynamics in PGA)
- *https://en.wikipedia.org/wiki/Newton-Euler* (Newton-Euler equations)
- *https://en.wikipedia.org/wiki/Rigidbody* (Rigid body dynamics)
- *https://en.wikipedia.org/wiki/Inertia* (Moment of inertia)
- *https://en.wikipedia.org/wiki/ScrewTheory* (Screw theory, twist and wrench)
- https://en.wikipedia.org/wiki/Chasles (Chasles' theorem)
- http://web.cs.iastate.edu/Pluecker (Plücker coordinates in projective spaces)
- *https://www.youtube.com/@sudgylacmoe-STA* (sudgylacmoe, A swift intro to spacetime algebra)

[18] URLs partly edited for improved readability. Don't use the printed URL directly, but rather click the link to go to the linked target.

1.8 Literature

References

[Baumgarte, 1972] Baumgarte, J. (1972). Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16.

[Browne, 2012] Browne, J. (2012). *Grassmann Algebra*, volume 1: Foundations. Barnard Publishing.

[Dorst and De Keninck, 2023] Dorst, L. and De Keninck, S. (August 15, 2023). May the forque be with you - dynamics in pga. *University of Amsterdam, The Netherlands*, V2.6.

[Dorst and De Keninck, 2022] Dorst, L. and De Keninck, S. (March 14, 2022). A guided tour to the plane-based geometric algebra pga. *University of Amsterdam, The Netherlands*, V2.0.

[Featherstone, 2008] Featherstone, R. (2008). *Rigid Body Dynamics Algorithms*. Springer.

[Gear et al., 1985] Gear, C. W., Gupta, G. K., and Leimkuhler, B. (1985). Automatic integration of Euler-Lagrange equations with constraints. *Journal of Computational and Applied Mathematics*, 12–13:77–90.

[Gunn, 2011] Gunn, C. (2011). *Geometry, Kinematics, and Rigid Body Mechanics in Cayley-Klein Geometries*. PhD thesis, Technische Universität Berlin.

[Gunn, 2020] Gunn, C. G. (August 2020). Projective geometric algebra: A new framework for doing euclidean geometry. *Raum+Gegenraum Falkensee, Germany*.

[Hestenes, 1986] Hestenes, D. (1986). *New foundations for classical mechanics*. D. Reidel publishing company.

[Kritchevsky, 2024] Kritchevsky, A. (February 28, 2024). The case against geometric algebra. *Alex Kritchevsky's blog*.

[Lengyel, 2024b] Lengyel, E. (2024b). *Projective Geometric Algebra Illuminated*. Terathon Software LLC.

[Lengyel, 2024a] Lengyel, E. (August 23, 2024a). Poor Foundations in Geometric Algebra. *Eric Lengyel's blog*.

[Lengyel, 2025] Lengyel, E. (May 23, 2025). The Transwedge Product. *Eric Lengyel's blog*.

[Lynch and Park, 2017] Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press.

[Macdonald, 2010] Macdonald, A. (2010). *Linear and Geometric Algebra*. CreateSpace Independent Publishing Platform.

[Murray et al., 1994] Murray, R. M., Li, Z., and Sastry, S. S. (1994). *A Mathematical Introduction to Robotic Manipulation*. CRC Press.

[Perwass, 2009] Perwass, C. (2009). *Geometric Algebra with Applications in Engineering*, volume 4 of *Geometry and Computing*. Springer Berlin, Heidelberg.

[Selig, 2005] Selig, J. M. (2005). *Geometric Fundamentals of Robotics*. Springer.

[Solà et al., 2018] Solà, J., Deray, J., and Atchuthan, D. (2018). A micro lie theory for state estimation in robotics. *arXiv preprint arXiv:1812.01537*.

[Wehage and Haug, 1982] Wehage, R. A. and Haug, E. J. (1982). Generalized coordinate partitioning for dimension reduction in analysis of constrained dynamic systems. *Journal of Mechanical Design*, 104(1):247–255.